INTRODUCTORY DATA STRUCTURES & ALGORITHMS

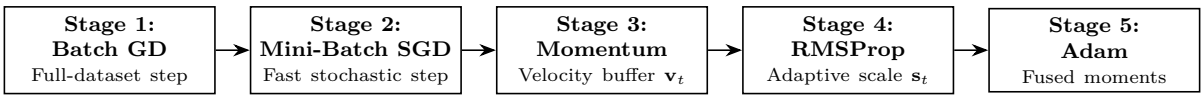
Official Guided Examination Papers (Problems 1–20)

STUDENT EDITION — STANDALONE EXAMINATION HANDOUTS

Examination Structure: The Progressive Algorithmic Ladder

Each examination paper in this collection investigates a foundational problem in computer science and scientific computing. Rather than presenting an optimized algorithm as a finished formula to memorize, each problem guides you through a **5-Stage Progressive Evolution Ladder**: you begin with the simplest baseline implementation, identify its computational or structural bottleneck on a concrete hand-worked instance, and apply a sequence of targeted adjustments to reach a state-of-the-art algorithm or data structure.

A classic illustration of this progression comes from continuous optimization:



Each standalone problem handout introduces all required definitions from first principles, illustrates the data structure mechanics visually, walks through concrete examples, and specifies the written and coding exercises for every stage.

Required Examination Deliverables (Per Assigned Problem)

For your assigned examination problem, you must submit three deliverables:

1. **Handwritten Step-by-Step Traces & Data Structure Explanations:**
   - **Manual Traces on Small Concrete Inputs:** Complete step-by-step hand calculations on the small arrays, matrices, trees, or graphs specified in the questions, showing all intermediate states and pointer/index updates.
   - **Data Structure Mechanics & Bottleneck Explanations:** Clear, plain-English explanations and simple diagrams showing how each stage stores its data in memory and what practical bottleneck of the previous stage it eliminates.
   - **Operation Counts & Big-O Summary Table:** Simple operation counts on concrete small inputs and a summary table comparing the time and space Big-O bounds of all five versions across the progression.
2. **Source Code Implementation & Empirical Benchmark Suite:**
   - Implement **all stages** of the algorithm/data structure progression (from the naive baseline through all intermediate adjustments to the final optimized version).
   - Provide automated unit tests verifying that all versions produce identical/valid outputs on shared test inputs.
   - Benchmark and plot **execution time** and **memory consumption** across increasing input sizes N, demonstrating empirically where each refinement overtakes its predecessor.
3. **Oral Defense Presentation Slides:**
   - A concise slide deck motivating the problem, walking through your hand-worked traces, explaining each step along the improvement ladder, and presenting your empirical runtime and memory plots.

Contents

Problem 1: Matrix Multiplication & Strassen's Algorithm **3**

Problem 2: Pocket Rubik's Cube & State-Space Search **7**

Problem 3: Shortest Paths: From Bellman-Ford & Dijkstra to A* **11**

Problem 4: Transformer Attention, Online Softmax & KV-Caching **15**

Problem 5: Web Ranking, PageRank & Sparse Power Iteration **19**

Problem 6: From Gradient Descent to Momentum, RMSProp & Adam **23**

Problem 7: Polynomial & Big-Integer Multiplication **27**

Problem 8: Lossless Data Compression **31**

Problem 9: Modular Arithmetic & Public-Key Cryptography **35**

Problem 10: Sequence Alignment & Edit Distance **39**

Problem 11: Disjoint-Set Union (Union-Find) & Percolation **43**

Problem 12: Bloom Filters & Count-Min Sketch **47**

Problem 13: Matrix Compression & Low-Rank SVD **50**

Problem 14: 3D Ray Tracing & Bounding Volume Hierarchies **53**

Problem 15: The Traveling Salesperson Problem (TSP) **57**

Problem 16: Substring Search & Pattern Matching **60**

Problem 17: Markov Chain Monte Carlo (MCMC) **64**

Problem 18: Storage Engines: LRU, B-Trees & LSM-Trees **68**

Problem 19: Network Flows & Image Segmentation **72**

Problem 20: Adversarial Game Trees & MCTS **75**

PROBLEM 1: MATRIX MULTIPLICATION & STRASSEN'S ALGORITHM

From Triple Loops to Cache-Friendly Tiling & Strassen's Sub-Cubic Algorithm

Core Data Structures & Concepts: 2D Arrays, Row-Major Memory Layout, Divide-and-Conquer Recursion, Pre-Allocated Workspaces

Algorithmic Progression: V1 (Naive `ijk`) → V2 (Cache-Friendly `ikj`) → V3 (Recursive 2×2 Block Split) → V4 (Strassen's 7-Multiply Trick) → V5 (Hybrid Cutoff n_0)

Why Matrix Multiplication Is the Heartbeat of Modern Computing

Whenever you rotate a 3D video game camera, apply a blur filter to a photograph, simulate weather patterns, or train a neural network, the computer spends over 90% of its time performing one fundamental operation: **multiplying two matrices** $C = A \times B$.

Suppose A and B are square tables (2D arrays) of size $n \times n$. By definition, the entry in row i and column j of the product matrix C is the **dot product** of row i of A with column j of B:

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \cdot B[k][j] = A[i][0] \cdot B[0][j] + A[i][1] \cdot B[1][j] + \cdots + A[i][n-1] \cdot B[n-1][j].$$

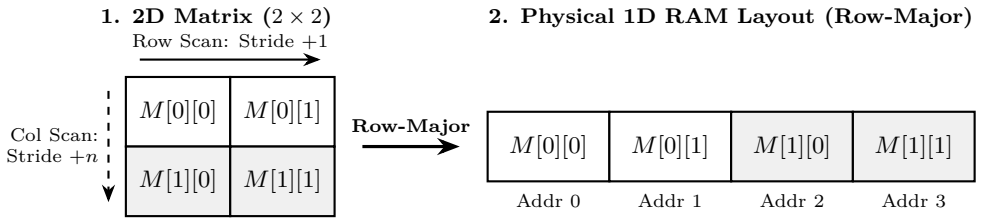
For a single entry $C[i][j]$, this sum performs n multiplications and $n-1$ additions. Because the output matrix C has $n \times n = n^2$ entries in total, computing the entire matrix C directly requires:

$$\text{Multiplications} = n^2 \times n = n^3, \qquad \text{Additions} = n^2 \times (n-1) = n^3 - n^2.$$

When $n = 10$, $n^3 = 1{,}000$ operations (instant). When $n = 10{,}000$, $n^3 = 10^{12}$ (one trillion multiplications!). For decades, mathematicians believed n^3 multiplications was an unbreakable law of nature. In this guided project, you will start from the 6-line textbook loop and build a 5-stage sequence of data-structure and algorithmic upgrades that both speed up hardware memory access and break the $O(n^3)$ barrier.

Visual Primer: How 2D Matrices Live in 1D Computer Memory

Even though we draw a matrix M as a 2D grid of rows and columns, your computer's Random Access Memory (RAM) is a **single 1D street of numbered addresses**. Almost all programming languages (C, C++, Python/NumPy, Java) store a $n \times n$ matrix in **Row-Major Order**: Row 0 is placed first, immediately followed by Row 1, then Row 2, and so on. Thus, element $M[i][j]$ lives at 1D index $\text{index}(i, j) = i \cdot n + j$.



When the CPU reads $M[i][0]$ from RAM, it automatically fetches the next several neighbors ($M[i][1]$, $M[i][2], \ldots$) into ultra-fast **CPU Cache** (a "cache line"). Therefore:

- **Walking across a row** ($j = 0, 1, 2, \ldots$, stride $+1$) hits the fast CPU cache almost every time!
- **Walking down a column** ($k = 0, 1, 2, \ldots$, jumping by $+n$ addresses each step) leaps over entire rows, causing expensive **cache misses** when n is large.

Stage 1 & Stage 2: Naive `ijk` Loop vs. Cache-Friendly `ikj` Loop Reordering

Let us compare two ways to write the triple loop for $C = A \times B$ on an $n \times n$ matrix initialized to zeros:

Version 1: Standard `ijk` (Dot-Product Order)

```
def matmul_v1_ijk(A, B, n):
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            total = 0
            for k in range(n):
                total += A[i][k] * B[
                    k][j]
            C[i][j] = total
    return C
```

Bottleneck: In the innermost loop (`for k`), $A[i][k]$ moves across row i (stride $+1$), but $B[k][j]$ jumps down column j (stride $+n$, cache miss!).

Version 2: Reordered `ikj` (Row-Scaling Order)

```
def matmul_v2_ikj(A, B, n):
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            a_ik = A[i][k] # fixed
                scalar!
            for j in range(n):
                C[i][j] += a_ik * B[k
                    ][j]
    return C
```

Adjustment: By swapping the j and k loops, $A[i][k]$ is loaded once into a register `a_ik`, and the innermost loop (`for j`) walks across row k of B and row i of C with **stride** $+1$!

Part A: Hand-Worked Trace & Memory Access Analysis (Stages 1 & 2)

Consider the following concrete 2×2 matrices A and B:

$$A = \begin{pmatrix} 3 & -1 \\ 4 & 2 \end{pmatrix}, \qquad B = \begin{pmatrix} 2 & 5 \\ -3 & 1 \end{pmatrix}.$$

Q1. Trace Version 1 (`ijk`): Compute $C = A \times B$ by hand using the standard row-by-column dot product. Write down all 8 scalar multiplications and 4 additions explicitly.

Q2. Trace Version 2 (`ikj`): Start from the zero matrix $C^{(0)} = \left(\begin{smallmatrix} 0 & 0 \\ 0 & 0 \end{smallmatrix}\right)$. Write down the state of the entire 2×2 matrix C after each of the four (i, k) outer steps: $(i = 0, k = 0)$, $(i = 0, k = 1)$, $(i = 1, k = 0)$, and $(i = 1, k = 1)$. Verify that the final matrix C is identical to Question 1.

Q3. Memory Stride Calculation: Suppose A, B, C are $n \times n$ matrices stored in 1D row-major arrays where $A[i][j]$ is at memory offset $i \cdot n + j$. In the innermost loop of `ijk` ($k \to k+1$) versus the innermost loop of `ikj` ($j \to j+1$), by how many memory addresses do the accesses to A, B, and C jump at each step? Why do both versions have the exact same $\Theta(n^3)$ asymptotic complexity yet very different real-world speeds?

Stage 3 & Stage 4: Block Divide-and-Conquer → Strassen's 7-Multiply Breakthrough

Can we reduce the total number of multiplications below n^3? Suppose n is a power of two ($n = 2^m$), so we can slice each $n \times n$ matrix into four $\frac{n}{2} \times \frac{n}{2}$ quadrant submatrices (blocks):

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \qquad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \qquad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Version 3 (Naive Recursive Block Split — 8 Multiplications): Directly applying the 2×2 matrix product formula to the blocks gives:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}, \quad C_{21} = A_{21}B_{11} + A_{22}B_{21}, \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}.$$

Notice that this makes **8 recursive calls** on matrices of size $n/2$. As you will verify below, 8 recursive calls of size $n/2$ still perform $8 \times (n/2)^3 = n^3$ scalar multiplications!

Version 4 (Strassen's 1969 Algebraic Trick — Only 7 Multiplications!): Volker Strassen discovered that instead of computing the 8 block products above, we can form **7 clever combinations** $M_1, M_2, \ldots, M_7$ using only **1 multiplication each**, and then recombine them by addition/subtraction to get $C_{11}, C_{12}, C_{21}, C_{22}$:

$\mathbf{M_1} = (A_{11} + A_{22})(B_{11} + B_{22})$
$\mathbf{M_2} = (A_{21} + A_{22})B_{11}$
$\mathbf{M_3} = A_{11}(B_{12} - B_{22})$
$\mathbf{M_4} = A_{22}(B_{21} - B_{11})$
$\mathbf{M_5} = (A_{11} + A_{12})B_{22}$
$\mathbf{M_6} = (A_{21} - A_{11})(B_{11} + B_{12})$
$\mathbf{M_7} = (A_{12} - A_{22})(B_{21} + B_{22})$

→

Reconstructing the 4 Quadrants of C:
$C_{11} = M_1 + M_4 - M_5 + M_7$
$C_{12} = M_3 + M_5$
$C_{21} = M_2 + M_4$
$C_{22} = M_1 - M_2 + M_3 + M_6$
Cost per level: **7 block multiplies + 18 block adds/subs**.

Why is saving 1 multiplication worth doing 14 extra additions? Because for large matrices of size $\frac{n}{2} \times \frac{n}{2}$, **adding** two blocks costs only $(n/2)^2 = \Theta(n^2)$ work, whereas **multiplying** two blocks is vastly more expensive!

Part B: Tracing Strassen & Comparing Multiplication Counts (Stages 3 & 4)

Q4. Hand-Worked Strassen Trace: Using the same 2×2 matrices $A = \left(\begin{smallmatrix} 3 & -1 \\ 4 & 2 \end{smallmatrix}\right)$ and $B = \left(\begin{smallmatrix} 2 & 5 \\ -3 & 1 \end{smallmatrix}\right)$ (where each block A_{ij}, B_{ij} is simply a 1×1 number), compute $M_1, M_2, M_3, M_4, M_5, M_6, M_7$ by hand, then compute $C_{11}, C_{12}, C_{21}, C_{22}$ and confirm they match Part A.

Q5. Checking How Extra Terms Cancel: Substitute the definitions of $M_3 = A_{11}(B_{12} - B_{22})$ and $M_5 = (A_{11} + A_{12})B_{22}$ into $C_{12} = M_3 + M_5$, and substitute $M_2 = (A_{21} + A_{22})B_{11}$ and $M_4 = A_{22}(B_{21} - B_{11})$ into $C_{21} = M_2 + M_4$. Multiply out the parentheses and circle the pair of terms that cancel out in each expression.

Q6. Comparing Multiplication Counts (n^3 vs. $7^{\log_2 n}$): For an $n \times n$ matrix where $n = 2^k$, **Version 3** (8 block multiplications per split) performs $8^k = n^3$ scalar multiplications, whereas **Version 4 (Strassen)** performs only $7^k = 7^{\log_2 n} \approx n^{2.81}$ scalar multiplications. Fill in a table comparing 8^k and 7^k for $k = 1, 2, 3, 4$ (corresponding to matrix sizes $n = 2, 4, 8, 16$). How many multiplications does Strassen save at $n = 16$?

Stage 5: Hybrid Strassen with Cutoff Threshold n_0 & Odd-Dimension Padding

If you run pure Strassen recursion (Version 4) all the way down to 1×1 numbers, it is actually **much slower** than the simple loop! Why?

1. **Arithmetic Overhead at Small n:** At $n = 2$, standard multiplication does $2(2^3) - 2^2 = 12$ operations, whereas Strassen does $7(1) + 18(1) = 25$ operations.
2. **Memory Allocation Overhead:** Creating new submatrices for the 18 additions and 7 products at every tiny recursion level floods the memory allocator.

Version 5 Adjustment (Hybrid Strassen): Whenever the submatrix dimension n drops to or below a tunable **base-case cutoff threshold** n_0 (for instance, $n_0 = 64$), we **stop recursing** and call the fast cache-friendly `ikj` loop (Version 2) directly! Furthermore, if an input dimension n is odd, we dynamically pad by a single zero row and column ($n \to n+1$) before splitting in half, then slice off the extra row/column at the end.

Part C: Hybrid Cutoff, Odd Padding & 5-Stage Summary (Stage 5)

Q7. Arithmetic Crossover Check: Taking into account both multiplications and additions, Standard Multiplication performs $T_{\text{std}}(n) = 2n^3 - n^2$ total operations, whereas Pure Strassen performs $T_{\text{Strassen}}(n) = 7n^{\log_2 7} - 6n^2$ total operations (note that $n^{\log_2 7} = 7^k$ when $n = 2^k$). Evaluate both formulas at $n = 256$ ($k = 8$), $n = 512$ ($k = 9$), and $n = 1024$ ($k = 10$). At which of these three sizes does Pure Strassen first use fewer total operations than Standard Multiplication?

Q8. Odd-Dimension Padding Example: Suppose you want to multiply two 3×3 matrices using Stage 5, which requires an even dimension to split into 2×2 quadrants. Write down the padded 4×4 matrix when

$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ is padded with one extra row and column of zeros, and state which sub-slice of the 4×4 output C_{pad} is returned at the end.

Q9. Master 5-Stage Comparison Table: Fill out a summary table comparing **Versions 1, 2, 3, 4, and 5** across four columns: (i) Core Algorithm / Loop Order, (ii) Asymptotic Time Big-O, (iii) Extra Memory Big-O, and (iv) Main Practical Strength or Bottleneck.

Part D: Implementation & Empirical Benchmarking across All 5 Versions

Implement all five versions in Python (using plain 2D lists or un-vectorized NumPy loops so the algorithmic structure is measured fairly):

- `v1_ijk(A, B)`: Standard triple loop ($O(n^3)$, poor cache locality on B).
- `v2_ikj(A, B)`: Reordered triple loop ($O(n^3)$, sequential row stride).
- `v3_block_dc(A, B, n0)`: Recursive 8-multiply block divide-and-conquer.
- `v4_strassen_pure(A, B)`: Pure 7-multiply Strassen (or with small base case $n_0 = 2$).
- `v5_strassen_hybrid(A, B, n0)`: Hybrid Strassen switching to `v2_ikj` at $n \leq n_0$, supporting arbitrary n via odd-dimension padding.

Required Experiments & Plots:

1. **Automated Correctness Suite:** Test all 5 functions on edge cases ($n = 1, 2, 4, 7, 15, 33$, negative entries, identity matrices) asserting $\max_{i,j} |C_v(i,j) - C_{v1}(i,j)| = 0$ on integer matrices.
2. **Plot 1 (Runtime vs. Matrix Size n):** Measure wall-clock time for $n \in \{16, 32, 64, 128, 256, 512\}$ across Versions 1–5 and compute the empirical doubling ratio $T(2n)/T(n)$ (comparing $2^3 = 8.0\times$ vs. $2^{2.807} = 7.0\times$).
3. **Plot 2 (Cutoff Sweep & Memory Usage):** Fix $n = 256$ or 512, sweep the cutoff $n_0 \in \{4, 8, 16, 32, 64, 128, 256\}$, and plot both runtime and peak memory (`tracemalloc`) to identify the practical sweet spot n_0^*.

PROBLEM 2: POCKET RUBIK'S CUBE & STATE-SPACE SEARCH

Solving the $2 \times 2 \times 2$ *Rubik's Cube: From Unpruned DFS to Meet-in-the-Middle & IDA**

Core Data Structures & Concepts: State Tuples, FIFO Queues, Hash Sets, Bidirectional Frontier Tables, IDA* Recursion Stack

Algorithmic Progression: V1 (Sticker Grid + DFS) → V2 (Cubie Tuple + BFS) → V3 (Move-Pruned BFS) → V4 (Bidirectional BFS) → V5 (IDA* + Heuristic)

Why Puzzle Solving & Planning Are Actually Graph Shortest-Path Problems

At first glance, twisting a physical Rubik's Cube seems to have nothing in common with finding a route on a road map. Yet one of the most powerful ideas in computer science is that **any step-by-step planning problem can be modeled as an implicit unweighted graph** $G = (V, E)$:

- **Vertices (V):** Every legal configuration ("state") of the puzzle is a vertex $s \in V$. For the $2 \times 2 \times 2$ Pocket Cube (made of 8 corner pieces), there are $|V| = 3{,}674{,}160$ reachable states; for the standard $3 \times 3 \times 3$ Rubik's Cube, there are $|V| \approx 4.325 \times 10^{19}$ states.
- **Directed/Undirected Edges (E):** If applying a single face turn (such as rotating the Right face 90° clockwise, denoted R) transforms state u into state v, we draw an edge $u \xrightarrow{R} v$.

Why do we call this graph **implicit**? Because we never build an adjacency list of all 3.67 million vertices in advance! Instead, our program only holds the initial scrambled state s_{start} and a transition function `apply_move(s, m)` that generates the neighbors of any state s on the fly.

Finding the **minimum number of face turns** to reach the solved state s_{goal} (often called *God's Algorithm*) is simply the **Single-Pair Shortest Path Problem on an Unweighted Graph**. However, because the number of states at distance d grows exponentially as b^d (where b is the branching factor), a naive search either loops forever or runs out of Random Access Memory (RAM) within seconds. In this guided project, you will build a 5-stage sequence of data-structure and algorithmic upgrades that shrink the search space from hundreds of millions of states down to a few thousand.

Visual Primer: Compact Cubie Encoding & The Exponential Search Frontier

How should we represent a $2 \times 2 \times 2$ Pocket Cube inside a computer program?

1. **Fixing One Corner Eliminates Whole-Cube Rotations:** A $2 \times 2 \times 2$ cube consists exclusively of 8 corner pieces ("cubies"). If you toss the entire cube in the air and rotate it, the puzzle has not changed! By permanently holding the **Down-Back-Left (DBL)** corner cubie (index 7) fixed in its home slot, we never need to turn the Left (L), Down (D), or Back (B) faces. Every reachable state can be solved using only quarter-turns of the three remaining faces: **Up (U)**, **Right (R)**, and **Front (F)**! Each face can be turned $+90°$ clockwise (U, R, F) or $-90°$ counter-clockwise (U', R', F'), giving a branching factor of $b = 6$ **moves** from any state.

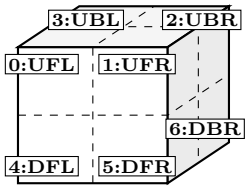
2. **The Immutable Tuple Data Structure (`perm, orient`):** Number the 7 movable corner slots $0, 1, 2, 3$ on the top (U) layer and $4, 5, 6$ on the bottom (D) layer:

   $$0 = \text{UFL},\ 1 = \text{UFR},\ 2 = \text{UBR},\ 3 = \text{UBL},\ 4 = \text{DFL},\ 5 = \text{DFR},\ 6 = \text{DBR}\ \ (\text{with } 7 = \text{DBL fixed}).$$

   Every state s is uniquely encoded by two length-7 integer tuples $s = (\texttt{perm}, \texttt{orient})$:

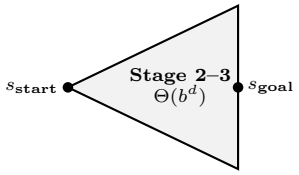
   - **Permutation tuple** $\texttt{perm} = (\pi_0, \pi_1, \ldots, \pi_6)$: $\pi_i \in \{0, \ldots, 6\}$ tells us *which* cubie currently sits in slot i. In the solved state, $\texttt{perm} = (0, 1, 2, 3, 4, 5, 6)$.
   - **Orientation tuple** $\texttt{orient} = (o_0, o_1, \ldots, o_6)$: $o_i \in \{0, 1, 2\}$ tells us how the cubie in slot i is twisted relative to the Up/Down reference faces (0 = U/D sticker faces Up or Down; 1 = twisted 120° clockwise; 2 = twisted 240° clockwise). In the solved state, $\texttt{orient} = (0, 0, 0, 0, 0, 0, 0)$.

1. $2 \times 2 \times 2$ **Corner Slots**

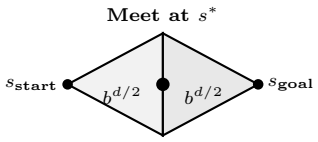


Slot 7 (DBL) held fixed $\implies$ Only U, R, F moves

2. Unidirectional BFS (b^d) **vs. Bidirectional** ($2b^{d/2}$)



Depth d



Two half-depths $d/2 + d/2$

The 5-Stage State-Space Search & Data Structure Evolution Ladder

Observe how each stage upgrades either the **state representation** or the **frontier search data structure**:

- **Stage 1: Naive 24-Sticker Array + Unpruned Depth-First Search (DFS)**
  - *Data Structure:* Represents the cube as a mutable 24-element color array and explores moves recursively using DFS without a visited set or depth cap.
  - *Why It Fails:* Firstly, DFS gets trapped in infinite cycles ($R \to R' \to R \to R' \dots$). Even if we cap the recursion depth at $D_{\max}$, plain DFS dives down a single path first and may return a 14-move solution when a 2-move solution existed right next to the root!
- **Stage 2: Compact Tuple `(perm, orient)` + Unidirectional BFS Queue & Hash Set**
  - *Data Structure Upgrade:* Encode states as immutable, hashable tuples $s = (\texttt{perm}, \texttt{orient})$. Maintain a **FIFO Queue** (`collections.deque`) initialized with s_{start} and a **Visited Hash Map** `dist[s]` (or `parent[s]`) so no state is ever enqueued twice.
  - *Guarantee & Bottleneck:* Because Breadth-First Search explores states strictly in order of distance $d = 0, 1, 2, \dots$, the first time it sees s_{goal} is **guaranteed to be a shortest path**! However, with 6 moves per step, the tree grows as 6^d (at $d = 11$, $6^{11} = 362{,}797{,}056$ move applications).
- **Stage 3: Move-Pruned BFS (Eliminating Redundant Consecutive Face Turns)**
  - *Algorithmic Adjustment:* Notice that following move R immediately by R' simply undoes R! Furthermore, turning the *same face* consecutively in the Quarter-Turn Metric (e.g., $R \cdot R \cdot R$ instead of R', or turning U right after U') explores duplicate states already reachable by fewer or canonical turns. By storing `last_move` with each queue entry and forbidding redundant same-face follow-ups (specifically: never follow X with X', and never apply X three times in a row; or in general, if 180° half-turns are grouped by face, never turn the same face twice in a row), the effective branching factor drops from $b = 6$ to $b_{\text{eff}} \approx 4$ (if we strictly alternate faces after step 1, $6 \cdot 4^{d-1}$ paths instead of 6^d).
- **Stage 4: Bidirectional BFS (Meet-in-the-Middle Frontier Search)**
  - *Data Structure Upgrade:* Maintain **two FIFO queues** (Q_F starting from s_{start}, Q_B starting from s_{goal}) and **two hash tables** (`parent_F` and `parent_B`). Alternate expanding one full depth level of Q_F and one full depth level of Q_B.
  - *Breakthrough:* As soon as a state s^* generated from one side is found in the **other side's hash table**, the two frontiers have met at depth $d/2$! Both time and memory drop from $\Theta(b_{\text{eff}}^d)$ to $\Theta(2 \cdot b_{\text{eff}}^{d/2})$ — turning $4^{12} = 16.7$ million states into $2 \times 4^6 = 8{,}192$ states!
- **Stage 5: Iterative Deepening A* (IDA*) with Admissible Corner Heuristic**
  - *Why Stage 4 Still Hits a Wall on* $3 \times 3 \times 3$*:* On a $3 \times 3 \times 3$ cube ($d = 20$), even $b^{d/2}$ requires gigabytes of hash tables. Richard Korf's **IDA*** eliminates the hash tables entirely!
  - *Algorithm:* Run Depth-First Search with a cost threshold T, pruning any branch as soon as $f(s) = g(s) + h(s) > T$, where $g(s)$ is the current depth and $h(s) = \left\lceil \frac{1}{4} \sum_{i=0}^{6} d_{\text{corner}}(i) \right\rceil$ is an **admissible lower bound** on the moves remaining. Start at $T = h(s_{\text{start}})$; if not found, update T to the smallest pruned f-value and repeat. Memory drops from $\Theta(b^{d/2})$ to $O(d)$ **stack space**!

Part A: Exact Move Mechanics, Hand Trace & State-Space Combinatorics (Stages 1 & 2)

In our 7-corner encoding $s = (\texttt{perm}, \texttt{orient})$ with slots $(0,1,2,3,4,5,6) = (\text{UFL}, \text{UFR}, \text{UBR}, \text{UBL}, \text{DFL}, \text{DFR}, \text{DBR})$, here are the exact transition rules for moves R, U, and R':

- **Move R ($90°$ clockwise on Right face $\{1,2,6,5\}$):** Slots $(1,2,6,5)$ receive the cubies previously in slots $(5,1,2,6)$ respectively, and their orientations increase by $(+1,+2,+1,+2)$ (mod 3). Slots $0,3,4$ do not change.
- **Move R' ($90°$ counter-clockwise on Right face $\{1,2,6,5\}$):** Slots $(1,2,6,5)$ receive the cubies previously in slots $(2,6,5,1)$ respectively, and their orientations also increase by $(+1,+2,+1,+2)$ (mod 3).
- **Move U ($90°$ clockwise on Up face $\{0,3,2,1\}$):** Slots $(0,3,2,1)$ receive the cubies previously in slots $(1,0,3,2)$ respectively, and their orientations change by $(0,0,0,0)$ (mod 3). Slots $4,5,6$ do not change.

Q1. Hand-Worked Tuple Trace: Starting from the solved state s_0 where $\texttt{perm}_0 = (0,1,2,3,4,5,6)$ and $\texttt{orient}_0 = (0,0,0,0,0,0,0)$, compute the exact state tuples $s_1 = (\texttt{perm}_1, \texttt{orient}_1)$, $s_2 = (\texttt{perm}_2, \texttt{orient}_2)$, and $s_3 = (\texttt{perm}_3, \texttt{orient}_3)$ after applying the 3-move sequence:

$$s_0 \xrightarrow{R} s_1 \xrightarrow{U} s_2 \xrightarrow{R'} s_3.$$

(*Self-check:* Also verify that applying R followed immediately by R' to s_0 returns s_0 because $+1+2 = 3 \equiv 0$ (mod 3)!)

Q2. Checking the Corner Twist Rule: In any valid Pocket Cube state, every face move adds $(0,0,0,0)$ (for U, U') or $(+1,+2,+1,+2)$ (mod 3) (for R, R', F, F') to the orientations of four corners, which changes the sum of orientations by 0 or 6 (both multiples of 3). Therefore, for every reachable state, the sum of all 7 corner orientations $\sum_{i=0}^{6} o_i$ is always divisible by 3.

- Compute the sum $\sum_{i=0}^{6} o_i$ for each of your states s_1, s_2, s_3 from Question 1 and confirm that each sum is a multiple of 3.
- Suppose a friend accidentally drops their Pocket Cube, pops out a single corner piece, and snaps it back in twisted by $+1$ ($120°$), giving $\texttt{orient} = (1,0,0,0,0,0,0)$. Using the rule above, explain why your BFS solver will never find a path to the solved state $(0,0,0,0,0,0,0)$.

Part B: Branching Factor Pruning & Bidirectional Meet-in-the-Middle Trace (Stages 3 & 4)

Q3. Quantifying the Frontier Collapse (Stages 2 → 3 → 4): Suppose a scrambled state requires $d = 12$ moves to solve. Compute the number of paths of length 12 explored by:

(a) Stage 2 (Unpruned 6-move BFS tree): $N_2 = 6^{12}$,

(b) Stage 3 (Face-alternating pruned BFS tree, where step 1 has 6 choices and steps $2 \ldots 12$ have 4 choices on the other two faces): $N_3 = 6 \times 4^{11}$,

(c) Stage 4 (Bidirectional pruned BFS expanding both trees to depth $d/2 = 6$): $N_4 = 2 \times (6 \times 4^5)$.

Calculate the exact numerical values of N_2, N_3, N_4 and the speedup ratio N_2/N_4.

Q4. Hand-Worked Bidirectional BFS Intersection & Path Reconstruction: Suppose s_{start} is the state s_3 you computed in Question 1 (so s_{start} was created by applying $R \cdot U \cdot R'$ to s_{goal}, meaning the shortest path back from s_{start} to s_{goal} has length $d = 3$: $s_{\text{start}} \xrightarrow{R} s_2 \xrightarrow{U'} s_1 \xrightarrow{R'} s_{\text{goal}}$).

- Trace **1 level of Forward BFS** from s_{start} (recording which state is reached by move R) and **2 levels of Backward BFS** from s_{goal} (recording which state is reached by $s_{\text{goal}} \xrightarrow{R} s_1 \xrightarrow{U} s_2$).
- Identify the exact meeting state s^* that appears in both `dist_F` and `dist_B`.
- *Subtle Algorithmic Bug Check:* In the backward tree starting from s_{goal}, the edge from s_{goal} to s^* followed moves (R, U). When you stitch the forward path ($s_{\text{start}} \to s^*$) and the backward path ($s^* \to s_{\text{goal}}$) together, why must you **reverse the order and invert the direction** of every move along the backward path ($(R,U) \to (U', R')$)?

Part C: Evaluating the Corner Heuristic & 5-Stage Summary (Stage 5)

Q5. Evaluating the Corner Distance Heuristic on State s_1: For each movable corner cubie $c \in \{0, \ldots, 6\}$, let $d_{\text{corner}}(c)$ be 0 if cubie c is currently in its home slot with orientation 0, and 1 if it is displaced or twisted. Because a single face turn moves 4 corners at once, Stage 5 divides the sum by 4:

$$h(s) = \left\lceil \frac{1}{4} \sum_{c=0}^{6} d_{\text{corner}}(c) \right\rceil .$$

- Look at state s_1 from Question 1 (reached from s_0 by a **single** turn R). How many of the 7 corners are displaced or twisted in s_1?
- What is the unscaled sum $\sum_{c=0}^{6} d_{\text{corner}}(c)$ on s_1, and what is $h(s_1)$ after dividing by 4? Why would using the unscaled sum (4) overestimate the true 1-move distance back to s_0?

Q6. Master 5-Stage Complexity Comparison Table: Create a summary table comparing **Stages 1, 2, 3, 4, and 5** across four columns: (i) Primary Data Structure(s) in RAM, (ii) Time Complexity to find a depth-d solution, (iii) Peak Memory (Space) Complexity in RAM, and (iv) Whether the returned path is guaranteed to be shortest (optimal).

Part D: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement the $2 \times 2 \times 2$ Pocket Cube state tuple and all five search stages in Python:

- `v1_dfs_unpruned(start, goal, max_depth)`: Depth-limited DFS without shortest-path guarantee.
- `v2_bfs_unidirectional(start, goal)`: Standard FIFO queue + visited hash set ($b = 6$).
- `v3_bfs_move_pruned(start, goal)`: FIFO queue pruning inverse/redundant consecutive moves on the same face.
- `v4_bfs_bidirectional(start, goal)`: Simultaneous forward/backward frontier expansion with path stitching.
- `v5_ida_star(start, goal)`: Iterative Deepening A* using the admissible corner heuristic $h(s) = \lceil \sum d_{\text{corner}}(c)/4 \rceil$ (and optionally a small corner-orientation pattern lookup table).

Required Experiments & Plots:

1. **Verification Suite:** Generate 20 random scrambles at depths $d \in \{1, 2, 3, 4, 5, 6\}$. Verify that applying the move sequence returned by Versions 2–5 transforms `start` into `goal` and that all four optimal solvers agree on the exact shortest path length d^*.
2. **Plot 1 (Expanded States vs. Scramble Depth d):** For optimal depths $d \in \{2, 4, 6, 8, 10, 11\}$, plot the average number of states expanded (on a $\log_{10}$ vertical axis) and wall-clock time across Versions 2, 3, 4, and 5.
3. **Plot 2 (Peak Memory Footprint):** Plot the maximum number of states stored simultaneously in memory (hash table size for V2–V4 vs. recursion stack depth for V5) as a function of d, demonstrating how V4 and V5 conquer the RAM explosion of V2.

PROBLEM 3: SHORTEST PATHS: FROM BELLMAN-FORD & DIJKSTRA TO A*

Shortest Paths on Road Networks: Priority Queue Engineering & Goal-Directed A Search*

Core Data Structures & Concepts: Adjacency Matrices vs. Adjacency Lists, Binary Min-Heaps, Lazy Deletion, Heuristic Functions

Algorithmic Progression: V1 (Matrix + Bellman-Ford) → V2 (Adj. List + Array PQ) → V3 (Binary Heap PQ) → V4 (Lazy-Deletion Heap) → V5 (Goal-Directed A*)

Why Every GPS Navigator & Internet Router Relies on Fast Shortest Paths

Whenever you ask a smartphone map for the fastest driving route from your home S to an airport T, or whenever an internet router forwards a data packet across fiber-optic cables, the computer models the network as a **weighted directed graph** $G = (V, E, w)$:

- **Vertices (V):** Road intersections or network routers (for a continental road map, $|V| \approx 20{,}000{,}000$).
- **Directed Edges (E):** One-way road segments $(u \to v)$ with a non-negative travel time or length $w(u, v) \geq 0$.

Every shortest-path algorithm maintains an array of tentative shortest-path estimates $d[v]$ (initialized to $d[S] = 0$ and $d[v] = +\infty$ for all $v \neq S$), along with a predecessor pointer `parent`$[v]$ to reconstruct the best route. The fundamental building block is **Edge Relaxation** on a directed edge $(u \to v)$ of weight $w(u, v)$:

$$\textbf{Relax}(u, v, w): \quad \text{if } d[u] + w(u, v) < d[v] \text{ then set } d[v] \leftarrow d[u] + w(u, v) \text{ and } \texttt{parent}[v] \leftarrow u.$$

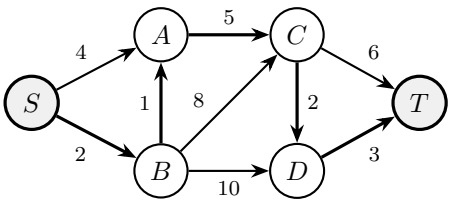
In words: *"Is going from S to u and then driving across road $(u \to v)$ faster than the best route to v we knew so far? If yes, update $d[v]$!"*

The entire art of shortest-path engineering comes down to two questions: **(1) How should we store the graph in memory?** and **(2) In what order should we relax edges so we only touch each road once and steer directly toward the destination T?**

Visual Primer: Road Network Graph, Priority Queues & Dijkstra vs. A* Wavefronts

Let us visualize our benchmark 6-vertex road network $G = (V, E, w)$ from source S to destination T, alongside the geometric intuition comparing Dijkstra's circular expansion against A* search:

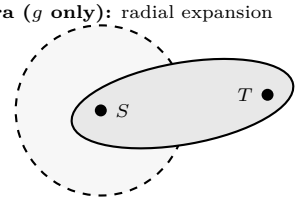
1. Road Network $G = (V, E, w)$



Shortest path: $S \to B \to A \to C \to D \to T$ **(cost** 13**)**

2. Search Wavefront: Dijkstra vs. A*

Dijkstra (g only): radial expansion



A* ($f = g + h$): goal-directed beam

The 5-Stage Shortest-Path & Priority Queue Evolution Ladder

Follow how each stage upgrades either the **graph storage**, the **priority queue data structure**, or the **ordering key**:

- **Stage 1: Adjacency Matrix + Bellman-Ford Repeated Relaxation ($\Theta(V^3)$ or $\Theta(VE)$)**

- *Data Structure & Algorithm:* Store weights in a $V \times V$ matrix $W[u][v]$ (or flat edge list). Since any simple shortest path contains at most $V-1$ edges, loop $V-1$ times over **every edge** $(u, v) \in E$ and call `Relax(u, v, w)`.
- *Strength & Bottleneck:* Works even when edges have **negative weights**! However, re-relaxing all E edges $V-1$ times costs $\Theta(VE)$ time (and $\Theta(V^2)$ space for an adjacency matrix). On $V = 10^6$ intersections, $V \cdot E \approx 3 \times 10^{12}$ operations takes hours.

- **Stage 2: Adjacency List + Naive Dijkstra with Unsorted Array Priority Queue ($\Theta(V^2)$)**
  - *Data Structure Upgrade:* Replace the $V \times V$ matrix with a compact **Adjacency List** `adj[u] = [(v, w), ...]` costing $\Theta(V + E)$ space.
  - *Dijkstra's Greedy Insight (1959):* When all edge weights are non-negative ($w(u, v) \geq 0$), the unvisited vertex u with the **smallest tentative distance** $d[u]$ can never be reached more cheaply via any other unvisited vertex! Thus we mark u as `settled`, relax its outgoing edges `adj[u]` **just once**, and never touch u again.
  - *Bottleneck:* If we find the minimum $d[u]$ among unvisited vertices by scanning a plain array $d[0 \ldots V-1]$ of length V at each of the V steps, finding minimums alone takes $\sum_{k=1}^{V} V = \Theta(V^2)$ time!

- **Stage 3: Binary Min-Heap Priority Queue with `Decrease-Key` ($O((V + E) \log V)$)**
  - *Data Structure Upgrade:* Store unsettled vertices in a **Binary Min-Heap** (a complete binary tree stored in an array where every parent key $\leq$ its children's keys), alongside an index lookup array `pos[v]` tracking where vertex v currently sits in the heap array.
  - *Complexity Gain:* Extracting the minimum vertex u (V times) and bubbling a key upward via `Decrease-Key` when $d[v]$ improves (at most E times) each cost $O(\log V)$ time, slashing total runtime from $\Theta(V^2)$ to $O((V + E) \log V)$!

- **Stage 4: Lazy-Deletion Binary Min-Heap (Real-World Standard Library Pattern)**
  - *Engineering Problem:* Built-in heaps in Python (`heapq`), C++ (`std::priority_queue`), and Java do **not** support an $O(\log V)$ `Decrease-Key` method because maintaining the `pos[v]` handle array adds pointer overhead!
  - *Lazy-Deletion Adjustment:* Whenever `Relax(u, v, w)` improves $d[v]$, simply **push a new duplicate tuple** `(d[v], v)` into the heap without deleting the old one! Later, when any pair `(dist_u, u)` is popped from the heap, check a single 1-line guard:

```
dist_u, u = heapq.heappop(pq)
if dist_u > d[u]:
    continue  # Stale duplicate entry from an older path -- skip in O(1)!
```

  Because each edge relaxation pushes at most one tuple, the heap holds at most E items, and $\log E \leq \log(V^2) = 2 \log V$, so worst-case runtime remains $O((V + E) \log V)$ with cleaner, faster code!

- **Stage 5: Goal-Directed A* Search with Potential Heuristic** $f(v) = d[v] + h(v)$
  - *Algorithmic Upgrade:* When navigating to a specific destination T, ordering the heap purely by $g(v) = d[v]$ explores vertices 50 miles *west* of S before exploring a vertex 51 miles *east* right outside T. By ordering the Min-Heap by $f(v) = d[v] + h(v)$, where $h(v)$ is an admissible lower bound on the remaining distance from v to T (such as straight-line Euclidean distance), the search prioritizes vertices heading toward T and terminates the instant T is popped!

Part A: Hand-Worked Traces & The Negative-Weight Trap (Stages 1–4)

Q1. Trace Stage 4 (Dijkstra with Lazy-Deletion Min-Heap): Consider the 6-vertex directed graph in the Concept Box with edges:

$$(S, A, 4),\ (S, B, 2),\ (B, A, 1),\ (A, C, 5),\ (B, C, 8),\ (B, D, 10),\ (C, D, 2),\ (C, T, 6),\ (D, T, 3).$$

Trace Stage 4 by hand starting from $d[S] = 0$ and pq $= [(0, S)]$. For every `heappop` event (including pops of stale duplicate tuples!), write down:

(a) The popped pair `(dist_u, u)` and whether it is **settled** or **skipped as stale** (`dist_u > d[u]`),

(b) The updated distance array $d[S, A, B, C, D, T]$ and predecessor array `parent`$[S, A, B, C, D, T]$,

(c) The exact list of tuples inside the Min-Heap `pq` after relaxing u's outgoing edges.

Q2. Tracing Where Greedy Dijkstra Breaks on a Negative Edge Weight: Consider the 4-vertex directed graph on $\{S, A, B, T\}$ with four edges:

$$(S, A, 3), \qquad (S, B, 2), \qquad (A, B, -2), \qquad (B, T, 1).$$

- What is the true shortest path from S to T and its total weight?
- Trace greedy Dijkstra (which permanently settles the vertex with the smallest $d[u]$ and never expands a settled vertex a second time). Which vertex between A and B does Dijkstra settle first, and what wrong distance $d[T]$ does it end up reporting?

Part B: Comparing Priority Queue Operation Counts (Stages 2–4)

Q3. Concrete Operation Count Comparison (Array vs. Min-Heap): On a graph with V vertices and E edges, **Stage 2 (Unsorted Array)** performs roughly V^2 operations to scan for the minimum vertex, whereas **Stage 4 (Lazy-Deletion Min-Heap)** performs roughly $(V + E) \log_2 V$ heap operations.

- **Sparse Road Network:** Suppose V = 1,000,000 intersections (2^{20}, so $\log_2 V \approx 20$) and E = 3,000,000 road segments. Calculate V^2 (for Stage 2) and $(V+E) \log_2 V$ (for Stage 4). By what factor is the Min-Heap faster than the unsorted array?
- **Dense Complete Graph:** Suppose V = 1,000 ($\log_2 V \approx 10$) and every vertex connects to every other vertex so E = 1,000,000. Compare V^2 against $(V + E) \log_2 V$: why does the simple unsorted array actually perform fewer operations than the heap when a graph is completely dense?

Part C: Hand-Worked A* Search Trace & 5-Stage Summary (Stage 5)

Q4. Hand-Worked Trace of Stage 5 (Goal-Directed A* Search): Suppose we want the shortest path from S to T on the 6-vertex graph from Question 1, and each vertex has the following heuristic estimate $h(v)$ of its remaining distance to T:

$$h(S) = 10, \quad h(A) = 7, \quad h(B) = 8, \quad h(C) = 3, \quad h(D) = 2, \quad h(T) = 0.$$

- Check that $h(v)$ never overestimates the true shortest remaining distance from v to T for each of the 6 vertices $v \in \{S, A, B, C, D, T\}$.
- Trace A* search by hand, where the Min-Heap is ordered by $f(v) = d[v] + h(v)$ (storing tuples `(f_v, d_v, v)`). Write down the sequence of popped entries and verify that A* finds the exact shortest path of length 13.

Q5. Master 5-Stage Complexity Comparison Table: Create a summary table comparing **Stages 1, 2, 3, 4, and 5** across four columns: (i) Graph & Priority Queue Data Structure, (ii) Time Complexity (Big-O), (iii) Space Complexity (Big-O), and (iv) Whether it handles negative edge weights or uses a goal heuristic.

Part D: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five shortest-path solvers in Python:

- `v1_bellman_ford(V, edges, s, t)`: $V - 1$ relaxation passes (plus V-th pass negative-cycle check).

- `v2_dijkstra_array(adj, s, t)`: Unsorted array scan for minimum $d[u]$ ($\Theta(V^2)$).
- `v3_dijkstra_indexed_heap(adj, s, t)`: Custom binary min-heap supporting in-place `decrease_key(v, new_d)` via a `pos[v]` index map.
- `v4_dijkstra_lazy_heap(adj, s, t)`: Standard library `heapq` with duplicate push and stale-entry skip.
- `v5_astar_euclidean(adj, coords, s, t)`: Lazy-deletion `heapq` ordered by $f(v) = d[v] + \|x_v - x_t\|_2$.

Required Experiments & Plots:

1. **Experiment 1 (Sparse Grid vs. Dense Graph Crossover):** Compare the runtime of `v1`, `v2`, `v3`, and `v4` on (a) sparse 2D road-like grids ($E \approx 4V$, $V \in \{500, 1000, 2500, 5000\}$) and (b) dense complete graphs ($E = V(V-1)$, $V \in \{100, 250, 500, 1000\}$). Show empirically where the simple $\Theta(V^2)$ array (`v2`) beats the heap on dense graphs and where `v3`/`v4` win by orders of magnitude on sparse graphs.
2. **Experiment 2 (2D Search Footprint Visualization — Dijkstra vs. A*):** On a 100×100 2D grid with random obstacle walls ($V = 10{,}000$), run `v4_dijkstra_lazy_heap` and `v5_astar_euclidean` from bottom-left $(5, 5)$ to top-right $(95, 95)$. Plot a 2D heatmap of all settled vertices for both algorithms and report the exact reduction in popped vertices!

PROBLEM 4: TRANSFORMER ATTENTION, ONLINE SOFTMAX & KV-CACHING

Sequence Attention in Transformers: From Naive Softmax to Online Streaming & KV-Caches

Core Data Structures & Concepts: 2D Activation Matrices, Running-Stat Accumulators (m_k, d_k), Pre-Allocated KV Append Buffers

Algorithmic Progression: V1 (Naive Unscaled Softmax) → V2 (Scaled 2-Pass Safe Softmax) → V3 (1-Pass Online Softmax) → V4 (Naive Causal Loop) → V5 (KV-Cached Decoding)

Why Every Modern AI Language Model Hinges on One Matrix Equation

When an AI assistant (such as ChatGPT, Gemini, or Claude) reads a sentence of n words ("tokens") and generates a reply, its core building block is not an 'if-else' rule or a recurrent loop, but a data-structure lookup operation called **Scaled Dot-Product Attention** (Vaswani et al., 2017).

Think of Attention as a **soft, differentiable hash table**:

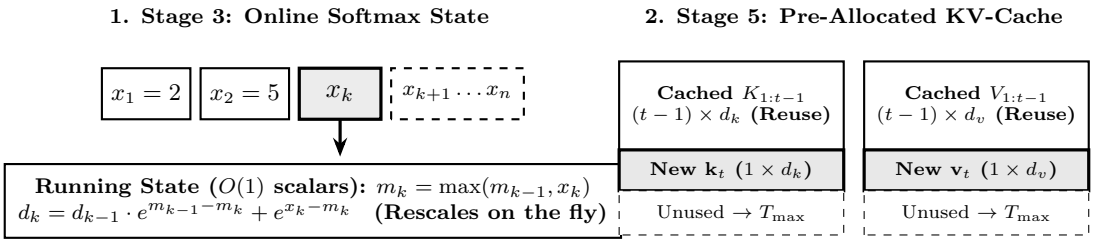
- Each token $i \in \{1, \dots, n\}$ emits a **Query vector** $\mathbf{q}_i \in \mathbb{R}^{d_k}$ (*"what information am I looking for?"*), a **Key vector** $\mathbf{k}_i \in \mathbb{R}^{d_k}$ (*"what topic/label do I contain?"*), and a **Value vector** $\mathbf{v}_i \in \mathbb{R}^{d_v}$ (*"what content do I provide if selected?"*).
- Stacking these vectors as rows of matrices $Q, K \in \mathbb{R}^{n \times d_k}$ and $V \in \mathbb{R}^{n \times d_v}$, token i measures its compatibility with every token j via the dot product $S_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$, converts row i of scores into probabilities $P_{i,1}, \dots, P_{i,n} \in (0, 1)$ summing to 1 via the **Softmax function**, and takes the weighted average of the Value rows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \qquad \text{where softmax}(\mathbf{x})_j = \frac{e^{x_j}}{\sum_{\ell=1}^{n} e^{x_\ell}}.$$

If you implement this textbook formula directly, it hits **three catastrophic walls**: (1) e^{x_j} overflows 32-bit floating-point numbers as soon as $x_j > 88.7$, crashing with `NaN`; (2) materializing the $n \times n$ matrix QK^T consumes $\Theta(n^2)$ memory; and (3) generating T tokens one by one naively takes $\Theta(T^3 d_k)$ time! In this project, you will build the exact 5-stage sequence of numerical and data-structure upgrades that power modern LLMs.

Visual Primer: Single-Pass Online Softmax & The Autoregressive KV-Cache Buffer

How do systems like **FlashAttention** avoid storing the entire n-element score row in fast memory before dividing by $\sum e^{x_j}$, and how does a **Key-Value (KV) Cache** avoid recomputing past tokens during text generation?



The 5-Stage Attention & Softmax Data Structure Evolution Ladder

- **Stage 1: Naive Unscaled Dot-Product Attention & Textbook Softmax**
  - *Formula:* Computes raw scores $S = QK^T$ and evaluates $\text{softmax}(\mathbf{x})_j = \frac{e^{x_j}}{\sum_{\ell=1}^{n} e^{x_\ell}}$ directly.

– *Two Fatal Flaws:* (1) Without dividing by $\sqrt{d_k}$, dot products of d_k-dimensional vectors have variance d_k (standard deviation $\sqrt{d_k}$), pushing inputs into extreme values where softmax saturates to 0 or 1 and gradients vanish; (2) In 32-bit IEEE floating-point (`float32`), the largest representable number is $\approx 3.40 \times 10^{38} \approx e^{88.72}$. Whenever any score $x_j \geq 89$, e^{x_j} overflows to `inf`, and `inf/inf` returns `NaN`!

- **Stage 2: Variance-Scaled ($1/\sqrt{d_k}$) Two-Pass Safe Softmax**
  – *Numerical Fix:* Scale scores by $1/\sqrt{d_k}$ and exploit the shift invariance of softmax by subtracting the row maximum $m = \max_{1 \leq \ell \leq n} x_\ell$ from every element before calling $\exp(\cdot)$:

$$\text{softmax}(\mathbf{x})_j = \frac{e^{x_j - m}}{\sum_{\ell=1}^{n} e^{x_\ell - m}}, \qquad \text{where } x_j - m \leq 0 \implies e^{x_j - m} \in (0, 1] \quad \text{(Zero overflow!)}.$$

  – *Memory Bottleneck:* Requires **two sequential passes** over the array $\mathbf{x}$ before any output can be produced: **Pass 1** scans all n elements to find $m = \max_\ell x_\ell$; **Pass 2** scans all n elements again to accumulate $d_n = \sum_{\ell=1}^{n} e^{x_\ell - m}$ (plus a third pass to divide and weight V).

- **Stage 3: Single-Pass Online Streaming Softmax (Milakov & Gimelshein, 2018)**
  – *Algorithmic Breakthrough:* Can we find the max m **and** the shifted sum $\sum e^{x_\ell - m}$ in a **single streaming pass** without knowing the true maximum in advance? Yes! Maintain a running pair (m_k, d_k) initialized to $m_0 = -\infty, d_0 = 0$. When element x_k arrives:

$$m_k = \max(m_{k-1},\, x_k), \qquad d_k = d_{k-1} \cdot e^{m_{k-1} - m_k} + e^{x_k - m_k}.$$

  Notice the magic correction factor $e^{m_{k-1} - m_k}$: whenever a new larger maximum $m_k > m_{k-1}$ is discovered, multiplying the old sum d_{k-1} by $e^{m_{k-1} - m_k}$ instantaneously rescales all $k-1$ previous terms from reference point m_{k-1} to the new reference point m_k! We can even maintain the running weighted value vector $\mathbf{o}_k \in \mathbb{R}^{d_v}$ in the same single pass:

$$\mathbf{o}_k = \frac{d_{k-1} e^{m_{k-1} - m_k}}{d_k} \mathbf{o}_{k-1} + \frac{e^{x_k - m_k}}{d_k} \mathbf{v}_k,$$

  computing the exact attention output for query $\mathbf{q}_i$ in **1 pass** and $O(d_v)$ **working RAM**!

- **Stage 4: Naive Causal Autoregressive Generation Loop ($\Theta(T^3 d_k)$ Time)**
  – *Context:* When generating a response of T tokens one word at a time, token i is only allowed to attend to past and present tokens $j \leq i$ (**Causal Masking**: set $S_{ij} = -\infty$ for all $j > i$).
  – *Bottleneck:* At generation step $t \in \{1, 2, \ldots, T\}$, a naive implementation passes all t tokens generated so far into the attention function, recomputing the entire $t \times t$ matrix $Q_{1:t} K_{1:t}^T$ from scratch in $\Theta(t^2 d_k)$ time. Summing over $t = 1 \ldots T$ gives $\sum_{t=1}^{T} \Theta(t^2 d_k) = \Theta(T^3 d_k)$ time!

- **Stage 5: Pre-Allocated Key-Value (`KVCache`) Data Structure ($\Theta(T^2 d_k)$ Time)**
  – *Data Structure Upgrade:* Because causal masking forbids earlier tokens $1 \ldots t-1$ from attending to token t, their keys $K_{1:t-1}$, values $V_{1:t-1}$, and outputs **never change** when token t arrives! Allocate a persistent `KVCache` object holding two dynamic/pre-allocated buffers `K_buf` and `V_buf`. At step t, append only the **single new row** $\mathbf{k}_t, \mathbf{v}_t$ to the cache in $O(d_k)$ time, and compute attention **only for the single new query vector** $\mathbf{q}_t \in \mathbb{R}^{1 \times d_k}$ against $K_{1:t}, V_{1:t}$ in $\Theta(t \cdot d_k)$ time. Total generation time across T steps collapses from $\Theta(T^3 d_k)$ to $\Theta(T^2 d_k)$!

Part A: Hand-Worked Causal Attention & Safe Max-Shifted Softmax (Stages 1 & 2)

Q1. Hand-Worked 3×3 Causal Attention Forward Pass: Consider a sequence of $n = 3$ tokens with embedding dimensions $d_k = d_v = 2$ (for simplicity in this hand trace, omit the $1/\sqrt{2}$ factor so numbers stay clean integers):

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 1 \end{pmatrix}, \qquad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 3 \end{pmatrix}.$$

Compute step by step by hand:

(a) The 3×3 raw dot-product matrix $M = QK^T$,

(b) The causally masked score matrix M_{mask} obtained by setting all entries strictly above the main diagonal ($j > i$) to $-\infty$,

(c) The row-wise probability matrix $P = \text{softmax}(M_{\text{mask}})$ (write exact expressions in terms of e and decimal approximations to 3 decimal places, noting that $e^{-\infty} = 0$), and

(d) The final 3×2 output matrix $O = PV$.

Q2. Hand-Worked Example of Why Max-Shifting Prevents Floating-Point Overflow (Stage 2): Suppose a 3-element score row is $\mathbf{x} = (1000, 1001, 1002)$.

- In standard 32-bit floating point (`float32`), any exponential e^z with $z > 88.7$ overflows to `inf`. What happens if Stage 1 tries to evaluate $\frac{e^{1002}}{e^{1000}+e^{1001}+e^{1002}}$ directly?
- Now apply **Stage 2 (Safe Softmax)**: find the maximum $m = \max(1000, 1001, 1002)$, write down the shifted vector $\mathbf{x} - m = (x_1 - m,\ x_2 - m,\ x_3 - m)$, and compute the three softmax probabilities to 3 decimal places.

Part B: Single-Pass Online Streaming Softmax Trace (Stage 3)

Q3. Hand-Worked Trace of Stage 3 (Online Softmax): Consider the 3-element score row $\mathbf{x} = (2, 5, 3)$.

- First, using **Stage 2 (Two-Pass Safe Softmax)**, write down the global maximum $m = \max(2, 5, 3)$ in Pass 1 and the exact shifted denominator $d = e^{2-m} + e^{5-m} + e^{3-m}$ in Pass 2.
- Next, trace **Stage 3 (Single-Pass Online Softmax)** starting from $m_0 = -\infty, d_0 = 0$. Write down the exact values of m_k and d_k at $k = 1$ ($x_1 = 2$), $k = 2$ ($x_2 = 5$), and $k = 3$ ($x_3 = 3$) using the recurrence:

$$m_k = \max(m_{k-1},\ x_k), \qquad d_k = d_{k-1} e^{m_{k-1} - m_k} + e^{x_k - m_k}.$$

  Confirm that d_3 at the end of Stage 3's single pass is identical to Stage 2's two-pass denominator d.

Q4. Comparing Array Passes: To find both the row maximum m and the shifted denominator d, how many times must Stage 2 loop over an n-element score array $\mathbf{x}$, and how many times does Stage 3 loop over $\mathbf{x}$?

Part C: Autoregressive KV-Caching Speedup & 5-Stage Summary (Stages 4 & 5)

Q5. Comparing Operation Counts of Stage 4 (No Cache) vs. Stage 5 (`KVCache`): When generating a sequence of T tokens one by one with embedding dimension d:

- **Stage 4 (No Cache)** recomputes all past token pairs at every step, performing $N_{\text{Stage 4}}(T) = \frac{T(T+1)(2T+1)}{3} d$ multiplications in total.
- **Stage 5 (`KVCache`)** appends each new Key and Value row to a buffer and only computes attention for the single newest token, performing $N_{\text{Stage 5}}(T) = T(T+1)d$ multiplications in total.
- Plug $T = 3{,}000$ tokens into the speedup ratio $\frac{N_{\text{Stage 4}}(T)}{N_{\text{Stage 5}}(T)} = \frac{2T+1}{3}$. How many times faster is Stage 5 than Stage 4 at $T = 3{,}000$? How many floating-point numbers are stored inside `K_buf` and `V_buf` combined when $T = 3{,}000$ and $d = 64$?

Q6. Master 5-Stage Summary Table: Construct a comparison table summarizing **Stages 1–5** across four columns: (i) Core Data Structure / Buffers Used, (ii) Number of Passes or Time Complexity, (iii) Peak Memory (Space) Complexity, and (iv) Main Numerical or Speed Improvement.

Part D: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five stages in Python / NumPy:

- `v1_naive_attention(Q, K, V)`: Unscaled QK^T and direct $\exp(x)/\sum\exp(x)$.
- `v2_safe_twopass_attention(Q, K, V)`: Scaled by $1/\sqrt{d_k}$ with 2-pass row-max subtraction.
- `v3_online_streaming_attention(Q, K, V)`: Single-pass streaming recurrence over keys/values $(m_k, d_k, \mathbf{o}_k)$ without materializing the $n \times n$ matrix in memory.
- `v4_autoregressive_nocache(W_q, W_k, W_v, tokens)`: Recomputes full causal attention at each step $t = 1 \dots T$.
- `v5_autoregressive_kvcache(W_q, W_k, W_v, tokens)`: Uses a pre-allocated `KVCache` class with an `append(k_t, v_t)` method.

Required Experiments & Plots:

1. **Overflow & Precision Test:** Test `v1`, `v2`, and `v3` in 32-bit float (`np.float32`) on vectors with scale $\sigma \in \{1, 10, 50, 100\}$. Show where `v1` crashes with `NaN` while `v2` and `v3` agree to $< 10^{-6}$ max absolute error.
2. **Plot 1 (Peak Memory vs. Sequence Length n):** Measure peak allocated memory for `v2` ($\Theta(n^2)$ matrix) vs. `v3` ($\Theta(nd)$ streaming) for $n \in \{256, 512, 1024, 2048, 4096\}$.
3. **Plot 2 (Generation Time vs. Sequence Length T):** Plot cumulative decoding time for $T \in \{64, 128, 256, 512, 1024\}$ comparing `v4_autoregressive_nocache` ($\Theta(T^3)$) against `v5_autoregressive_kvcache` ($\Theta(T^2)$).

PROBLEM 5: WEB RANKING, PAGERANK & SPARSE POWER ITERATION

Ranking the World Wide Web: From Citation Counts to the Google Matrix & Sparse Power Iteration

Core Data Structures & Concepts: Directed Graphs, Column-Stochastic Matrices, CSR/Adjacency Lists, State Probability Vectors

Algorithmic Progression: V1 (In-Degree Count) → V2 (Raw Markov Chain P) → V3 (Dense Google Matrix G) → V4 (Sparse CSR Power Iteration) → V5 (Gauss-Seidel + Early Stop)

How Do You Rank Billions of Web Pages Without Reading Them?

In the early 1990s, web search engines ranked pages simply by counting how many times a search word appeared on the page—an approach spammers defeated overnight by hiding thousands of invisible keywords at the bottom of their HTML files.

In 1998, PhD students Larry Page and Sergey Brin proposed a radical graph-theoretic idea called **PageRank**: model the entire World Wide Web as a **directed graph** $G = (V, E)$ where the $n = |V|$ vertices are web pages and each directed edge $(j \to i) \in E$ is a hyperlink from page j pointing to page i.

- **Attempt 1 (Simple In-Degree Voting):** Count how many incoming links $\text{indeg}(i)$ point to page i. *Why it fails:* A spammer can create a "link farm" of 10,000 fake pages all linking to their online store!
- **The PageRank Recursive Principle:** A link from an important page (like `wikipedia.org`) should count far more than a link from an obscure spam page! Specifically, if page j has importance score $\pi_j \geq 0$ and $\text{outdeg}(j)$ outgoing links, it splits its vote equally among the $\text{outdeg}(j)$ pages it links to:

$$\pi_i = \sum_{j\,:\,(j\to i)\in E} \frac{\pi_j}{\text{outdeg}(j)}, \qquad \text{subject to } \sum_{i=1}^{n} \pi_i = 1.$$

Imagine a **Random Web Surfer** who starts at a random page and repeatedly clicks a uniformly chosen outgoing link on the current page. Then π_i is simply the long-run fraction of time the surfer spends on page i! Yet turning this beautiful idea into a working algorithm for $n = 10^9$ web pages requires solving two nasty graph traps (**Spider Traps** and **Dead Ends**) and avoiding a catastrophic $\Theta(n^2)$ dense-matrix memory trap.

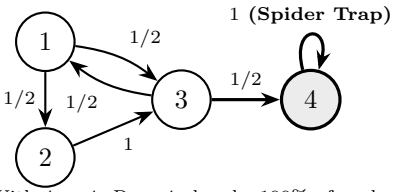
Visual Primer: Spider Traps, Dead Ends & Sparse vs. Dense Graph Storage

Let us define the **Column-Stochastic Transition Matrix** $P \in \mathbb{R}^{n\times n}$ of a web graph: entry P_{ij} is the probability that a surfer at page j (column j) clicks a link to page i (row i):

$$P_{ij} = \begin{cases} \dfrac{1}{\text{outdeg}(j)} & \text{if } (j \to i) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Notice that each **column** j of P lists the outgoing transition probabilities from page j, so column j sums to 1 whenever $\text{outdeg}(j) \geq 1$. One click of the random surfer updates the probability vector $\pi^{(k)}$ via matrix-vector multiplication: $\pi^{(k+1)} = P\pi^{(k)}$.

1. 4-Page Web Graph



With $4 \to 4$: Page 4 absorbs 100% of rank.
Without $4 \to 4$: Page 4 is a Dead End (outdeg = 0).

2. Stage 3 Dense G vs. Stage 4 Sparse CSR

Stage 3: Dense G
$G_{ij} = \alpha\tilde{P}_{ij} + \frac{1-\alpha}{n}$
All n^2 entries > 0
$\Theta(V^2)$ RAM & Time

→

Stage 4: Sparse CSR
Store only E links
$\pi^{(k+1)} = \alpha P\pi^{(k)} + c\mathbf{1}$
$\Theta(V + E)$ RAM & Time

The 5-Stage Web Ranking & PageRank Evolution Ladder

- **Stage 1: Simple In-Degree Citation Counting**
  - *Rule:* Score each page i by $\text{indeg}(i) = |\{j : (j \to i) \in E\}|$ (or normalized $\text{indeg}(i)/|E|$).
  - *Why It Fails:* Completely ignores the quality of the linking page j and whether page j spams links to thousands of sites. Easily manipulated by creating disposable link farms.
- **Stage 2: Raw Markov Chain Power Iteration ($\pi^{(k+1)} = P\pi^{(k)}$)**
  - *Rule:* Initialize $\pi^{(0)} = \frac{1}{n}\mathbf{1} = (1/n, \ldots, 1/n)^T$ and repeatedly multiply $\pi^{(k+1)} = P\pi^{(k)}$.
  - *Why It Fails on Real Web Graphs:*
    1. **Spider Traps:** Any closed subset of pages with no outgoing links to the rest of the web (like node 4 with self-loop $4 \to 4$) acts as a black hole that absorbs 100% of the surfer's probability ($\pi_4^{(k)} \to 1$, while all other pages $\pi_i^{(k)} \to 0$).
    2. **Dead Ends (Dangling Nodes):** Any page j with $\text{outdeg}(j) = 0$ (such as an image or PDF file with no outgoing links) produces an **all-zero column** in P, causing total probability $\sum_{i=1}^{n} \pi_i^{(k)}$ to leak out of the graph toward 0!
- **Stage 3: Dangling-Node Patch + Dense Google Teleportation Matrix G ($\Theta(V^2)$)**
  - *Brin & Page's Two-Part Fix:*
    1. **Fix Dead Ends:** If page j has $\text{outdeg}(j) = 0$, replace column j of P with the uniform column $\frac{1}{n}\mathbf{1}$ (if the surfer hits a dead end, they jump to a random page). Call this patched column-stochastic matrix $\tilde{P}$.
    2. **Fix Spider Traps (Teleportation / Damping α):** At every step, with probability $\alpha \in (0, 1)$ (standard choice $\alpha = 0.85$) the surfer follows an outgoing link via $\tilde{P}$, and with probability $1 - \alpha$ gets bored and **teleports to a uniformly random page in the entire web**:

$$G = \alpha\tilde{P} + \frac{1-\alpha}{n}\mathbf{1}\mathbf{1}^T, \qquad \pi^{(k+1)} = G\pi^{(k)},$$

       where $\mathbf{1}\mathbf{1}^T$ is the $n \times n$ matrix of all ones.
  - *The Dense Matrix Trap:* Because $\frac{1-\alpha}{n}\mathbf{1}\mathbf{1}^T$ adds a positive constant to **every single entry** of the $n \times n$ matrix, G is 100% **dense** (n^2 non-zeros)! Storing G in RAM and multiplying $G\pi^{(k)}$ takes $\Theta(V^2)$ space and $\Theta(V^2)$ time per step—utterly impossible when $V = 10^8$.
- **Stage 4: Sparse Adjacency-List / CSR Power Iteration ($\Theta(V + E)$ Time & Space)**
  - *Algebraic Breakthrough:* We **never** need to form the dense matrix G or $\tilde{P}$! Let $D = \{j : \text{outdeg}(j) = 0\}$ be the set of dangling nodes, and note that for any probability vector $\pi^{(k)}$, $\mathbf{1}^T\pi^{(k)} = \sum_{j=1}^{n} \pi_j^{(k)} = 1$. Multiplying G by $\pi^{(k)}$ algebraically simplifies to:

$$\pi^{(k+1)} = G\pi^{(k)} = \alpha P\pi^{(k)} + \underbrace{\left(\frac{\alpha \sum_{j \in D} \pi_j^{(k)} + (1-\alpha)}{n}\right)}_{\text{single scalar } c^{(k)} \text{ added to every entry!}} \mathbf{1}.$$

    Using a sparse adjacency list or Compressed Sparse Row/Column (CSR/CSC) representation of P, computing $P\pi^{(k)}$ touches only the E actual hyperlinks, and adding the scalar $c^{(k)}$ touches the V vertices. Cost per iteration drops from $\Theta(V^2)$ to $\Theta(V + E)$ **time and memory**!
- **Stage 5: In-Place Asynchronous Gauss-Seidel Sweep & ℓ_1-Residual Early Stopping**
  - *Engineering Upgrade:* Standard Power Iteration (Stage 4) is a synchronous *Jacobi* update: it keeps two arrays π_{old} and π_{new}, waiting until all n pages are computed before swapping arrays. By maintaining a single array $\pi[1 \ldots n]$ and writing updated values back into $\pi[i]$ **in place** as we sweep through $i = 1, \ldots, n$ (*Gauss-Seidel* style, re-normalizing $\sum \pi_i = 1$), newly updated ranks propagate immediately to downstream neighbours within the same pass, cutting iterations by $\approx$ 30%–50% and halting automatically once the ℓ_1 residual $\|\pi^{(k+1)} - \pi^{(k)}\|_1 = \sum_{i=1}^{n} |\pi_i^{(k+1)} - \pi_i^{(k)}| \leq \varepsilon$.

Part A: Hand-Worked Traces of Spider Traps, Dead Ends & Teleportation (Stages 1–3)

Consider the 4-page web graph in the Concept Box on vertices $\{1, 2, 3, 4\}$ with directed edges:

$$1 \to 2, \quad 1 \to 3, \quad 2 \to 3, \quad 3 \to 1, \quad 3 \to 4, \quad 4 \to 4.$$

Q1. Trace Stage 1 (In-Degree) & Stage 2 (Raw Markov Matrix P):

- Compute the Stage 1 in-degree score $\text{indeg}(i)$ for each page $i \in \{1, 2, 3, 4\}$.
- Write down the 4×4 transition matrix P. Starting from the uniform probability vector $\pi^{(0)} = (1/4,\ 1/4,\ 1/4,\ 1/4)^T$, compute $\pi^{(1)} = P\pi^{(0)}$ and $\pi^{(2)} = P\pi^{(1)}$ by hand.
- Check what happens if the surfer is at $\pi_{\text{trap}} = (0, 0, 0, 1)^T$: compute $P\pi_{\text{trap}}$ and explain in plain English why node 4 is called a "Spider Trap".
- Now suppose the self-loop $4 \to 4$ is **deleted** so page 4 has $\text{outdeg}(4) = 0$ (making column 4 of P_{dead} all zeros). Compute $P_{\text{dead}}\pi^{(0)}$ by hand: what is the sum of its entries, and where did the missing probability go?

Q2. Tracing Stage 3 (How Teleportation Escapes the Spider Trap): Keep the edge $4 \to 4$ (so $\tilde{P} = P$) and set the damping factor to $\alpha = 0.8$ (so the uniform teleportation term added to every page is $\frac{1-\alpha}{4} = \frac{0.2}{4} = 0.05$).

- Using the Stage 3 update $\pi^{(k+1)} = 0.8P\pi^{(k)} + (0.05, 0.05, 0.05, 0.05)^T$, compute one step $\pi^{(1)}$ starting from $\pi^{(0)} = (0.25, 0.25, 0.25, 0.25)^T$.
- Also compute one step starting from the trapped state $\pi_{\text{trap}} = (0, 0, 0, 1)^T$, and confirm that pages $1, 2, 3$ immediately regain non-zero probability!

Part B: Sparse Iteration with a Dead-End Node (Stage 4)

Q3. Hand Trace with a Dangling Node: Recall the Stage 4 sparse iteration formula from the Ladder Box:

$$\pi^{(k+1)} = \alpha P\pi^{(k)} + c^{(k)}\mathbf{1}, \qquad \text{where } c^{(k)} = \frac{\alpha S^{(k)}_{\text{dangling}} + (1-\alpha)}{n} \text{ and } S^{(k)}_{\text{dangling}} = \sum_{j \in D} \pi^{(k)}_j.$$

On the 4-page graph from Part A with edge $4 \to 4$ **removed** (so $D = \{4\}$ is a dangling dead end), compute $P\pi^{(0)}$, $S^{(0)}_{\text{dangling}}$, the scalar shift $c^{(0)}$, and $\pi^{(1)}$ starting from $\pi^{(0)} = (0.25, 0.25, 0.25, 0.25)^T$ with $\alpha = 0.8$. Verify that $\sum_{i=1}^{4} \pi^{(1)}_i = 1.00$ without ever building a 4×4 dense matrix!

Part C: Iteration Counts, Memory Comparison & 5-Stage Summary (Stages 4 & 5)

Q4. Comparing Iteration Counts & Memory Storage:

- At each step of PageRank, the error shrinks by a factor of α, so reaching an accuracy of $\varepsilon = 10^{-6}$ takes roughly $k \approx \frac{\ln(2\times 10^6)}{-\ln \alpha}$ iterations. Evaluate this formula for $\alpha = 0.85$ versus $\alpha = 0.99$. Why does choosing α very close to 1 require so many more iterations?
- Suppose a web graph has $V = 100{,}000$ pages and $E = 500{,}000$ links. How many floating-point numbers must Stage 3 store to hold the full $V \times V$ matrix G, compared to Stage 4 storing only the E links?

Q5. Master 5-Stage Complexity & Vulnerability Comparison Table: Construct a comparison table across **Stages 1, 2, 3, 4, and 5** comparing: (i) Memory complexity in RAM, (ii) Time complexity per iteration, (iii) Behavior on Spider Traps & Dead Ends, and (iv) Main practical advantage or limitation.

Part D: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five web-ranking algorithms in Python:

- `v1_indegree_rank(V, edges)`: Normalized in-degree citation counts.
- `v2_raw_markov_power(V, edges, num_iters)`: Un-damped transition matrix iteration $\pi^{(k+1)} = P\pi^{(k)}$.
- `v3_dense_google_power(V, edges, alpha, tol)`: Explicitly constructs the $V \times V$ dense matrix G in memory.
- `v4_sparse_csr_power(V, edges, alpha, tol)`: Adjacency-list / CSR sparse power iteration with dangling-node scalar shift $c^{(k)}$ ($\Theta(V + E)$ memory and time).
- `v5_gauss_seidel_pagerank(V, edges, alpha, tol)`: In-place asynchronous updates with ℓ_1-residual early stopping.

Required Experiments & Plots:

1. **Verification on the 4-Page Graph:** Run `v3`, `v4`, and `v5` on the 4-node graph from Part A with $\alpha = 0.8$ and verify that all three converge to the exact same stationary PageRank vector $\pi^* \approx (0.1208, 0.0983, 0.1770, 0.6039)^T$.
2. **Plot 1 (Convergence Rate vs. Damping α):** On a synthetic scale-free web graph with $V = 2{,}000, E = 10{,}000$, plot $\log_{10} \|\pi^{(k+1)} - \pi^{(k)}\|_1$ vs. iteration k for $\alpha \in \{0.50, 0.85, 0.95, 0.99\}$ and compare synchronous Jacobi (`v4`) vs. asynchronous Gauss-Seidel (`v5`).
3. **Plot 2 (Dense $\Theta(V^2)$ vs. Sparse $\Theta(V + E)$ Scaling):** Measure per-iteration time and peak memory for $V \in \{500, 1000, 2000, 5000, 10000\}$ (with $E = 5V$) comparing `v3_dense` against `v4_sparse`.

PROBLEM 6: FROM GRADIENT DESCENT TO MOMENTUM, RMSPROP & ADAM

Iterative Optimization & Root Finding: Gradient Descent → SGD → Momentum → RMSProp → Adam

Core Data Structures & Concepts: Parameter Arrays, Velocity & Second-Moment State Buffers, Exponential Moving Averages (EMA)

Algorithmic Progression: V1 (Batch GD) → V2 (Mini-Batch SGD) → V3 (Momentum) → V4 (RMSProp) → V5 (Adam & Newton Rescaling)

Why Every Machine Learning & Engineering System Needs an Optimizer

Suppose you are calibrating a physical sensor, fitting a line through $N = 1{,}000{,}000$ data points, or training a neural network with d parameters $\mathbf{w} = (w_1, w_2, \ldots, w_d)$. Your goal is to find the parameter vector $\mathbf{w}^*$ that minimizes an error (or "loss") function:

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\mathbf{w}),$$

where $\ell_i(\mathbf{w})$ measures how badly parameter choice $\mathbf{w}$ predicts data point i.

Imagine standing blindfolded on a hilly mountain range in heavy fog where your altitude at position $\mathbf{w}$ is $L(\mathbf{w})$. You cannot see the bottom of the valley from afar, but with your feet you can feel the local **slope (gradient)** $\nabla L(\mathbf{w}) = \left(\frac{\partial L}{\partial w_1}, \ldots, \frac{\partial L}{\partial w_d}\right)$, which always points in the direction of **steepest ascent** (uphill). Therefore, pointing in the opposite direction $-\nabla L(\mathbf{w})$ points directly **downhill**!

However, as engineers quickly discovered, simply taking steps proportional to $-\nabla L(\mathbf{w})$ fails miserably on realistic problems: it is too slow on large datasets, it zig-zags wildly across narrow canyons, and it stalls on flat plateaus. In this project, you will trace the exact 5-stage evolutionary ladder that led from basic Gradient Descent to the **Adam** optimizer that trains virtually every modern AI model.

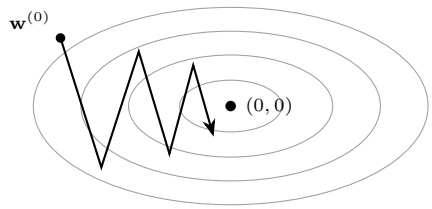
Visual Primer: The Narrow Canyon Problem & Why Naive Gradient Descent Zig-Zags

To see why simple downhill steps struggle, consider a 2D bowl with different steepness along w_1 (gentle horizontal valley floor) and w_2 (very steep vertical canyon walls):

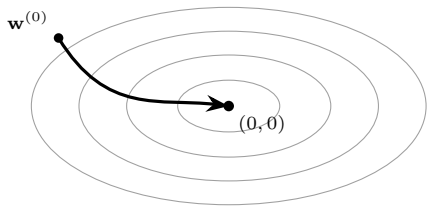
$$f(w_1, w_2) = \frac{1}{2}w_1^2 + \frac{10}{2}w_2^2 = 0.5w_1^2 + 5w_2^2, \qquad \nabla f(w_1, w_2) = \begin{pmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \end{pmatrix} = \begin{pmatrix} w_1 \\ 10w_2 \end{pmatrix}.$$

The true minimum is at the origin $(w_1^*, w_2^*) = (0, 0)$. Notice that at a point like $(w_1, w_2) = (4, 1)$, the gradient is $\nabla f(4, 1) = (4, 10)$: it points almost entirely vertically (10) toward the opposite canyon wall rather than horizontally (4) toward the minimum $(0, 0)$!

Stages 1–2: Vanilla GD (Zig-Zags Across Steep Canyon Walls)



Stages 3–5: Momentum + RMSProp (Adam) (Smooth Trajectory to Minimum)



Two complementary physical ideas cure this zig-zagging:

1. **Momentum (Stage 3):** Maintain a **velocity vector** $\mathbf{v}_t$ that averages recent gradients. Because the vertical gradient flips sign every step $(+10, -10, +10, \ldots)$, the vertical oscillations **cancel out** in $\mathbf{v}_t$, while the consistent horizontal pull builds up speed!

2. **Adaptive Coordinate Scaling (Stages 4 & 5):** Track the running average of **squared gradients** $s_{t,j} \approx \text{mean}(g_j^2)$ for each coordinate j, and divide the step by $\sqrt{s_{t,j}}$. Coordinates with huge noisy gradients (w_2) automatically get a **smaller effective step size**, while flat coordinates (w_1) get a **larger effective step size**!

The 5-Stage Optimizer & Data Structure Evolution Ladder

At every iteration $t = 1, 2, \ldots, T$, let $\mathbf{g}_t \in \mathbb{R}^d$ denote the gradient vector evaluated at $\mathbf{w}_{t-1}$, and let $\eta > 0$ be the learning rate (step size). Observe how each stage upgrades the **Optimizer State Data Structure** stored in memory:

- **Stage 1: Full-Batch Gradient Descent (GD)**
  - *Gradient Computation:* Loops over **all** N **data points**: $\mathbf{g}_t = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\mathbf{w}_{t-1})$. Cost: $\Theta(Nd)$ per step!
  - *Update Rule:* $\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{g}_t$. *Extra State Memory:* 0 vectors (only $\mathbf{w}$ and $\mathbf{g}_t$).
- **Stage 2: Stochastic / Mini-Batch Gradient Descent (SGD)**
  - *What Changed:* Waiting to scan $N = 10^6$ points just to take **one** tiny step wastes massive compute. Instead, sample a small random mini-batch $B_t \subset \{1, \ldots, N\}$ of size $b \ll N$ (e.g., $b = 1$ or $b = 32$) and estimate $\mathbf{g}_t = \frac{1}{b}\sum_{i \in B_t} \nabla \ell_i(\mathbf{w}_{t-1})$.
  - *Complexity Gain:* Step cost drops from $\Theta(Nd)$ to $\Theta(bd)$ — a $N/b \approx 30{,}000\times$ speedup per step! However, mini-batch noise and canyon curvature still cause severe zig-zagging.
- **Stage 3: SGD with Heavy-Ball Momentum (Polyak, 1964)**
  - *Data Structure Upgrade:* Allocate a persistent **velocity array** $\mathbf{v} \in \mathbb{R}^d$ (initialized to $\mathbf{v}_0 = \mathbf{0}$) with friction decay parameter $\beta_1 \in [0, 1)$ (typically $\beta_1 = 0.9$):
    $$\mathbf{v}_t = \beta_1 \mathbf{v}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \qquad \mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{v}_t.$$
  - *Extra State Memory:* $1d$ floats (for $\mathbf{v}$).
- **Stage 4: RMSProp (Root Mean Square Propagation — Hinton, 2012)**
  - *Data Structure Upgrade:* Allocate a persistent **squared-gradient accumulator array** $\mathbf{s} \in \mathbb{R}^d$ (initialized to $\mathbf{s}_0 = \mathbf{0}$) with decay rate $\beta_2 \in [0, 1)$ (e.g., $\beta_2 = 0.99$) and tiny safety constant $\varepsilon = 10^{-8}$ (to prevent division by zero):
    $$s_{t,j} = \beta_2 s_{t-1,j} + (1 - \beta_2) g_{t,j}^2, \qquad w_{t,j} = w_{t-1,j} - \frac{\eta}{\sqrt{s_{t,j}} + \varepsilon} g_{t,j} \quad \text{(for each } j = 1, \ldots, d\text{)}.$$
  - *Extra State Memory:* $1d$ floats (for $\mathbf{s}$).
- **Stage 5: Adam (Adaptive Moment Estimation — Kingma & Ba, 2014) & Newton's Method**
  - *Data Structure Upgrade:* Maintain **both** the first-moment velocity array $\mathbf{m}_t \in \mathbb{R}^d$ (like Momentum) and the second-moment squared-gradient array $\mathbf{s}_t \in \mathbb{R}^d$ (like RMSProp), plus an integer step counter t:
    $$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \qquad \mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2 \quad \text{(elementwise square)}.$$
    Because $\mathbf{m}_0 = \mathbf{0}$ and $\mathbf{s}_0 = \mathbf{0}$, in the first few steps $t = 1, 2$ the averages are biased toward zero! Adam fixes this with **Bias Correction** before updating $\mathbf{w}_t$:
    $$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \qquad \hat{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \beta_2^t}, \qquad w_{t,j} = w_{t-1,j} - \frac{\eta}{\sqrt{\hat{s}_{t,j}} + \varepsilon} \hat{m}_{t,j}.$$
  - *Bonus Comparison (Newton's Method):* When the exact second derivative $f''(w)$ (or Hessian matrix H) is known and cheap to invert, Newton's method scales the gradient by the exact inverse curvature: $w_t = w_{t-1} - [f''(w_{t-1})]^{-1} f'(w_{t-1})$.

Part A: Hand-Worked Trace Across the Optimizer Ladder

Consider minimizing the 2D anisotropic canyon function $f(w_1, w_2) = 0.5w_1^2 + 5w_2^2$, where the gradient at any point is $\nabla f(w_1, w_2) = (w_1,\, 10w_2)$. Start all optimizers at initial point $\mathbf{w}_0 = (4.0,\, 1.0)$ with learning rate $\eta = 0.18$.

Q1. Trace Stage 1 (Vanilla GD): Compute $\mathbf{g}_1 = \nabla f(\mathbf{w}_0)$, $\mathbf{w}_1 = \mathbf{w}_0 - 0.18\mathbf{g}_1$, $\mathbf{g}_2 = \nabla f(\mathbf{w}_1)$, and $\mathbf{w}_2 = \mathbf{w}_1 - 0.18\mathbf{g}_2$ by hand.

- Look at the w_2 coordinate across $t = 0, 1, 2$: why does w_2 flip sign from $+1.0 \to -0.8 \to +0.64$ while w_1 creeps slowly downward?
- Now compute a single step $\mathbf{w}_1 = \mathbf{w}_0 - \eta\mathbf{g}_1$ if the learning rate is increased slightly to $\eta = 0.25$. What is the new w_2 coordinate, and why is it farther from 0 than where it started?

Q2. Trace Stage 3 (Momentum): Starting from $\mathbf{w}_0 = (4.0, 1.0)$ and $\mathbf{v}_0 = (0.0, 0.0)$ with $\beta_1 = 0.5$ and $\eta = 0.18$, compute $\mathbf{g}_1, \mathbf{v}_1, \mathbf{w}_1$ and $\mathbf{g}_2, \mathbf{v}_2, \mathbf{w}_2$ by hand. Show how averaging $+10$ (from step 1) with the gradient at step 2 dampens the vertical jump in $\mathbf{v}_2$!

Q3. Trace Stage 4 (RMSProp) & Stage 5 (Adam): Starting from $\mathbf{w}_0 = (4.0, 1.0)$ with $\beta_1 = 0.9$, $\beta_2 = 0.9$, $\varepsilon = 0$, and $\eta = 0.5$:

- Compute Step $t = 1$ of **RMSProp** ($s_{1,1}, s_{1,2}$ and $w_{1,1}, w_{1,2}$) by hand.
- Compute Step $t = 1$ of **Adam** ($m_{1,j}, s_{1,j}$, bias-corrected $\hat{m}_{1,j}, \hat{s}_{1,j}$, and $w_{1,j}$) by hand.
- Notice a remarkable property at Step $t = 1$ of Adam (when $\varepsilon = 0$): what is $\hat{m}_{1,j}/\sqrt{\hat{s}_{1,j}}$ equal to for **both** $j = 1$ and $j = 2$, regardless of the fact that $g_{1,1} = 4$ and $g_{1,2} = 10$ differ by a factor of 2.5×?

Part B: Checking Bias Correction, Newton's Step & 5-Stage Summary

Q4. Numerical Check of Adam's Bias Correction: Suppose the gradient at step 1 is $g_1 = 10$, we start from $m_0 = 0$, and $\beta_1 = 0.99$.

- Compute the raw moving average $m_1 = \beta_1 m_0 + (1 - \beta_1)g_1$ at step $t = 1$. How small is m_1 compared to the actual gradient $g_1 = 10$?
- Now compute the bias-corrected value $\hat{m}_1 = \frac{m_1}{1-\beta_1^1}$ and confirm that it restores the true gradient value 10!

Q5. Newton's Method vs. First-Order Optimizers:

- On our quadratic bowl $f(w_1, w_2) = 0.5w_1^2 + 5w_2^2$, the diagonal second-derivative (Hessian) matrix is $H = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 10 \end{smallmatrix}\right)$, whose inverse is $H^{-1} = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 0.1 \end{smallmatrix}\right)$. Compute a **single step** of Newton's method $\mathbf{w}_1 = \mathbf{w}_0 - H^{-1}\nabla f(\mathbf{w}_0)$ from $\mathbf{w}_0 = (4, 1)$ by hand. Where does $\mathbf{w}_1$ land after just 1 step?
- For finding the square root of 2 (solving $g(x) = x^2 - 2 = 0$), Newton's update simplifies to averaging x_k and $2/x_k$: $x_{k+1} = \frac{1}{2}\left(x_k + \frac{2}{x_k}\right)$. Starting from $x_0 = 1$, compute x_1 and x_2 by hand and compare x_2 to $\sqrt{2} \approx 1.4142$.

Q6. Data Structure & Memory Complexity Table: Fill out a comparison table across **Batch GD, Mini-Batch SGD, Momentum, RMSProp, Adam, and Full-Hessian Newton's Method** for a model with d parameters and N training examples, listing: (i) Time complexity per step, (ii) Extra state arrays stored in memory (as a multiple of d), and (iii) Main strength or limitation.

Part C: Implementation & Empirical Benchmarking Lab

Implement all five first-order optimizer classes (`BatchGD`, `MiniBatchSGD`, `MomentumOptimizer`, `RMSProp Optimizer`, `AdamOptimizer`) plus `NewtonOptimizer` using a clean object-oriented data structure interface (`step(w, grad)`):

1. **Experiment 1 (2D Trajectory Visualization on the Rosenbrock & Canyon Landscapes):** Run all optimizers for $T = 200$ steps from $\mathbf{w}_0 = (-1.5, 2.0)$ on both (a) the ill-conditioned quadratic canyon $f(w_1, w_2) = 0.5w_1^2 + 25w_2^2$ and (b) the curved Rosenbrock banana valley $f(w_1, w_2) = (1 - w_1)^2 + 100(w_2 - w_1^2)^2$. Plot their 2D paths on top of the contour lines and plot $\log_{10} f(\mathbf{w}_t)$ vs. iteration t.
2. **Experiment 2 (Linear Regression on $N = 50{,}000$ Samples — Batch GD vs. Mini-Batch Ladder):** Generate a synthetic dataset $X \in \mathbb{R}^{50000\times 50}$, $\mathbf{y} \in \mathbb{R}^{50000}$ with differing feature scales. Plot the loss $L(\mathbf{w})$ against **wall-clock time (in seconds)** and **memory usage** for Full-Batch GD vs. Mini-Batch ($b = 64$) SGD, Momentum, RMSProp, and Adam.

PROBLEM 7: POLYNOMIAL & BIG-INTEGER MULTIPLICATION

From Schoolbook Convolution to Karatsuba & the Fast Fourier Transform (FFT)

Core Data Structures & Concepts: Coefficient Arrays, Point-Value Arrays, Complex Roots of Unity, Bit-Reversal Permutation

Algorithmic Progression: V1 (Schoolbook $\Theta(n^2)$) → V2 (Naive 4-Way Split $\Theta(n^2)$) → V3 (Karatsuba 3-Multiply $\Theta(n^{1.585})$) → V4 (Recursive Cooley-Tukey FFT $\Theta(n \log n)$) → V5 (Iterative In-Place Bit-Reversal FFT)

Why Multiplying Polynomials & Giant Integers Drives Signal Processing and Cryptography

How do we represent a polynomial of degree $n-1$, such as $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$, inside a computer? We simply store its coefficients in a **1D array of length** n, where array index j holds the coefficient a_j of x^j:

$$A(x) = 1 + 5x + 2x^2 + 3x^3 \quad \Longleftrightarrow \quad \texttt{a = [1, 5, 2, 3]} \quad (\texttt{a[0]} = 1, \texttt{a[1]} = 5, \texttt{a[2]} = 2, \texttt{a[3]} = 3).$$

Notice that a **multi-digit integer** is just a polynomial evaluated at a base like $x = 10$ (or $x = 2^{64}$ in computer words): for example, $3{,}251 = 1 + 5(10)^1 + 2(10)^2 + 3(10)^3$!

When we multiply two degree-$(n-1)$ polynomials $A(x)$ and $B(x)$, their product $C(x) = A(x) \cdot B(x)$ has degree $2n-2$ (requiring an output array `c` of length $2n-1$). Collecting all terms whose powers add up to x^k ($x^i \cdot x^j = x^{i+j} = x^k$) gives the fundamental **Discrete Convolution Formula**:

$$c_k = \sum_{i+j=k} a_i b_j = \sum_{i=0}^{k} a_i b_{k-i} \qquad \text{for } k = 0, 1, \ldots, 2n-2.$$

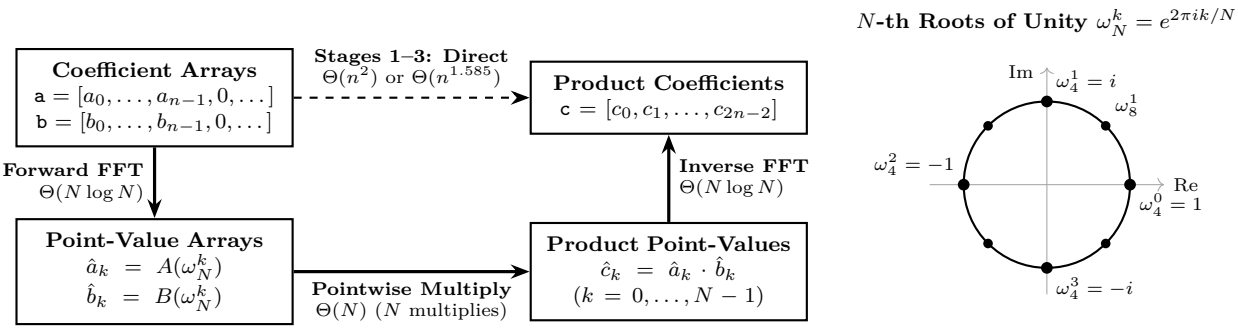
This exact operation is the workhorse of **audio filtering and image blurring** (convolving an audio signal of $n = 10^6$ samples with an echo filter), **cryptographic big-integer arithmetic**, and **error-correcting codes**. Yet computing all pairs a_ib_j directly requires $n \times n = n^2$ multiplications — for a 1-second audio clip at $n = 10^6$, $n^2 = 10^{12}$ operations! In this problem, you will climb the 5-stage evolutionary ladder from the $\Theta(n^2)$ schoolbook loop to Karatsuba's $\Theta(n^{1.585})$ divide-and-conquer trick and finally to the $\Theta(n \log n)$ in-place Fast Fourier Transform (FFT).

Foundational Primer: Coefficient Arrays vs. Point-Value Arrays & Complex Roots of Unity

Why does multiplying two polynomials in the standard **Coefficient Representation** $\texttt{a} = [a_0, \ldots, a_{n-1}]$ cost $\Theta(n^2)$? Because every coefficient a_i in $A(x)$ must interact with every coefficient b_j in $B(x)$.

However, recall a basic fact from algebra: a line (degree 1) is uniquely determined by 2 points; a parabola (degree 2) is uniquely determined by 3 points; and in general, **any polynomial** $C(x)$ **of degree** $d \leq N-1$ **is uniquely determined by its values at** N **distinct points** $x_0, x_1, \ldots, x_{N-1}$! This gives a second way to store a polynomial in memory:

- **Point-Value Representation:** Store the array of evaluations $\hat{\texttt{a}} = [A(x_0), A(x_1), \ldots, A(x_{N-1})]$ at $N \geq 2n-1$ chosen points.
- **Why Point-Value Multiplication Is** $\Theta(N)$**:** Since $C(x) = A(x) \cdot B(x)$ holds at every point x_k, multiplying two polynomials in point-value form requires just N **elementwise multiplications**: $C(x_k) = A(x_k) \cdot B(x_k)$ for $k = 0, \ldots, N-1$!



Normally, evaluating a polynomial at N arbitrary points still takes $N \times \Theta(N) = \Theta(N^2)$ work. The magic of the **Fast Fourier Transform (FFT)** is to choose $N = 2^m$ points to be the **complex N-th roots of unity**:

$$\omega_N^k = \cos\left(\frac{2\pi k}{N}\right) + i\sin\left(\frac{2\pi k}{N}\right) = e^{2\pi i k/N} \qquad (k = 0, 1, \ldots, N-1, \text{ where } i^2 = -1).$$

Because the roots of unity come in opposite pairs ($\omega_N^{k+N/2} = -\omega_N^k$), when we square the N roots of unity, they collapse into just $N/2$ **roots of unity** ($\left(\omega_N^k\right)^2 = \left(-\omega_N^k\right)^2 = \omega_{N/2}^k$), halving the problem size at every recursive step!

Stages 1, 2 & 3: Schoolbook Convolution → 4-Way Split → Karatsuba's 3-Multiply Trick

Assume n is a power of two ($n = 2^k$), and let $m = n/2$. We can slice the coefficient arrays of $A(x)$ and $B(x)$ at index m into a **low-degree half** (A_L, B_L of length m) and a **high-degree half** (A_H, B_H of length m):

$$A(x) = A_H(x)x^m + A_L(x), \qquad B(x) = B_H(x)x^m + B_L(x).$$

Multiplying these two expressions out gives:

$$C(x) = A(x)B(x) = \underbrace{\big(A_H(x)B_H(x)\big)}_{P_1(x)} x^{2m} + \underbrace{\big(A_H(x)B_L(x) + A_L(x)B_H(x)\big)}_{\text{Middle Cross-Term}} x^m + \underbrace{\big(A_L(x)B_L(x)\big)}_{P_2(x)}.$$

- **Stage 1 (V1 — Schoolbook Nested-Loop Convolution):** Directly initialize an output array `c = [0]*(2*n - 1)` and run a double loop: `for i in range(n): for j in range(n): c[i+j] += a[i] * b[j]`. Performs exactly n^2 multiplications and $(n-1)^2$ additions ($\Theta(n^2)$ time, $\Theta(1)$ extra space).
- **Stage 2 (V2 — Naive 4-Way Divide-and-Conquer Split):** Compute the four half-size products A_HB_H, A_HB_L, A_LB_H, and A_LB_L via **4 recursive calls** on arrays of size $n/2$, then shift (by x^{2m} and x^m) and add them in $\Theta(n)$ time.
  - *Why Stage 2 Fails to Speed Up:* Its recurrence is $T_2(n) = 4T_2(n/2) + \Theta(n)$, which still solves to $\Theta(n^{\log_2 4}) = \Theta(n^2)$!
- **Stage 3 (V3 — Karatsuba's 1960 Algebraic Trick: 3 Multiplications Instead of 4!):** Anatolii Karatsuba noticed that we do not need A_HB_L and A_LB_H separately — we only need their **sum**! By adding the halves *before* multiplying, we make only **3 recursive calls** of size $n/2$:

  $$P_1(x) = A_H(x)B_H(x), \quad P_2(x) = A_L(x)B_L(x), \quad P_3(x) = \big(A_H(x) + A_L(x)\big)\big(B_H(x) + B_L(x)\big).$$

  Since $P_3 = A_HB_H + A_HB_L + A_LB_H + A_LB_L$, subtracting P_1 and P_2 recovers the middle term $A_HB_L + A_LB_H = P_3 - P_1 - P_2$ for the price of two array subtractions:

  $$C(x) = P_1(x)x^n + \big(P_3(x) - P_1(x) - P_2(x)\big)x^{n/2} + P_2(x).$$

  Reducing 4 recursive branches to 3 drops the time complexity from $\Theta(n^2)$ to $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$!

Part A: Hand-Worked Karatsuba Trace & Multiplication Counts (Stages 1–3)

Consider two degree-3 polynomials ($n = 4$ coefficients each):

$$A(x) = 3x^3 + 2x^2 + 5x + 1 \text{ (a = [1, 5, 2, 3])}, \quad B(x) = 2x^3 + x^2 + 4x + 3 \text{ (b = [3, 4, 1, 2])}.$$

Q1. Hand-Worked Trace of Stage 1 vs. Stage 3 (Karatsuba):

- First compute $C(x) = A(x)B(x)$ using **Stage 1 (Schoolbook Convolution)** by listing all $4 \times 4 = 16$ products $a_i b_j$ and summing along each diagonal $i + j = k$ for $k = 0, 1, \ldots, 6$.
- Now split $A(x)$ and $B(x)$ at $m = 2$ into linear polynomials:

$$A_H(x) = 3x + 2, \quad A_L(x) = 5x + 1, \qquad B_H(x) = 2x + 1, \quad B_L(x) = 4x + 3.$$

  Execute **Stage 3 (Karatsuba's Algorithm)** by hand: compute $P_1(x) = A_H B_H$, $P_2(x) = A_L B_L$, and $P_3(x) = (A_H + A_L)(B_H + B_L)$ by applying Karatsuba's 3-multiply rule inside each of the three degree-1 products (performing only $3 \times 3 = 9$ scalar multiplications instead of 16). Finally, compute $P_3(x) - P_1(x) - P_2(x)$ and assemble $C(x) = P_1(x)x^4 + (P_3 - P_1 - P_2)x^2 + P_2(x)$, verifying that it matches your Stage 1 result!

Q2. Comparing Multiplication Counts & Big-Integer Carry Propagation:

- For $n = 2^k$ coefficients, Stage 2 performs $4^k = n^2$ scalar multiplications, whereas Stage 3 (Karatsuba) performs only $3^k = n^{\log_2 3} \approx n^{1.585}$ scalar multiplications. Evaluate both 4^k and 3^k at $k = 2$ ($n = 4$), $k = 4$ ($n = 16$), and $k = 10$ ($n = 1024$). How many multiplications does Karatsuba save at $n = 1024$?
- Suppose we use your polynomial product $C(x)$ from Question 1 to multiply the two 4-digit base-10 integers $A(10) = 3{,}251$ and $B(10) = 2{,}143$. Take your coefficient array $\texttt{c} = [c_0, c_1, \ldots, c_6]$ and perform the single right-to-left base-10 **carry propagation pass** ($\texttt{digit}_k = (c_k + \text{carry}) \bmod 10$, $\text{carry} = \lfloor (c_k + \text{carry})/10 \rfloor$) to obtain the exact integer $3{,}251 \times 2{,}143$.

Stages 4 & 5: Recursive Cooley-Tukey FFT → Iterative In-Place Bit-Reversal FFT

To multiply two polynomials of degree $n - 1$ even faster, choose $N = 2^m \geq 2n$ and pad both coefficient arrays with zeros to length N. The **Discrete Fourier Transform (DFT)** evaluates $A(x)$ at all N complex roots of unity $\omega_N^0, \omega_N^1, \ldots, \omega_N^{N-1}$.

Stage 4 (V4 — Recursive Cooley-Tukey Radix-2 FFT): Split the length-N array $A = [a_0, a_1, \ldots, a_{N-1}]$ into two arrays of length $N/2$ by **even vs. odd array indices**:

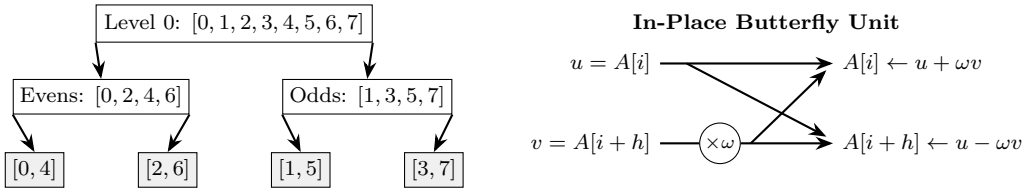
$$A_{\text{even}}(y) = a_0 + a_2 y + a_4 y^2 + \cdots + a_{N-2} y^{N/2-1}, \qquad A_{\text{odd}}(y) = a_1 + a_3 y + a_5 y^2 + \cdots + a_{N-1} y^{N/2-1},$$

so that $A(x) = A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2)$. Because $(\omega_N^k)^2 = \omega_{N/2}^k$ and $\omega_N^{k+N/2} = -\omega_N^k$, calling FFT recursively on A_{even} and A_{odd} (at the $(N/2)$-th roots of unity) lets us compute all N outputs in a single loop of $N/2$ **Butterfly Operations** for $k = 0, 1, \ldots, \frac{N}{2} - 1$:

$$y_k = y_k^{\text{even}} + \omega_N^k \cdot y_k^{\text{odd}}, \qquad y_{k+N/2} = y_k^{\text{even}} - \omega_N^k \cdot y_k^{\text{odd}}.$$

To interpolate back from point-values $\hat{c}_k = \hat{a}_k \cdot \hat{b}_k$ to coefficients (**Inverse FFT**), we run the exact same algorithm using the conjugate root $\omega_N^{-1} = e^{-2\pi i/N}$ and divide the final answers by N!

Stage 5 (V5 — Iterative In-Place Bit-Reversal FFT): Stage 4 allocates new sub-arrays `a[0::2]` and `a[1::2]` at every recursive call, costing extra memory and function-call overhead. Look closely at what recursive even/odd splitting does to the array indices $0, 1, \ldots, 7$ when $N = 8 = 2^3$:



Leaf Order: $[0, 4, 2, 6, 1, 5, 3, 7] = [000_2, 100_2, 010_2, 110_2, 001_2, 101_2, 011_2, 111_2]$ **(Bit-Reversed)**

In **Stage 5**, we first swap each element $A[j]$ with $A[\text{rev}_m(j)]$ in-place in $\Theta(N)$ time, and then run three nested loops bottom-up (for block sizes $s = 2, 4, 8, \ldots, N$) that overwrite pairs $(A[i], A[i + s/2])$ in-place using $\Theta(1)$ auxiliary memory!

Part B: Hand-Worked 4-Point FFT, Bit-Reversal Table & 5-Stage Summary (Stages 4–5)

Q3. Hand-Worked 4-Point FFT & Inverse FFT Multiplication: Let $P(x) = 3 + 2x$ and $Q(x) = 1 + 4x$. Their product $R(x) = P(x)Q(x)$ has degree 2, so we pad both coefficient arrays to power-of-two length $N = 4$: $\mathtt{p} = [3, 2, 0, 0]$ and $\mathtt{q} = [1, 4, 0, 0]$.

- Using the four 4-th roots of unity $\omega_4^0 = 1, \omega_4^1 = i, \omega_4^2 = -1, \omega_4^3 = -i$, trace the Stage 4 Cooley-Tukey butterfly split by hand to compute $\hat{\mathtt{p}} = \mathrm{FFT}_4([3, 2, 0, 0])$ and $\hat{\mathtt{q}} = \mathrm{FFT}_4([1, 4, 0, 0])$. (Show the even sub-transform, odd sub-transform, and butterfly combinations.)
- Multiply the point-value vectors elementwise to find $\hat{\mathtt{r}} = [\hat{p}_0\hat{q}_0, \hat{p}_1\hat{q}_1, \hat{p}_2\hat{q}_2, \hat{p}_3\hat{q}_3]$ (remembering that $i^2 = -1$).
- Apply the 4-point **Inverse FFT** by hand (using $\omega_4^{-1} = -i$ and dividing the final array by $N = 4$) to recover the exact coefficient array $\mathtt{r} = [r_0, r_1, r_2, r_3]$ of $R(x) = P(x)Q(x)$.

Q4. Bit-Reversal Index Table ($N = 8$) & 5-Stage Complexity Comparison Table:

- For $N = 8$ ($m = 3$ bits), write each index $j \in \{0, 1, 2, 3, 4, 5, 6, 7\}$ as a 3-bit binary string $(b_2b_1b_0)_2$, reverse the three bits to form $(b_0b_1b_2)_2$, and write down the resulting bit-reversed integer $\mathrm{rev}_3(j)$. Which pairs of indices $(j, \mathrm{rev}_3(j))$ actually need to be swapped in memory?
- Complete a summary table comparing **V1 (Schoolbook), V2 (4-Way Split), V3 (Karatsuba), V4 (Recursive FFT), and V5 (Iterative In-Place FFT)** in terms of asymptotic time complexity, auxiliary memory complexity, and exact integer vs. floating-point arithmetic.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five polynomial multiplication versions in Python:

- `poly_mul_v1_schoolbook(a, b)`: Nested convolution loop ($\Theta(n^2)$ time, $\Theta(1)$ auxiliary space).
- `poly_mul_v2_dc4(a, b)`: Naive 4-branch recursive divide-and-conquer ($\Theta(n^2)$ time).
- `poly_mul_v3_karatsuba(a, b, cutoff=16)`: 3-branch Karatsuba recursion with base-case cutoff ($\Theta(n^{1.585})$ time).
- `poly_mul_v4_fft_recursive(a, b)`: Recursive Cooley-Tukey FFT & IFFT using complex roots of unity ($\Theta(n \log n)$ time).
- `poly_mul_v5_fft_iterative(a, b)`: In-place bit-reversal iterative FFT & IFFT ($\Theta(n \log n)$ time, $\Theta(1)$ auxiliary space beyond the padded output buffer), rounding `round(val.real)` to recover integer coefficients.

Required Experiments & Plots:

1. **Differential Correctness & Precision Stress Test:** Verify that Versions 1–5 produce identical integer coefficient arrays on random polynomials of degrees $n \in \{8, 64, 512, 2048\}$ with coefficients in $[-100, 100]$. Then increase coefficient magnitudes to $10^6, 10^7, 10^8$ at $n = 4096$ to observe the exact threshold where 64-bit IEEE floating-point (`float64`, 53-bit mantissa) fails to round to the exact integer sum $\sum a_i b_{k-i}$.
2. **Runtime Crossover & Memory Plot:** Benchmark wall-clock runtime and peak memory (`tracemalloc`) across $n = 2^4, 2^5, \ldots, 2^{15}$ to find the practical crossover degree where Karatsuba beats Schoolbook and where Iterative FFT beats Karatsuba.

PROBLEM 8: LOSSLESS DATA COMPRESSION

From Fixed-Length Codes & RLE to Huffman Trees, Canonical Headers & LZ77

Core Data Structures & Concepts: Frequency Tables, Binary Min-Heaps, Prefix-Free Binary Trees, Sliding-Window Ring Buffers

Algorithmic Progression: V1 (Fixed-Length Block Code) → V2 (Run-Length Encoding RLE) → V3 (Min-Heap Greedy Huffman Tree) → V4 (Canonical Huffman Codebook) → V5 (LZ77 Sliding Window + Chained DEFLATE)

How Small Can a File Be Compressed Without Losing a Single Bit?

Every file on a computer — whether a novel, a DNA genome over {`A, C, G, T`}, or a Python program — is stored as a sequence of symbols drawn from a finite alphabet Σ of size $|\Sigma|$. In **lossless data compression**, our goal is to encode an N-character message into the shortest possible bitstring of `0`s and `1`s so that a decoder can reconstruct the **exact original message bit-for-bit**.

Two fundamental insights govern all modern compression utilities (`gzip`, `ZIP`, `PNG`, `zstd`):

1. **Skewed Symbol Frequencies (Statistical Coding):** In English text, `'e'` and `' '` occur constantly while `'z'` and `'q'` are rare. Assigning **short bitstrings to frequent symbols** and **longer bitstrings to rare symbols** shrinks the total bit count. In 1948, Claude Shannon proved that if symbol $s_i \in \Sigma$ appears with probability (relative frequency) $p_i = \text{count}(s_i)/N$, then no symbol-by-symbol prefix code can achieve an average length strictly smaller than the **Shannon Entropy**:

$$H(X) = \sum_{i=1}^{|\Sigma|} p_i \log_2\left(\frac{1}{p_i}\right) = -\sum_{i=1}^{|\Sigma|} p_i \log_2 p_i \quad \text{(bits per symbol)}.$$

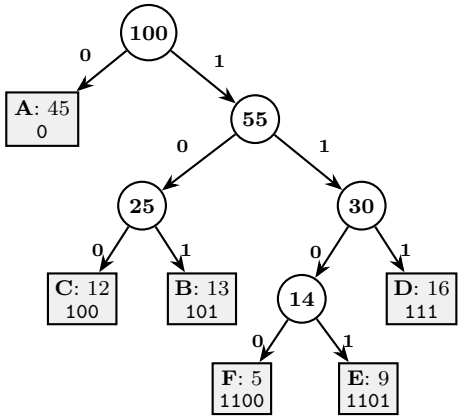
2. **Repeated Multi-Character Phrases (Dictionary Coding):** Real files contain whole repeated words and phrases (like `def` , `return` , or `ABRACADABRA`). By replacing a repeated phrase with a short pointer ‘(offset, length)‘ pointing backward into recently seen text, we can compress far below the single-symbol entropy $H(X)$!

Visual Primer: Prefix-Free Binary Trees (Huffman) & Sliding Windows (LZ77)

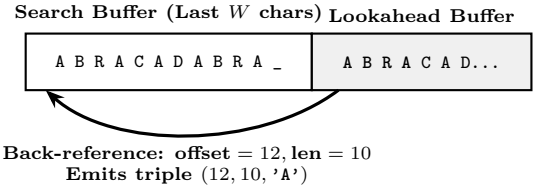
If we assign variable-length bitstrings to symbols, how does the decoder know where one codeword ends and the next begins without inserting separator spaces?

- **The Ambiguity Trap (Non-Prefix-Free Code):** Suppose `A` = `0`, `B` = `1`, and `C` = `01`. When the decoder sees the bitstring `01`, it is completely stuck: does `01` mean `"AB"` (`0` followed by `1`) or `"C"` (`01`)?
- **The Prefix-Free Rule (Binary Leaf Trie):** A code is **prefix-free** (instantaneous) if *no codeword is a prefix of any other codeword.* Geometrically, this means every symbol lives at a **leaf of a binary tree**, where stepping left appends `0` and stepping right appends `1`. To decode any bitstream, simply walk from the root down the tree bit-by-bit until you hit a leaf, emit that symbol, and jump back to the root!

1. Stage 3: Optimal Huffman Tree



2. Stage 5: LZ77 Sliding-Window Match



Stages 1, 2 & 3: Fixed-Length Codes → Run-Length Encoding → Min-Heap Huffman Coding

Suppose we want to compress a message of length N over an alphabet Σ of size $m = |\Sigma|$:

- **Stage 1 (V1 — Fixed-Length Block Binary Encoding):** Assign every symbol in Σ a binary integer of identical length $b = \lceil \log_2 m \rceil$ bits (e.g., ASCII uses 7 or 8 bits per character; a 6-letter alphabet uses $\lceil \log_2 6 \rceil = 3$ bits per character: `A` $= 000$, `B` $= 001, \ldots,$ `F` $= 101$).
  - *Limitation:* Ignores symbol frequencies completely! Rare `'F'` and common `'A'` both cost 3 bits.
- **Stage 2 (V2 — Run-Length Encoding / RLE):** Scan the string and replace each contiguous run of identical characters with a pair (count, char): for example, `AAAAABBBCCCCCC` becomes `(5,A)(3,B)(6,C)`.
  - *Limitation:* Works great on simple bitmap icons with long solid-color runs, but on normal text or alternating strings like `ABABABAB` where every run has length 1, storing `(1,A)(1,B)`... **expands** the file size!
- **Stage 3 (V3 — David Huffman's 1951 Min-Heap Greedy Prefix Tree):** Given symbol counts $f(s_1), \ldots, f(s_m)$, we want a binary prefix tree T minimizing total encoded bits $\sum_{i=1}^{m} f(s_i) \cdot \text{depth}_T(s_i)$. Huffman proved that we can build the exact optimal tree **bottom-up** using a **Binary Min-Heap (Priority Queue)** Q:
  1. Create m leaf nodes $(f(s_i), s_i)$ and insert all m leaves into the Min-Heap Q (`heapify` in $\Theta(m)$ time).
  2. While $|Q| > 1$ (repeats exactly $m - 1$ times):
     - Pop the two smallest-frequency trees $x =$ `extract_min`(Q) and $y =$ `extract_min`(Q).
     - Create a new internal parent node z with left child x (edge `0`), right child y (edge `1`), and combined frequency $f(z) = f(x) + f(y)$.
     - Push z back into the Min-Heap: `insert`(Q, z).
  3. The last remaining node in Q is the root of the optimal Huffman tree! Total build time: $\Theta(m \log m)$ using a binary min-heap (or $\Theta(m)$ using **Two FIFO Queues** Q_1, Q_2 if the m leaves are already sorted!).

Part A: Hand-Worked Huffman Tree Construction & Decoding Trace (Stages 1–3)

Consider a 100-character message over the 6-symbol alphabet $\Sigma =$ {`A, B, C, D, E, F`} with character counts:

`A` : 45, `B` : 13, `C` : 12, `D` : 16, `E` : 9, `F` : 5 (Total $N = 100$).

Q1. Hand-Worked Comparison of Stage 1, Stage 2, Shannon Entropy & Stage 3 (Huffman):

- **Stage 1 & Stage 2:** How many total bits does **Stage 1** ($\lceil \log_2 6 \rceil = 3$ bits/char) use for the 100-character message? Suppose **Stage 2 (RLE)** encodes each run as a 7-bit run-length integer (1–100)

plus a 3-bit character (10 bits per run): compare the total RLE bit size if (a) all identical letters are contiguous (`AAAAA...BBBBB...`, 6 runs total) versus (b) no two adjacent letters are equal (100 runs of length 1).

- **Shannon Lower Bound:** Compute the Shannon Entropy $H(X) = -\sum_{i=1}^{6} p_i \log_2 p_i$ (where $p_i = \text{count}_i/100$) in bits per symbol (to 3 decimal places). What is the theoretical minimum number of bits $100 \cdot H(X)$ needed by any symbol-by-symbol prefix code?
- **Stage 3 Min-Heap Trace:** Trace Huffman's algorithm by hand! Write down the state of the Min-Heap initially and after each of the 5 merge steps (placing the smaller-frequency child on the left `0` and larger on the right `1`). List the final binary codeword and bit-length l_i for each symbol `A, B, C, D, E, F`, and compute the total bits $\sum_{i=1}^{6} \text{count}_i \cdot l_i$ for the 100-character message.

Q2. Hand-Worked Bitstream Decoding Trace: Using your Stage 3 Huffman tree (`A` = `0`, `C` = `100`, `B` = `101`, `F` = `1100`, `E` = `1101`, `D` = `111`):

- Decode the 13-bit binary string `0101011001110` into characters by walking down the tree from the root bit by bit.
- Why are we able to decode this bitstring uniquely without any spaces or commas separating the codewords?

Stages 4 & 5: Canonical Huffman Codebooks → LZ77 Sliding-Window + Chained DEFLATE

Stage 4 (V4 — Canonical Huffman Codebook for Compact File Headers): To decompress a Huffman-encoded file, the decoder needs the tree! If we serialize explicit left/right pointers or bit-paths into the file header, the header itself wastes precious bytes. **Canonical Huffman Coding** observes that the decoder only needs to know the **bit-length l_i of each symbol's codeword** — just an array of m small integers! Both encoder and decoder then deterministically reconstruct the exact same prefix-free code in 3 lines:

1. Sort all symbols by (l_i, **symbol**) **ascending** (shortest bit-length first; ties broken alphabetically).
2. Assign the first sorted symbol the integer value code = 0, formatted as an $l_{(1)}$-bit binary string.
3. For each subsequent symbol $k = 2, \ldots, m$, increment code by 1 and bit-shift left by the length jump $\Delta l = l_{(k)} - l_{(k-1)}$:

$$\text{code} \leftarrow (\text{code} + 1) \ll \left(l_{(k)} - l_{(k-1)}\right) = (\text{code} + 1) \times 2^{l_{(k)} - l_{(k-1)}}.$$

Stage 5 (V5 — Lempel-Ziv LZ77 Sliding-Window Compression & Chained DEFLATE): Even an optimal Huffman code treats every character independently and can never use fewer than 1 bit per character. To compress repeated words and phrases, **LZ77** (Lempel & Ziv, 1977) slides a two-part window across the input string at cursor index p:

- **Search Buffer** $S[\max(0, p-W) \,..\, p-1]$: the last W already-processed characters (our dynamic dictionary!).
- **Lookahead Buffer** $S[p \,..\, \min(N, p+L) - 1]$: the next L characters waiting to be encoded.

At cursor p, LZ77 finds the longest prefix of the Lookahead Buffer (of length $\ell \geq 0$) that matches a substring starting offset $\in [1, W]$ characters behind p in the Search Buffer (Notice: the match is even allowed to *overlap* cursor p if a pattern repeats periodically!). It outputs the triple:

$$(\text{offset},\ \text{length}\ \ell,\ \text{next_char}\ S[p+\ell]) \qquad (\text{or } (0, 0, S[p]) \text{ if } S[p] \text{ has not appeared in the window}),$$

and advances the cursor by $p \leftarrow p + \ell + 1$. Finally, the **DEFLATE** algorithm (`gzip`/`ZIP`/`PNG`) chains **Stage 5 + Stage 4**: it runs LZ77 first to turn repeated phrases into (offset, length) tokens and literal characters, and then compresses those tokens using Canonical Huffman trees!

Part B: Canonical Codebook Reconstruction & LZ77 Sliding-Window Trace (Stages 4–5)

Q3. Hand-Worked Stage 4 (Canonical Huffman Codebook): Take the six codeword bit-lengths from your Stage 3 Huffman tree in Part A:

$$l_{\texttt{A}} = 1, \quad l_{\texttt{B}} = 3, \quad l_{\texttt{C}} = 3, \quad l_{\texttt{D}} = 3, \quad l_{\texttt{E}} = 4, \quad l_{\texttt{F}} = 4.$$

- Sort the 6 symbols by (l_i, symbol) and execute the Canonical Huffman assignment rule $\text{code} \leftarrow (\text{code}+1) \ll (l_{(k)} - l_{(k-1)})$ step-by-step by hand to determine the canonical binary codeword for each of `A, B, C, D, E, F`.
- Confirm that the resulting canonical code is prefix-free and uses the exact same 224 total bits as Stage 3, while requiring only the six length integers $[1, 3, 3, 3, 4, 4]$ in the file header!

Q4. Hand-Worked Stage 5 (LZ77 Sliding-Window Trace): Trace LZ77 encoding by hand on the 23-character string (where `_` is a space character at index 11):

$$S[0..22] = \texttt{ABRACADABRA_ABRACADABRA}$$

using a Search Buffer window of size $W = 12$ and Lookahead Buffer of size $L = 12$.

- Starting at cursor $p = 0$, write a table showing at each step: the cursor index p, the contents of the Search Buffer, the longest matching prefix in the Lookahead Buffer, the emitted triple $(\text{offset}, \text{length}, \text{next_char})$, and the new cursor position $p' = p + \text{length} + 1$.
- How many total triples are emitted to represent all 23 characters?

Q5. 5-Stage Complexity Comparison Table: Create a summary table across **V1 (Fixed-Length), V2 (RLE), V3 (Min-Heap Huffman), V4 (Canonical Huffman), and V5 (LZ77 + Canonical Huffman)** comparing time complexity, auxiliary space, header overhead, and ability to compress (a) skewed single characters vs. (b) long repeated phrases.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five lossless compression stages (both `compress` and `decompress`) in Python:

- `v1_fixed_length`: Packs each symbol into $\lceil \log_2 |\Sigma| \rceil$ bits.
- `v2_rle`: Encodes contiguous runs as $(\text{count}, \text{symbol})$ pairs.
- `v3_huffman_tree`: Builds the Huffman tree via `heapq` and decodes by walking tree pointers.
- `v4_huffman_canonical`: Serializes only the bit-length array l_i as the header and uses canonical codewords.
- `v5_lz77_deflate`: Sliding-window LZ77 ($W = 4096, L = 32$) followed by Canonical Huffman coding of literals, lengths, and offsets.

Required Experiments & Plots:

1. **Round-Trip Bit-Exact Verification:** Assert `decompress(compress(data)) == data` across all 5 versions on four test datasets: (a) a long repeated run (`"A"*5000 + "B"*5000`), (b) skewed random DNA ($p_{\texttt{A}} = 0.7, p_{\texttt{C}} = p_{\texttt{G}} = p_{\texttt{T}} = 0.1$), (c) source code / English prose (50 KB of Python files), and (d) uniform random bytes (`os.urandom(10000)`).
2. **Compression Ratio & Throughput Bar Charts:** For each of the 4 datasets, plot the achieved **bits per byte** (comparing against raw 8.0 bits/byte and the empirical Shannon entropy H_0) and encoding/decoding speed (MB/s).

PROBLEM 9: MODULAR ARITHMETIC & PUBLIC-KEY CRYPTOGRAPHY

From Naive Loops to Fast Exponentiation, Extended Euclid, Miller-Rabin & RSA

Core Data Structures & Concepts: Modular Integers $\mathbb{Z}_N$, Binary Bit-Scans, Bézout Tuple States, Primality Witness Chains

Algorithmic Progression: V1 (Naive $O(e)$ Power & $O(b)$ GCD) → V2 (Binary Squaring & Euclid's Remainder Loop) → V3 (Extended Euclid for Modular Inverse e^{-1}) → V4 (Trial Division → Fermat → Miller-Rabin Primality) → V5 (Complete RSA with CRT 4× Speedup)

Why Every Secure Internet Connection Depends on "Clock Arithmetic"

Whenever you visit an `https://` website, make a bank transfer, or connect via SSH, your computer must send a secret key across an open network where anyone might be eavesdropping. In 1977, Ron Rivest, Adi Shamir, and Leonard Adleman invented the **RSA Public-Key Cryptosystem**, built entirely on **modular ("clock") arithmetic** modulo a large integer N:

$$a \bmod N = \text{the remainder when } a \text{ is divided by } N \quad (\text{always in } \{0, 1, \ldots, N-1\}).$$

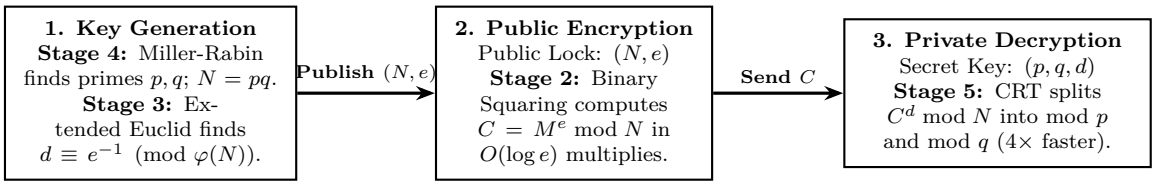
In RSA, Alice picks two giant secret prime numbers p and q (each 1024 bits long, or ≈ 308 decimal digits) and publishes their product $N = p \cdot q$ along with a public encryption exponent e. Anyone can lock a message $M \in \{0, \ldots, N-1\}$ into a ciphertext C via modular exponentiation:

$$\textbf{Encrypt:} \quad C = M^e \bmod N, \qquad\qquad \textbf{Decrypt:} \quad M = C^d \bmod N,$$

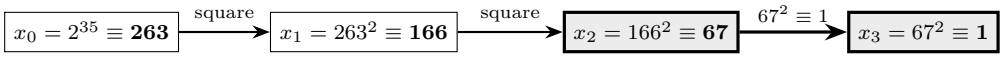
where the secret decryption exponent d satisfies $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$.

Here is the algorithmic catch: when N has $k = 2048$ bits, its numerical value is $N \approx 2^{2048} \approx 10^{616}$ (whereas the observable universe contains only $\approx 10^{80}$ atoms!). Any algorithm whose runtime is proportional to N or even $\sqrt{N} \approx 2^{1024}$ will never finish before the sun burns out. Every building block of cryptography must run in polynomial time in the **bit-length** $k = \log_2 N$ — that is, $O(\log N)$, $O(\log^2 N)$, or $O(\log^3 N)$!

Visual Primer: The RSA Cryptographic Pipeline & Miller-Rabin Witness Squaring



Stage 4: How Miller-Rabin Unmasks Carmichael Number $n = 561$ ($n - 1 = 560 = 2^4 \times 35$, Base $a = 2$)



$67 \not\equiv \pm 1 \pmod{561}$ **yet** $67^2 \equiv 1 \implies$ **Non-trivial square root of** 1 **detected (**561 **is composite)**

Stages 1, 2 & 3: From Linear Loops to Binary Squaring & Extended Euclid

Let us trace how three fundamental number-theoretic operations evolve from unusable $O(N)$ loops into blazingly fast $O(\log N)$ algorithms:

- **Stage 1 (V1 — Naive Linear Loops):**
  - *Naive Modular Power:* Multiply a by itself e times modulo N: `res = (res * a) % N`. Takes $\Theta(e)$ steps (impossible when $e \approx 2^{2048}$!).

– *Naive GCD Scan:* To check if $\gcd(a, b) = 1$, test every integer $d = \min(a, b)$ down to 1. Takes $\Theta(\min(a, b))$ divisions.

- **Stage 2 (V2 — Binary Exponentiation by Squaring & Euclid's Remainder GCD):**
  – *Binary Exponentiation ($O(\log e)$ multiplies):* Write the exponent e in binary. Since $a^{2k} = (a^k)^2$ and $a^{2k+1} = a \cdot (a^k)^2$, we repeatedly square the base (`base = (base * base) % N`) and multiply into `result` whenever the current lowest bit of e is 1 (`e & 1`), then shift $e \leftarrow e \gg 1$. For a 2048-bit exponent, this takes at most $2 \times 2048 = 4096$ multiplications instead of 2^{2048}!
  – *Euclid's GCD Loop ($O(\log b)$ divisions):* Since any common divisor of a and b also divides the remainder $r = a \bmod b = a - \lfloor a/b \rfloor b$, we have $\gcd(a, b) = \gcd(b, a \bmod b)$. Repeatedly replacing $(a, b) \leftarrow (b, a \bmod b)$ until $b = 0$ finishes in at most $2 \log_2 b$ steps!

- **Stage 3 (V3 — Extended Euclidean Algorithm for Modular Inverses $e^{-1} \bmod \varphi(N)$):** In RSA, we cannot divide by e using floating-point numbers; we need an integer d such that $e \cdot d \equiv 1 \pmod{\varphi(N)}$, which means $e \cdot d + \varphi(N) \cdot k = 1$ for some integer k. **Bézout's Identity** states that for any integers a, b, there exist integers x, y such that $ax + by = \gcd(a, b)$. The **Extended Euclidean Algorithm** maintains a 6-variable state $(r_{k-1}, r_k, x_{k-1}, x_k, y_{k-1}, y_k)$ initialized with $(a, b, 1, 0, 0, 1)$ so that $ax_k + by_k = r_k$ always holds! At each division $q_k = \lfloor r_{k-1}/r_k \rfloor$:

$$r_{k+1} = r_{k-1} - q_k r_k, \qquad x_{k+1} = x_{k-1} - q_k x_k, \qquad y_{k+1} = y_{k-1} - q_k y_k.$$

When $r_k = 1 = \gcd(a, b)$, the coefficient $y_k \bmod a$ (when $a = \varphi(N), b = e$) is the exact modular inverse $d = e^{-1} \bmod \varphi(N)$!

```python
def mod_pow_v2(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp & 1: # lowest bit is 1
            result = (result * base)
                % mod
        base = (base * base) % mod
        exp >>= 1   # shift right 1
            bit
    return result
```

```python
def extended_gcd_v3(a, b):
    r0, r1, x0, x1, y0, y1 = a, b, 1,
        0, 0, 1
    while r1 != 0:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return r0, x0, y0 # gcd, x, y: a*
        x + b*y = gcd
```

Part A: Hand-Worked Extended Euclid & Binary Squaring Traces (Stages 1–3)

Consider generating an RSA key pair with primes $p = 17$, $q = 23$, and public exponent $e = 3$.

Q1. Hand-Worked RSA Key Generation & Binary Exponentiation Trace:

- Compute the public modulus $N = p \cdot q$ and Euler's totient $\varphi(N) = (p - 1)(q - 1)$, and verify that $\gcd(e, \varphi(N)) = 1$.
- Trace **Stage 3 (`extended_gcd_v3`)** by hand on $(a, b) = (\varphi(N), e) = (352, 3)$ in a step-by-step table showing k, quotient q_k, remainder r_k, and Bézout coefficients x_k, y_k at every row until $r_k = 0$. Write down the resulting Bézout equation $352x + 3y = 1$, and compute the unique private decryption exponent $d = y \bmod 352 \in \{1, \ldots, 351\}$.
- Encrypt the plaintext message $M = 42$ by computing $C = 42^3 \bmod 391$ by hand. Then write your private exponent d in binary and trace **Stage 2 (`mod_pow_v2`)** row-by-row to compute $C^d \bmod 391$ and confirm it recovers $M = 42$!

Q2. Checking the Bézout Property & Comparing Operation Counts:

- **Bézout Property Check:** For each row k of your Extended GCD table from Question 1, plug the numbers (x_k, y_k, r_k) into $352x_k + 3y_k$ to verify numerically that it equals r_k at every step.
- **Operation Count Comparison:** Count how many modular multiplications **Stage 1** (Naive Loop) would perform to compute $189^{235} \bmod 391$ versus the exact number of squarings and `result` multiplications **Stage 2** performed in Question 1. How many multiplications does Stage 2 need at most when the exponent d is a 2048-bit integer?

Stages 4 & 5: Primality Testing (Miller-Rabin) → Complete RSA with CRT $4\times$ Speedup

Stage 4 (V4 — Primality Testing Evolution: Trial Division → Fermat → Miller-Rabin): How does Alice find two 1024-bit prime numbers p and q in the first place? By picking random odd 1024-bit integers and testing them for primality!

1. **Sub-Stage 4a (Trial Division up to $\sqrt{n}$):** Check if any integer $d \in [2, \lfloor\sqrt{n}\rfloor]$ divides n. Takes $O(\sqrt{n}) = O(2^{k/2})$ time — completely intractable for $k = 1024$ bits.
2. **Sub-Stage 4b (Fermat Primality Test):** By **Fermat's Little Theorem**, if n is prime, then $a^{n-1} \equiv 1 \pmod{n}$ for every $1 \leq a < n$. So we pick a random base a and compute $a^{n-1} \bmod n$ in $O(\log^3 n)$ time via Stage 2!
   - *Fatal Flaw:* Certain composite numbers called **Carmichael Numbers** (the smallest is $n = 561 = 3 \times 11 \times 17$) satisfy $a^{n-1} \equiv 1 \pmod{n}$ for *every* base a coprime to n, fooling Fermat's test 100% of the time!
3. **Sub-Stage 4c (Miller-Rabin Repeated-Squaring Witness Test):** If n is prime, the polynomial $x^2 - 1 = (x-1)(x+1) \equiv 0 \pmod{n}$ has **only two solutions**: the trivial square roots $x \equiv 1$ and $x \equiv -1 \equiv n-1 \pmod{n}$. To test if odd n is prime, factor out powers of 2 from $n-1$: write $n - 1 = 2^s \cdot u$ with u odd. For a random base $a \in [2, n-2]$, compute $x_0 = a^u \bmod n$, and then square s times: $x_i = x_{i-1}^2 \bmod n$ for $i = 1, \ldots, s$. If $x_s \not\equiv 1 \pmod{n}$ (Fermat failure) **OR** if at any squaring step $x_i \equiv 1 \pmod{n}$ while the previous value was $x_{i-1} \not\equiv 1$ and $x_{i-1} \not\equiv n-1$ (a **non-trivial square root of** 1), then n is **provably composite**! Every composite number (even a Carmichael number!) is caught with probability $\geq 3/4$ per random base a; after $t = 40$ rounds, the false-prime probability is $< 4^{-40} = 2^{-80}$.

Stage 5 (V5 — Complete RSA with Chinese Remainder Theorem / CRT $4\times$ Decryption Speedup): When decrypting $M = C^d \bmod N$, the private exponent d has the full bit-length of $N = pq$ (e.g., 2048 bits). Because the owner of the private key knows p and q, **Stage 5** uses Fermat's Little Theorem ($C^{p-1} \equiv 1 \pmod{p}$) and the **Chinese Remainder Theorem (CRT)** to split decryption into two half-size exponentiations (1024 bits each):

1. Precompute reduced exponents $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, and coefficient $q_{\text{inv}} = q^{-1} \bmod p$ (or $p_{\text{inv}} = p^{-1} \bmod q$) once during key generation.
2. Given ciphertext C, compute $M_p = (C \bmod p)^{d_p} \bmod p$ and $M_q = (C \bmod q)^{d_q} \bmod q$.
3. Recombine via CRT (Garner's formula): $h = \big((M_q - M_p) \cdot p_{\text{inv}}\big) \bmod q$, and return $M = M_p + p \cdot h$.

Part B: Unmasking Carmichael Number 561 & CRT $4\times$ Decryption Speedup (Stages 4–5)

Q3. Carmichael Numbers vs. The Miller-Rabin Witness Test on $n = 561$:

- The composite integer $n = 561 = 3 \times 11 \times 17$ is a Carmichael number: for every base a coprime to 561 (such as $a = 2$), it satisfies $a^{560} \equiv 1 \pmod{561}$. Explain in plain words why Sub-Stage 4b (the Fermat Primality Test, which only checks whether $a^{n-1} \equiv 1 \pmod{n}$) fails to detect that 561 is composite.
- Now trace **Stage 4c (Miller-Rabin)** by hand on $n = 561$ with base $a = 2$: write $n-1 = 560 = 2^4 \times 35$ ($s = 4, u = 35$). Starting from the initial power $x_0 = 2^{35} \bmod 561 = 263$, compute the four repeated squarings by hand:

  $$x_1 = x_0^2 \bmod 561, \qquad x_2 = x_1^2 \bmod 561, \qquad x_3 = x_2^2 \bmod 561, \qquad x_4 = x_3^2 \bmod 561.$$

  At which step does a number $x_{i-1} \not\equiv \pm 1 \pmod{561}$ square to $1 \pmod{561}$? Explain how this unmasks 561 as composite.

Q4. Hand-Worked Stage 5 (CRT Decryption) & 5-Stage Summary Table:

- Using your RSA key from Part A ($p = 17, q = 23, N = 391, e = 3, d = 235$) and ciphertext $C = 189$ (the encryption of $M = 42$), execute **Stage 5 (CRT Decryption)** by hand:
  (a) Compute $d_p = 235 \bmod 16$, $d_q = 235 \bmod 22$, and verify that $p_{\text{inv}} = 19$ satisfies $17 \times 19 \equiv 1 \pmod{23}$.
  (b) Compute $M_p = (189 \bmod 17)^{d_p} \bmod 17$ and $M_q = (189 \bmod 23)^{d_q} \bmod 23$.
  (c) Combine M_p and M_q via $M = M_p + 17 \cdot \big((M_q - M_p) \cdot p_{\text{inv}} \bmod 23\big)$ and verify that $M = 42$!
- **Why CRT Gives a 4× Speedup:** Modular exponentiation on a k-bit modulus takes time $T(k) = c \cdot k^3$. Plug $k/2$ into this formula to compute the ratio $\frac{T(k)}{2 \cdot T(k/2)}$ and explain why doing two half-sized exponentiations (mod p and mod q) is 4× faster than one full-sized exponentiation mod N.
- **5-Stage Summary Table:** Complete a table comparing **V1** through **V5** across: algorithm/data structure used, time complexity in terms of bit-length $k = \log_2 N$, and key practical limitation or advantage.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five stages in Python (using Python's native arbitrary-precision integers):

- `v1_naive_power_and_gcd(a, e, N)`: Linear $O(e)$ modular power and linear $O(\min(a, b))$ GCD scan.
- `v2_binary_pow_and_euclid(a, e, N)`: Bitwise $O(\log e)$ modular exponentiation and Euclid's remainder loop.
- `v3_extended_gcd_inv(e, phi)`: Iterative Extended Euclid returning $d = e^{-1} \bmod \varphi(N)$.
- `v4_primality_suite(n, k=40)`: Trial Division vs. Fermat Test vs. k-round Miller-Rabin Witness Test.
- `v5_rsa_crt(bits=1024)`: Complete RSA key generator, encryptor, standard decryptor, and CRT-accelerated decryptor.

Required Experiments & Plots:

1. **Carmichael Number Hunter ($n \leq$ 100,000):** Scan all odd integers $n \in [3, 100{,}000]$, using Trial Division as ground truth, to find all 7 Carmichael numbers below 100,000 ($561, 1105, 1729, 2465, 2821, 6601, 8911, \ldots, 75361$). For each Carmichael number, measure the fraction of coprime bases $a \in [2, n-2]$ that fool Fermat's test (100%) vs. the fraction that fool 1 round of Miller-Rabin (verify it is always $\leq 25\%$!).
2. **Standard vs. CRT Decryption Benchmark:** Generate random RSA keys of bit-lengths $k = \log_2 N \in \{256, 512, 1024, 2048\}$ and plot the wall-clock time of Standard Decryption ($C^d \bmod N$) vs. CRT Decryption, verifying the empirical 3.5×–4.0× speedup.

PROBLEM 10: SEQUENCE ALIGNMENT & EDIT DISTANCE

From Exponential Recursion to Needleman-Wunsch DP & Hirschberg's Linear-Space Algorithm

Core Data Structures & Concepts: Recursion Trees, 2D Dynamic Programming Tables, Rolling Two-Row Arrays, Divide-and-Conquer Midpoint Splits

Algorithmic Progression: V1 (Naive 3-Branch Recursion $\Theta(3^{m+n})$) $\rightarrow$ V2 (Top-Down Memoization Cache) $\rightarrow$ V3 (Bottom-Up 2D Needleman-Wunsch Table + Traceback) $\rightarrow$ V4 (Two-Row Rolling Array $\Theta(n)$ Space, Score Only) $\rightarrow$ V5 (Hirschberg's Divide-and-Conquer + Two-Row Full Alignment)

From Spell-Checkers and `git diff` to Human Genome Sequencing

How does a spell-checker know that `"teh"` is distance 2 from `"the"`? How does `git diff` find the minimal set of added and deleted lines between two versions of your code? And how do biologists compare two DNA strands $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ over {`A, C, G, T`} to pinpoint evolutionary mutations?

All three are instances of the **Global Sequence Alignment / Edit Distance (Levenshtein Distance)** problem: we want to place the characters of X and Y into columns (inserting gap characters `'-'` where needed) to minimize the total penalty of three edit operations:

- **Match ($x_i = y_j$):** Cost 0 (both characters align identically).
- **Mismatch / Substitution ($x_i \neq y_j$):** Cost $\alpha = 1$ (replace x_i with y_j).
- **Gap / Indel (Insertion or Deletion):** Cost $\delta = 1$ (align x_i with `'-'`, or align `'-'` with y_j).

In 1970, Saul Needleman and Christian Wunsch showed how to solve this in $\Theta(mn)$ time by filling an $(m+1)\times(n+1)$ Dynamic Programming (DP) table. However, there is a massive memory catch: when aligning two chromosome segments of length $m = n = 100{,}000$, storing an $m\times n$ table of 10^{10} integers requires 40 **Gigabytes of RAM**, crashing a standard computer! If we keep only the current and previous rows, memory drops to less than 1 Megabyte — but we lose the table needed to trace back the alignment! In Stage 5, you will master Dan Hirschberg's brilliant 1975 divide-and-conquer algorithm that recovers the **exact optimal alignment path** in $\Theta(mn)$ time using only $\Theta(m+n)$ linear memory.

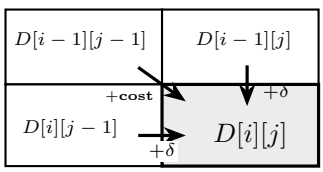
Visual Primer: The 3-Way Grid Recurrence & Hirschberg's Midpoint Split

Let $D[i][j]$ denote the minimum edit distance between the prefix $X[1..i]$ (the first i characters of X) and the prefix $Y[1..j]$ (the first j characters of Y). Consider the very last column of an optimal alignment between $X[1..i]$ and $Y[1..j]$. There are only **three possibilities**, corresponding to three incoming edges on a 2D grid:

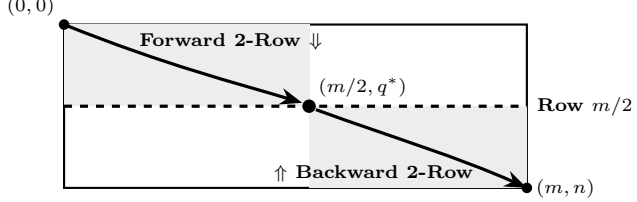
$$D[i][j] = \min \begin{cases} D[i-1][j-1] + \text{cost}(x_i, y_j) & (\textbf{Diagonal } \nwarrow\text{: match} = 0, \text{ mismatch } \alpha = 1), \\ D[i-1][j] + \delta & (\textbf{Vertical } \uparrow\text{: delete } x_i, \text{ gap cost } \delta = 1), \\ D[i][j-1] + \delta & (\textbf{Horizontal } \leftarrow\text{: insert } y_j, \text{ gap cost } \delta = 1), \end{cases}$$

with base cases $D[i][0] = i \cdot \delta$ (deleting all i characters of X) and $D[0][j] = j \cdot \delta$ (inserting all j characters of Y).

1. 3-Way DP Dependency



2. Stage 5: Hirschberg Midpoint Split



Shaded subproblems have total area $\frac{m}{2}q^* + \frac{m}{2}(n - q^*) = \frac{1}{2}mn$

Stages 1, 2 & 3: Naive Recursion → Top-Down Memoization → Bottom-Up 2D Table

Observe how our first three stages tame the time complexity from exponential $\Theta(3^{m+n})$ down to polynomial $\Theta(mn)$:

- **Stage 1 (V1 — Naive 3-Branch Recursive Alignment):** Directly translate the recurrence into a recursive function `edit_v1(i, j)` that calls `edit_v1(i-1, j-1)`, `edit_v1(i-1, j)`, and `edit_v1(i, j-1)`.
  - *Why Stage 1 Explodes:* Both `(i-1, j) -> (i-1, j-1)` and `(i, j-1) -> (i-1, j-1)` solve the exact same subproblem `(i-1, j-1)` from scratch! The recursion tree branches 3 ways at each step, taking $\Omega(3^{\min(m,n)})$ time.
- **Stage 2 (V2 — Top-Down Recursion with Memoization Cache):** Before recursing on (i, j), check if (i, j) is already stored in a lookup table `memo` (an $(m+1) \times (n+1)$ 2D array initialized to `-1`, or a hash dictionary). If `memo[i][j] != -1`, return it immediately in $O(1)$ time! Since there are only $(m+1)(n+1)$ distinct pairs (i, j) and each is computed once in $O(1)$ work, time drops to $\Theta(mn)$!
- **Stage 3 (V3 — Bottom-Up Needleman-Wunsch 2D DP Table + Traceback):** Eliminate recursion stack frames entirely by allocating the 2D array $D[0..m][0..n]$ and filling it iteratively row-by-row (`for i in range(1, m+1): for j in range(1, n+1)`). Once the table is filled, $D[m][n]$ holds the optimal edit distance, and we reconstruct the actual aligned strings by **walking backward** from $(i, j) = (m, n)$ to $(0, 0)$:
  - If $D[i][j] == D[i-1][j-1] + \text{cost}(x_i, y_j)$, prepend (x_i, y_j) and step diagonally to $(i-1, j-1)$.
  - Else if $D[i][j] == D[i-1][j] + \delta$, prepend $(x_i, \texttt{'-'})$ and step up to $(i-1, j)$.
  - Else prepend $(\texttt{'-'}, y_j)$ and step left to $(i, j-1)$.

Part A: Hand-Worked Needleman-Wunsch Table & Optimal Traceback (Stages 1–3)

Consider aligning the two 7-character DNA sequences ($m = 7, n = 7$):

$$X = \texttt{GATTACA}, \qquad Y = \texttt{GCATGCU},$$

under standard Levenshtein edit distance costs: **Match** $= 0$, **Mismatch** $\alpha = 1$, and **Gap** $\delta = 1$.

Q1. Overlapping Subproblems in Stage 1: Draw the top two levels of the Stage 1 recursion tree starting from $(i, j) = (7, 7)$ down to depth 2, and circle at least three duplicate calls that Stage 1 recomputes redundantly.

Q2. Hand-Worked 8×8 Needleman-Wunsch DP Table (Stage 3): Draw and fill out the complete 8×8 table $D[0..7][0..7]$ by hand (with rows labeled ϵ, `G, A, T, T, A, C, A` and columns labeled ϵ, `G, C, A, T, G, C, U`).

- What is the optimal edit distance $D[7][7]$ between `GATTACA` and `GCATGCU`?
- Trace back from cell $(7, 7)$ to $(0, 0)$ by following all valid parent arrows ($\nwarrow, \uparrow, \leftarrow$) that achieve the minimum in each cell. Write out at least **two distinct optimal alignments** of `GATTACA` and `GCATGCU`, showing for each alignment the exact count of matches, mismatches, insertions, and deletions (verifying their costs sum to $D[7][7]$).

Stages 4 & 5: Two-Row Rolling Array → Hirschberg's Linear-Space Full Alignment

Stage 4 (V4 — Two-Row Rolling Array Optimization: $\Theta(n)$ Memory, Score Only): Look at the recurrence for row i: computing $D[i][j]$ only ever reads from **row** $i-1$ ($D[i-1][j-1], D[i-1][j]$) and the **current row** i ($D[i][j-1]$)! It never looks at row $i-2$ or earlier. Therefore, we can throw away the 2D table and allocate just **two 1D arrays of length** $n+1$: `prev[0..n]` and `curr[0..n]`! After finishing row i, we swap `prev, curr = curr, prev`.

- *Memory Triumph:* Space drops from $\Theta(mn)$ (40 GB at $m = n = 10^5$) to $2(n+1)$ integers (≈ 800 KB)!

- *The Traceback Paradox:* When the loop finishes, `prev[n]` gives the exact optimal score $D[m][n]$, but all earlier rows have been overwritten — we have **lost the backpointers** needed to reconstruct the alignment path!

Stage 5 (V5 — Dan Hirschberg's 1975 Divide-and-Conquer + Two-Row Algorithm): How can we find the optimal alignment path in $\Theta(m+n)$ space without storing the 2D table? Hirschberg realized we only need to find **where the optimal path crosses the middle row** $m_{\text{mid}} = \lfloor m/2 \rfloor$:

1. Split X in half at $m_{\text{mid}} = \lfloor m/2 \rfloor$ into $X_{\text{top}} = X[1..m_{\text{mid}}]$ and $X_{\text{bot}} = X[m_{\text{mid}}+1..m]$.
2. **Forward 2-Row Sweep:** Run Stage 4 forward on (X_{top}, Y) to compute row m_{mid}: for each column $q \in \{0, \ldots, n\}$, $F[q]$ is the optimal cost of aligning $X[1..m_{\text{mid}}]$ with prefix $Y[1..q]$.
3. **Backward 2-Row Sweep:** Run Stage 4 on the **reversed** strings (X_{bot}^R, Y^R) to compute $B[q]$: the optimal cost of aligning the bottom half $X[m_{\text{mid}}+1..m]$ with the remaining suffix $Y[q+1..n]$.
4. **Find the Optimal Midpoint Column q^*:** Any global alignment path must cross row m_{mid} at some column $q \in \{0, \ldots, n\}$, paying $F[q]$ above row m_{mid} and $B[q]$ below row m_{mid}. Thus the optimal crossing column is simply:
$$q^* = \arg\min_{0 \le q \le n} \big(F[q] + B[q]\big)!$$
5. **Divide and Conquer:** Recursively solve the top-left subproblem $\big(X[1..m_{\text{mid}}],\, Y[1..q^*]\big)$ and the bottom-right subproblem $\big(X[m_{\text{mid}}+1..m],\, Y[q^*+1..n]\big)$ (using Stage 3 as the base case whenever $m \le 1$ or $n \le 1$), and concatenate their returned alignments!

Part B: Hand-Worked Hirschberg Midpoint Split & Memory Comparison (Stages 4–5)

Q3. Why Can't Stage 4 Just Store Alignment Strings Inside `curr[j]`? A student proposes avoiding Hirschberg's algorithm by modifying Stage 4 so that each entry of `curr[j]` stores both the integer score *and* the best alignment string of length $\Theta(i+j)$ found so far for $(X[1..i], Y[1..j])$. Explain what this does to the memory and time per cell of Stage 4, and why it defeats the purpose of the rolling array!

Q4. Hand-Worked Trace of Hirschberg's Midpoint Split (Stage 5): Execute the top level of Hirschberg's algorithm by hand on $X =$ `GATTACA` ($m = 7$) and $Y =$ `GCATGCU` ($n = 7$) at midpoint row $m_{\text{mid}} = \lfloor 7/2 \rfloor = 3$, which splits X into $X_{\text{top}} =$ `GAT` and $X_{\text{bot}} =$ `TACA`:

- Write down the Forward score array $F[0..7]$ (the edit distance between `GAT` and each prefix $Y[1..q]$; note this is just Row 3 of your table from Part A!).
- Compute the Backward score array $B[0..7]$ by aligning $X_{\text{bot}}^R =$ `ACAT` against $Y^R =$ `UCGTACG` (so $B[q]$ is the edit distance between `TACA` and the suffix $Y[q+1..7]$).
- Compute the sum array $S[q] = F[q] + B[q]$ for all $q \in \{0, 1, \ldots, 7\}$. Which column(s) q^* minimize $F[q] + B[q]$? Verify that $\min_q(F[q] + B[q]) = D[7][7]$ and that $(3, q^*)$ lies directly on the optimal traceback paths you found in Part A!

Q5. Concrete Memory Comparison & 5-Stage Summary Table:

- Suppose we align two DNA sequences of length $m = n = 50{,}000$, where each integer entry uses 4 bytes of RAM. Calculate how many integers (and how many Megabytes or Gigabytes of RAM) **Stage 3** allocates for its $(m+1) \times (n+1)$ table versus **Stage 4**'s two rows of length $n+1$.
- Complete a 5-stage summary table comparing **V1** through **V5** across: time complexity, space complexity, and whether the full optimal alignment strings can be reconstructed.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five sequence alignment stages in Python:

- `align_v1_recursive(X, Y)`: Pure 3-branch recursion ($\Theta(3^{m+n})$ time; test only on $m, n \le 10$).

- `align_v2_memo(X, Y)`: Top-down recursion with 2D memoization cache ($\Theta(mn)$ time, $\Theta(mn)$ space).
- `align_v3_needleman_wunsch(X, Y)`: Bottom-up 2D table + full alignment traceback ($\Theta(mn)$ time, $\Theta(mn)$ space).
- `align_v4_two_row_score(X, Y)`: Rolling two-row DP returning optimal score $D[m][n]$ in $\Theta(n)$ space.
- `align_v5_hirschberg(X, Y)`: Linear-space divide-and-conquer returning both optimal score and full aligned strings $(X_{\text{aligned}}, Y_{\text{aligned}})$ in $\Theta(mn)$ time and $\Theta(m+n)$ space.

Required Experiments & Plots:

1. **Automated Alignment Verifier:** On 100 random DNA string pairs of lengths $m, n \in [10, 80]$, verify that `align_v5_hirschberg` returns aligned strings of equal length that (a) strip gaps `'-'` to recover X and Y exactly, and (b) have total mismatch + gap cost equal to `align_v3_needleman_wunsch` and `align_v4_two_row_score`.
2. **Memory Wall vs. Linear-Space Benchmark:** For sequence lengths $m = n \in \{250, 500, 1000, 2000, 4000, 8000\}$, measure both wall-clock runtime and peak memory allocation (`tracemalloc`) for Versions 3, 4, and 5. Plot (a) Peak RAM (MB) vs. n (showing V3's quadratic $\Theta(n^2)$ memory explosion vs. V5's flat $\Theta(n)$ line) and (b) Runtime (seconds) vs. n (verifying $T_{\text{V5}}(n) \approx 2.0 \times T_{\text{V4}}(n)$).

PROBLEM 11: DISJOINT-SET UNION (UNION-FIND) & PERCOLATION

Dynamic Connectivity: From BFS Flood-Fill to Union by Rank, Path Compression & Percolation

Core Data Structures & Concepts: Component-ID Arrays, Parent-Pointer Up-Trees, Rank Bounds, Virtual Boundary Nodes

Algorithmic Progression: V1 (Graph BFS/DFS) → V2 (Quick-Find Array) → V3 (Naive Up-Trees) → V4 (Union by Rank) → V5 (Path Compression + Virtual Nodes)

Why Dynamic Connectivity Is One of Computer Science's Greatest Success Stories

Imagine you are managing a network of n elements (numbered $0, 1, \ldots, n-1$) that start completely disconnected. Over time, links arrive one by one in a live stream, and you must support two fundamental operations at blazing speed:

- `Union(u, v)`: Add a connection between u and v, merging their connected groups (sets) into one.
- `Connected(u, v)` (via `Find(u) == Find(v)`): Answer immediately whether u and v belong to the same connected group (even if only through a long chain of intermediate links).

This **Disjoint-Set Union (DSU)** or **Union-Find** data structure is the engine behind:

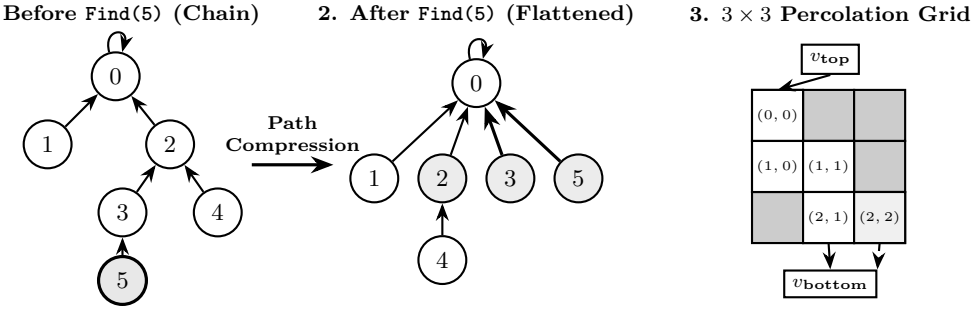
1. **Kruskal's Minimum Spanning Tree (MST) Algorithm:** Building the cheapest fiber-optic or road network connecting n cities by greedily adding the lightest edge that does not create a cycle (`Find(u) != Find(v)`).
2. **Image Processing & Medical Scans:** Labeling connected regions of pixels or 3D voxels in MRI scans.
3. **Physical Percolation & Phase Transitions:** Modeling how oil seeps through porous rock, how forest fires jump across trees, or how an $L \times L$ insulating grid suddenly conducts electricity the instant the fraction of open sites crosses a sharp mathematical threshold $p_c \approx 0.592746$.

As you will discover in this problem, evolving from a naive graph search to **Union by Rank with Path Compression** reduces the cost per operation from $\Theta(n)$ down to $O(\alpha(n))$—where $\alpha(n)$ grows so mind-bogglingly slowly that $\alpha(n) \leq 4$ for every input size n smaller than the number of atoms in the observable universe!

Visual Primer: Parent-Pointer Up-Trees, Path Compression & Virtual Grid Nodes

Instead of storing heavy pointers or adjacency lists, a Disjoint-Set forest represents *all* sets inside a **single flat integer array** `parent[0..n-1]`. Each element x points upward to its parent `parent[x]`. If `parent[r] == r`, then r is the **root** (canonical representative) of its entire tree!

- **To check if u and v are in the same set:** Follow `parent` pointers up from u to its root $r_u = \texttt{Find}(u)$, and from v to its root $r_v = \texttt{Find}(v)$. They are connected if and only if $r_u == r_v$.
- **To merge two sets (`Union(u, v)`):** Find their roots r_u and r_v. If $r_u \neq r_v$, simply make one root point to the other: `parent[r_u] = r_v`!



Look at the middle diagram: while walking up $5 \to 3 \to 2 \to 0$ during `Find(5)`, we already discovered that 0 is the root of 5, 3, and 2. **Path Compression** rewires `parent[5] = 0` and `parent[3] = 0` on the spot so future lookups on 5 or 3 finish in 1 hop!

The 5-Stage Dynamic Connectivity & Union-Find Evolution Ladder

Suppose we have n elements $\{0, 1, \ldots, n-1\}$ and process a stream of m operations (`Union` and `Connected`):

- **Stage 1 (V1): Explicit Adjacency-List Graph + Full BFS/DFS Flood-Fill per Query**
  - *Idea:* Store an undirected graph adjacency list `adj[0..n-1]`. `Union(u, v)` appends edge (u, v) in $O(1)$ time.
  - *Bottleneck:* Every `Connected(u, v)` query must run a fresh Breadth-First Search (BFS) from u to see if v is reachable, costing $\Theta(n + e)$ time *per query*! Across m queries, total runtime explodes to $\Theta(m(n + e))$.
- **Stage 2 (V2): Quick-Find Component-ID Array (`id[0..n-1]`)**
  - *Data Structure Upgrade:* Store a flat array `id[i]` (initialized to `id[i] = i`) holding the component number of element i. Now `Find(x)` simply returns `id[x]` in $\Theta(1)$ time!
  - *Bottleneck:* To execute `Union(u, v)`, if `old_id = id[u]` and `new_id = id[v]` differ, we must scan the **entire array** (`for i in range(n): if id[i] == old_id: id[i] = new_id`) to update every team member's badge. Thus every `Union` costs $\Theta(n)$, and $n - 1$ unions take $\Theta(n^2)$ time!
- **Stage 3 (V3): Naive Parent-Pointer Up-Trees (`parent[0..n-1]`)**
  - *Data Structure Upgrade:* Store `parent[i]` (initially `parent[i] = i`). `Find(x)` follows parent links up to the root (`while x != parent[x]: x = parent[x]`), and `Union(u, v)` links the two roots via a single assignment: `parent[Find(u)] = Find(v)`.
  - *Bottleneck:* If we repeatedly attach a tall tree under a single new node (e.g., `Union(0,1)`, `Union(1,2)`, ..., `Union(n-2,n-1)`), the tree degenerates into a stick of height $n - 1$, making `Find` take $\Theta(n)$ hops!
- **Stage 4 (V4): Weighted Union by Rank (or Size)**
  - *Algorithmic Fix:* Never attach a tall tree under a short tree! Maintain an array `rank[0..n-1]` (initialized to 0) recording an upper bound on each root's subtree height:

```
def union_by_rank(u, v):
    ru, rv = find(u), find(v)
    if ru == rv: return False
    if rank[ru] < rank[rv]:
        parent[ru] = rv          # shorter root attaches under taller root
    elif rank[ru] > rank[rv]:
        parent[rv] = ru
    else:
        parent[rv] = ru          # equal height: tie-break & increment rank
            by +1
        rank[ru] += 1
    return True
```

  - *Guarantee:* Because a tree's height only grows by $+1$ when it merges with another tree of *at least equal size*, every tree of rank k contains at least 2^k nodes. Thus tree height never exceeds $\lfloor \log_2 n \rfloor$, guaranteeing $O(\log n)$ worst-case time per `Find` and `Union`!
- **Stage 5 (V5): Path Compression + Virtual Boundary Nodes for Percolation**
  - *Algorithmic Fix:* Upgrade `find(x)` to flatten every node along the search path directly onto the root:

```
def find_compressed(x):
    root = x
    while root != parent[root]:
        root = parent[root]
    while x != root:             # Second pass: rewire every node directly
        to root!
```

```
        nxt = parent[x]
        parent[x] = root
        x = nxt
    return root
```

– *Percolation Virtual Node Trick:* On an $L \times L$ grid (with $N = L^2$ cells indexed $0 \dots L^2 - 1$), testing whether any open top-row cell connects to any open bottom-row cell would naively take $L \times L = L^2$ calls to `Connected`. By adding two extra elements $v_{\text{top}} = L^2$ (unioned with every open top-row cell) and $v_{\text{bottom}} = L^2 + 1$ (unioned with every open bottom-row cell), testing whether the entire grid percolates reduces to a **single call** `Connected(v_top, v_bottom)` in $O(\alpha(L^2))$ time!

Part A: Hand-Worked Traces Across the DSU Ladder & Kruskal's MST

Q1. Tracing V2 vs. V3 vs. V4 on a 6-Element Chain: Start with $n = 6$ isolated elements $\{0, 1, 2, 3, 4, 5\}$ and execute the 5 operations in order:

`Union(0, 1)`, `Union(1, 2)`, `Union(2, 3)`, `Union(3, 4)`, `Union(4, 5)`.

- **V2 (Quick-Find `id[]`):** Assume `Union(u, v)` changes all entries equal to `id[u]` into `id[v]`. Write the array `id[0..5]` after each of the 5 unions, and count the total number of array entries overwritten.
- **V3 (Naive Up-Tree `parent[]`):** Assume `Union(u, v)` sets `parent[Find(u)] = Find(v)` without rank checks. Write the final `parent[0..5]` array after all 5 unions. What is the height of the tree, and how many pointer hops does `Find(0)` require?
- **V4 (Union by Rank):** Using the `union_by_rank` code above (where equal-rank ties set `parent[rv] = ru` and increment `rank[ru]`), write the final `parent[0..5]` and `rank[0..5]` arrays after all 5 unions. What is the tree height now?

Q2. Building a Height-3 Binomial Tree & Tracing Path Compression (Stages 4 & 5): Starting with $n = 8$ elements $\{0, 1, \dots, 7\}$ under **Union by Rank alone (V4)**, execute:

- Round 1 (pairs): `Union(0,1)`, `Union(2,3)`, `Union(4,5)`, `Union(6,7)`.
- Round 2 (4-tuples): `Union(0,2)`, `Union(4,6)`.
- Round 3 (all 8): `Union(0,4)`.

Draw the resulting 8-node up-tree and write down `parent[0..7]` and `rank[0..7]`, verifying that node 7 sits at depth 3 ($7 \to 6 \to 4 \to 0$). Next, execute a **single call** to `find_compressed(7)` (Stage 5) and write the updated `parent[0..7]` array and the new depth of every node.

Q3. Kruskal's Minimum Spanning Tree (MST) Hand Trace: Consider a connected undirected graph $G = (V, E)$ on 7 vertices $\{A, B, C, D, E, F, G\}$ with 11 weighted edges:

$$(A, B, 7), \quad (A, D, 5), \quad (B, C, 8), \quad (B, D, 9), \quad (B, E, 7), \quad (C, E, 5),$$
$$(D, E, 15), \quad (D, F, 6), \quad (E, F, 8), \quad (E, G, 9), \quad (F, G, 11).$$

Sort all 11 edges in non-decreasing order of weight (breaking ties alphabetically by endpoints, e.g., $(A, D, 5)$ before $(C, E, 5)$, and $(A, B, 7)$ before $(B, E, 7)$). Trace Kruskal's algorithm step by step using Union-Find: for each edge (u, v, w), state `Find`(u), `Find`(v), whether the edge is **Accepted** (merged via `Union`) or **Rejected** (would form a cycle because `Find`$(u) ==$ `Find`(v)), and report the final 6 MST edges and their total weight.

Part B: The Percolation Backwash Bug & 5-Stage Complexity Table

Q4. Diagnosing & Fixing the Percolation "Backwash Bug" (Stage 5): Look at the 3×3 grid on the right side of the Visual Primer diagram above, where cells $(0, 0), (1, 0), (1, 1), (2, 1)$ form an open percolating path from the top row to the bottom row, and cell $(2, 2)$ at the bottom-right is *also* open, while $(0, 2)$ and $(1, 2)$ above it are blocked.

- Suppose we want to shade every **"full" cell** (an open cell that water pouring in from the top row $r = 0$ can actually reach) by checking `Connected(v_top, cell)`. Trace which cells belong to the same set as v_{top} when *both* v_{top} and v_{bottom} are wired into the same Union-Find structure. Why does `Connected(v_top, (2,2))` return `True` even though water from the top cannot physically reach $(2, 2)$?
- Explain how maintaining **two** Union-Find instances—`uf_perc` (with both v_{top} and v_{bottom} for testing `percolates()`) and `uf_full` (with v_{top} only, omitting v_{bottom}, for testing `is_full(r, c)`)—completely eliminates the Backwash Bug in $O(\alpha(L^2))$ time per step.

Q5. 5-Stage Complexity Comparison Table: Create a summary table comparing **V1 (Graph BFS)**, **V2 (Quick-Find Array)**, **V3 (Naive Up-Trees)**, **V4 (Union by Rank)**, and **V5 (Union by Rank + Path Compression)** across: (a) time per `Union`, (b) time per `Find`/`Connected`, (c) total time to process $n - 1$ unions + n connectivity queries on n elements, and (d) auxiliary space complexity.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five connectivity engines in Python and run two experiments:

1. **Experiment 1 (Adversarial Chain Scaling & Pointer-Hop Counter):** For $n \in \{500, 1000, 2000, 5000\}$, build a worst-case chain of $n - 1$ unions followed by n queries. Measure wall-clock runtime and average parent-pointer hops per `Find` across V1, V2, V3, V4, and V5. Verify that V4 caps depth at $\leq \log_2 n$ while V5 drops average hops per query to < 1.5.
2. **Experiment 2 (Monte Carlo Percolation Phase Transition on an $L \times L$ Lattice):** Using your V5 Union-Find (with the dual-UF backwash fix), simulate site percolation on an $L \times L$ grid for $L \in \{40, 80, 160\}$. Sweep open probability $p \in [0.45, 0.75]$ in steps of 0.01 (running 100 random trials per p) and plot the empirical percolation probability $\hat{\mathbb{P}}(\text{percolates} \mid p)$ versus p, verifying the sharp sigmoid phase transition at $p_c \approx 0.593$.

PROBLEM 12: BLOOM FILTERS & COUNT-MIN SKETCH

High-Speed Stream Filtering: From Exact Hash Sets to Probabilistic Bit Arrays & Frequency Sketches

Core Data Structures & Concepts: Bit Arrays, k-Hash Double Hashing, Saturating 4-Bit Counters, 2D Frequency Sketch Tables

Algorithmic Progression: V1 (Linear Log Scan) → V2 (Exact Hash Set) → V3 (1-Hash Bit Array) → V4 (k-Hash Bloom Filter) → V5 (Counting Bloom + Count-Min Sketch)

Why Trading 1% Certainty Saves 95% of Your Memory

Suppose you are building Google Chrome's Safe Browsing filter (which checks every URL you visit against millions of known phishing sites) or a storage engine like Apache Cassandra (which checks whether a user ID exists in an on-disk file before performing a slow disk read).

- If you store $n = 100{,}000{,}000$ URLs (average length $L = 64$ bytes) inside an exact hash table in RAM, the raw strings alone consume 6.4 GB of memory—far too big for a mobile phone browser or router cache!
- Yet 99.9% of the URLs a user visits are completely safe (*not* in the phishing list). What if we could build a tiny "bouncer" data structure using just $1.44 \log_2(1/\varepsilon)$ **bits per item** (about 10 bits ≈ 1.25 bytes per URL, shrinking 6.4 GB down to 120 MB!) that guarantees:
  1. **Zero False Negatives:** If a URL *is* dangerous ($x \in S$), the filter **always** says "Possibly Dangerous" (100% recall).
  2. **Tunable False Positive Rate ε:** If a URL is safe ($x \notin S$), the filter says "Definitely Safe" 99% of the time, and only 1% of the time raises a harmless false alarm that triggers a backup check!

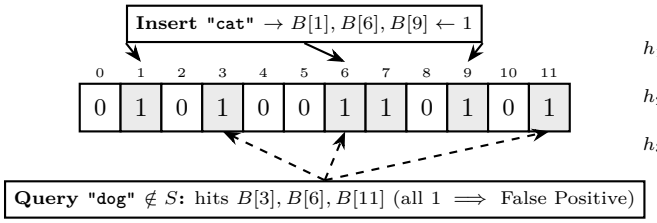
In 1970, Burton Bloom invented this exact structure—the **Bloom Filter**—and in 2003, Graham Cormode and S. Muthukrishnan generalized the idea into the **Count-Min Sketch** to track heavy-hitter item frequencies in massive high-speed streams using fixed sublinear memory.

Visual Primer: How a k-Hash Bloom Filter & a $d \times w$ Count-Min Sketch Work

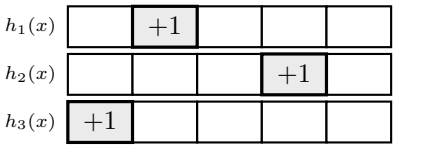
A **Bloom Filter** allocates a bit array $B[0..m-1]$ of m bits (all initialized to 0) and uses k hash functions $h_1, h_2, \ldots, h_k$, each mapping an item x to a bit position in $\{0, \ldots, m-1\}$:

- `Insert(x):` Compute all k indices $h_1(x), \ldots, h_k(x)$ and set those k bits to 1: $B[h_i(x)] \leftarrow 1$.
- `Contains(x):` Check all k bits $B[h_1(x)], \ldots, B[h_k(x)]$. If **any** bit is 0, x was *definitely never inserted* (return `False`). If **all** k **bits are** 1, return `True` (either x was inserted, or other items happened to set all k of x's bits!).

1. Bloom Filter ($m = 12$ bits, $k = 3$ hashes)



2. Count-Min Sketch ($d = 3 \times w = 5$)



Estimate $\hat{f}_x = \min_{1 \le j \le d} C[j][h_j(x)] \ge f_x$

The 5-Stage Stream Filtering & Sketching Evolution Ladder

Suppose a system ingests n distinct keys (each of length L bytes) from a fast stream and answers membership or frequency queries:

- **Stage 1 (V1): Unsorted Log / Dynamic Array + Linear Scan**
  - *Structure:* Append every arriving key to a list. `Insert` is $O(1)$, but every `Contains(x)` query scans all n strings in $\Theta(n \cdot L)$ time, and memory is $\Theta(n \cdot L)$ bytes.
- **Stage 2 (V2): Exact Hash Table Set (`set` / `unordered_set`)**
  - *Upgrade:* Hash each key into a bucket array for $O(L)$ average lookup time and 0% error.
  - *Bottleneck:* To resolve hash collisions exactly, the hash table **must store every original key string** in RAM ($\geq n \cdot L$ bytes plus pointer overhead).
- **Stage 3 (V3): Single-Hash Bit Array Filter ($k = 1$ Hash Function)**
  - *Upgrade:* Throw away the raw strings! Keep only an m-bit array $B[0..m-1]$ and set $B[h_1(x)] = 1$.
  - *Bottleneck:* With only $k = 1$ hash function, the false-positive rate decays only inversely with m: $\varepsilon_1 = 1 - (1 - 1/m)^n \approx 1 - e^{-n/m}$. Even at $m = 10n$ bits (10 bits/item), $\varepsilon_1 \approx 1 - e^{-0.1} \approx 9.5\%$ (nearly 1 in 10 safe queries triggers a false alarm!).
- **Stage 4 (V4): k-Hash Burton Bloom Filter + Kirsch-Mitzenmacher Double Hashing**
  - *Upgrade:* Probe k bit positions per item! A false positive now requires **all k bits** to be 1 simultaneously, dropping the false-positive rate exponentially to $\varepsilon(k) \approx (1 - e^{-kn/m})^k$. At $m/n = 10$ bits/item with optimal $k^* = 7$, the false-positive rate plunges from 9.5% down to 0.82% (11× fewer false alarms in the exact same memory!).
  - *Kirsch-Mitzenmacher Speedup:* Instead of computing k slow independent hash functions on string x, compute just **two** hashes $h_1(x), h_2(x)$ and generate all k indices via simple modular arithmetic:

$$g_i(x) = \big(h_1(x) + i \cdot h_2(x)\big) \bmod m \qquad \text{for } i = 0, 1, \ldots, k-1.$$

- **Stage 5 (V5): 4-Bit Counting Bloom Filter (Deletions) & $d \times w$ Count-Min Sketch (Frequencies)**
  - *Supporting Deletions:* You cannot delete x from a standard Bloom filter by setting $B[h_i(x)] \leftarrow 0$, because another element $y \in S$ might share that bit! Replacing each bit with a **4-bit counter** (0..15) lets `Insert(x)` increment $C[h_i(x)] \leftarrow \min(15, C[h_i(x)] + 1)$ and `Delete(x)` decrement $C[h_i(x)] \leftarrow C[h_i(x)] - 1$ safely.
  - *Streaming Frequencies (Count-Min Sketch):* To count how many times x appears in a stream of N total arrivals, use a $d \times w$ integer table $C[0..d-1][0..w-1]$ where row j has its own hash function $h_j : \mathcal{U} \to \{0, \ldots, w-1\}$. On arrival of (x, Δ), set $C[j][h_j(x)]$ += Δ for every row j, and estimate $\hat{f}_x = \min_{0 \leq j < d} C[j][h_j(x)]$.

Part A: Hand-Worked Traces of Bloom Filters, Deletions & Count-Min Sketch

Q1. Hand-Worked Bloom Filter Trace (Stage 4): Consider a Bloom filter with bit array $B[0..15]$ of size $m = 16$ bits (initially all 0) and $k = 3$ hash functions:

$$h_1(x) = (3x + 1) \bmod 16, \qquad h_2(x) = (5x + 3) \bmod 16, \qquad h_3(x) = (7x + 2) \bmod 16.$$

- Insert the three elements of $S = \{2, 5, 15\}$ one by one by hand. For each element, list its three hash indices $(h_1(x), h_2(x), h_3(x))$ and write down the full 16-bit array $B[0..15]$ after all three insertions. How many of the 16 bits are now equal to 1?
- Now test three queries by hand: $q_1 = 5$, $q_2 = 8$, and $q_3 = 4$. For each query, compute $(h_1(q), h_2(q), h_3(q))$, check the corresponding bits in B, and classify the result as a **True Positive**, **True Negative**, or **False Positive**. For the False Positive, identify exactly which previously inserted elements of S caused each of its three bit positions to be 1!

Q2. Why Naive Deletion Fails vs. Counting Bloom Filter (Stage 5): Suppose that after inserting $S = \{2, 5, 15\}$ in Question 1, a user tries to delete $x = 2$ from the standard 1-bit Bloom filter by setting $B[h_1(2)] = B[7] \leftarrow 0$, $B[h_2(2)] = B[13] \leftarrow 0$, and $B[h_3(2)] = B[0] \leftarrow 0$.

- What happens when we subsequently query $x = 5$ (which is *still* in the set)? Why does this violate the sacred "Zero False Negatives" guarantee of a Bloom filter?

- Trace the same sequence ($\texttt{Insert}(2), \texttt{Insert}(5), \texttt{Insert}(15), \texttt{Delete}(2), \texttt{Query}(5)$) using a **Counting Bloom Filter** $C[0..15]$ to show how $x = 5$ remains properly detected.

Q3. Hand-Worked Count-Min Sketch Trace (Stage 5): Consider a $d \times w = 2 \times 4$ Count-Min Sketch table $C[0..1][0..3]$ (initialized to zeros) with row hash functions:

$$h_0(x) = x \bmod 4, \qquad h_1(x) = \lfloor x/2 \rfloor \bmod 4.$$

Process the 5-element arrival stream $[3, 7, 3, 1, 3]$ (so true frequencies are $f_3 = 3, f_7 = 1, f_1 = 1$). Write out the 2×4 matrix C after all 5 arrivals, and compute the Count-Min estimates $\hat{f}_3 = \min(C[0][h_0(3)], C[1][h_1(3)])$ and $\hat{f}_7 = \min(C[0][h_0(7)], C[1][h_1(7)])$. Explain how taking the minimum across the two rows removes the collision between 3 and 7 that occurred in Row 0!

Part B: Practical Bloom Filter Sizing & 5-Stage Complexity Table

Q4. Concrete Engineering Sizing of a Bloom Filter & Count-Min Sketch Property (Stages 4–5): A standard Bloom filter achieves its lowest false-positive rate when roughly 50% of its m bits are set to 1, which gives two simple engineering sizing formulas for n inserted items and target false-positive rate ε:

$$\text{Bits per item: } \frac{m}{n} \approx 1.44 \log_2\left(\frac{1}{\varepsilon}\right), \qquad \text{Optimal hash count: } k^* \approx 0.693 \left(\frac{m}{n}\right).$$

- Using these two formulas, calculate the required bits per item m/n, optimal integer hash count k^*, and total filter memory in Megabytes for $n = 10{,}000{,}000$ URLs at a target false-positive rate $\varepsilon = 0.01$ (1%). Compare this memory footprint against storing 10,000,000 raw URL strings of length $L = 64$ bytes each in a Stage 2 exact hash set!
- In a Stage 5 Count-Min Sketch where all item counts are positive ($\Delta > 0$), explain in plain words why a bucket counter $C[j][h_j(x)]$ can never be *smaller* than the true frequency f_x (so $\hat{f}_x \geq f_x$ always holds).

Q5. 5-Stage Complexity & Space Comparison Table: Construct a comparison table across **V1 (Unsorted Array)**, **V2 (Exact Hash Set)**, **V3 (1-Hash Bit Array)**, **V4 (k-Hash Bloom Filter)**, and **V5 (Counting Bloom / Count-Min Sketch)** comparing: insert time, query time, memory footprint for n items of byte-length L at $\varepsilon = 1\%$, false positive rate, false negative rate, and deletion support.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five stages in Python (using `bytearray` or integer bitmasks for the bit array and Kirsch-Mitzenmacher double hashing with `hashlib.md5` or polynomial rolling hashes):

1. **Experiment 1 (The U-Shaped False-Positive Curve vs. Number of Hashes k):** Fix $n = 50{,}000$ inserted strings and $m = 10n = 500{,}000$ bits ($c = m/n = 10$ bits/item). Sweep $k \in \{1, 2, 3, \ldots, 15\}$ (where $k = 1$ is Stage 3 and $k > 1$ is Stage 4). For each k, test 50,000 unseen query strings, plot the empirical false-positive rate alongside the theoretical curve $(1 - e^{-k/10})^k$, and verify that $k^* = 7$ achieves the minimum (≈ 0.0082) while both too few hashes ($k = 1$, too many collisions) and too many hashes ($k = 15$, bit array saturates with 1s) increase errors!
2. **Experiment 2 (Memory Compression & Zipfian Heavy-Hitter Stream):** Compare the RAM footprint of V1, V2, and V4 storing 100,000 URLs. Then generate a skewed Zipfian stream of $N = 200{,}000$ tokens and plot the empirical frequency estimation error $|\hat{f}_x - f_x|/N$ of your Stage 5 Count-Min Sketch as table width w increases.

PROBLEM 13: MATRIX COMPRESSION & LOW-RANK SVD

From Dense Matrices to Alternating Power Iteration, Hotelling Deflation & Truncated SVD

Core Data Structures & Concepts: 2D Dense Arrays, Gram Matrices, Left/Right Singular Vectors, Factored Rank-k Outer Products

Algorithmic Progression: V1 (Full Dense Storage) → V2 (Gram A^TA Power Iteration) → V3 (Matrix-Free Alternating Power) → V4 (Hotelling Deflation) → V5 (Truncated Rank-k SVD)

How Do You Compress a Million-Pixel Image or a Billion-Parameter AI Model?

A grayscale photograph of size $m \times n = 1000 \times 1000$ is simply a 2D table (matrix) $A \in \mathbb{R}^{1000\times 1000}$ containing 1,000,000 brightness numbers. Similarly, a layer inside a Large Language Model (LLM) is a massive weight matrix $W \in \mathbb{R}^{4096\times 4096}$ containing 16.7 million numbers.

Must we store all $m \times n$ numbers to represent the matrix accurately? Suppose a matrix A happened to be a **rank-1 outer product** of a single column vector $\mathbf{u} \in \mathbb{R}^m$ and a single row vector $\mathbf{v}^T \in \mathbb{R}^{1\times n}$ scaled by $\sigma > 0$:

$$A = \sigma \mathbf{u}\mathbf{v}^T = \sigma \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{pmatrix} \begin{pmatrix} v_1 & v_2 & \dots & v_n \end{pmatrix} = \sigma \begin{pmatrix} u_1v_1 & u_1v_2 & \dots & u_1v_n \\ u_2v_1 & u_2v_2 & \dots & u_2v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_mv_1 & u_mv_2 & \dots & u_mv_n \end{pmatrix}.$$

Instead of storing $m \times n = 1{,}000{,}000$ numbers, we only need to store σ, the m entries of $\mathbf{u}$, and the n entries of $\mathbf{v}$—just $1 + 1000 + 1000 = \mathbf{2{,}001}$ numbers (a 500× compression!).

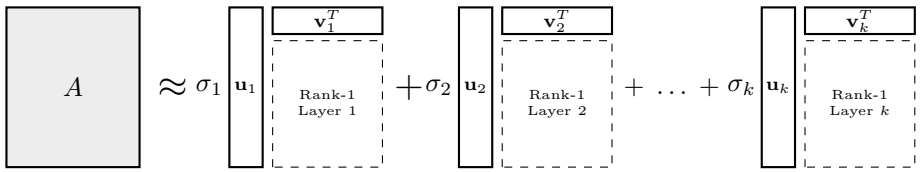
While real-world images and AI weight matrices are not rank 1, the **Singular Value Decomposition (SVD)** proves that *every* matrix $A \in \mathbb{R}^{m\times n}$ is a sum of sorted rank-1 outer products:

$$A = \sigma_1 \mathbf{u}_1\mathbf{v}_1^T + \sigma_2 \mathbf{u}_2\mathbf{v}_2^T + \cdots + \sigma_r \mathbf{u}_r\mathbf{v}_r^T \qquad (\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0),$$

where the orthonormal columns $\mathbf{u}_i \in \mathbb{R}^m$ and $\mathbf{v}_i \in \mathbb{R}^n$ capture the most important patterns first! Keeping only the top $k \ll r$ terms gives the **provably best rank-k approximation** A_k in existence.

Visual Primer: Sum of Rank-1 Outer Products & Factored Storage (U_k, Σ_k, V_k^T)

Every term $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ in the SVD is a "layer" (an $m \times n$ rank-1 matrix) weighted by its **singular value** $\sigma_i \geq 0$. Because the singular values $\sigma_1 \geq \sigma_2 \geq \dots$ of real images and data matrices decay rapidly toward zero, the first k layers capture almost 100% of the visual energy!



$m \times n$ (mn floats)

Factored Storage: k columns $\mathbf{u}_i$ (km) + k scalars σ_i (k) + k rows $\mathbf{v}_i^T$ (kn) = $k(m+n+1)$ floats

---

The 5-Stage Matrix Compression & SVD Computation Ladder

How do we actually *compute* the singular values σ_i and singular vectors $\mathbf{u}_i, \mathbf{v}_i$ of an $m \times n$ matrix A (assuming $m \geq n$), and how should we store the compressed representation?

- **Stage 1 (V1): Full Dense $m \times n$ Storage & Naive Subsampling**

– *Baseline:* Storing A directly requires mn floats. Naively throwing away every second row/column to save memory destroys sharp edges and text because it ignores the global structure of A.

- **Stage 2 (V2): Explicit Gram Matrix $G = A^TA$ + Single-Vector Power Iteration**
  – *Mathematical Link:* Because $A\mathbf{v}_i = \sigma_i\mathbf{u}_i$ and $A^T\mathbf{u}_i = \sigma_i\mathbf{v}_i$, multiplying the first equation on the left by A^T gives:
  $$(A^TA)\mathbf{v}_i = A^T(\sigma_i\mathbf{u}_i) = \sigma_i^2\mathbf{v}_i.$$
  Thus the right singular vectors $\mathbf{v}_i$ of A are simply the **orthonormal eigenvectors** of the symmetric $n \times n$ Gram matrix $G = A^TA$, with eigenvalues $\lambda_i = \sigma_i^2 \geq 0$! We can find $\mathbf{v}_1$ by forming $G = A^TA$ and repeatedly multiplying $\mathbf{v}^{(t+1)} = G\mathbf{v}^{(t)}/\|G\mathbf{v}^{(t)}\|_2$.
  – *Two Severe Bottlenecks:* (1) Explicitly multiplying A^TA costs $\Theta(mn^2)$ time up front; (2) Squaring the singular values ($\lambda_i = \sigma_i^2$) **squares the condition number** $\kappa(A^TA) = (\sigma_1/\sigma_r)^2$, causing catastrophic floating-point roundoff loss on small singular values!

- **Stage 3 (V3): Matrix-Free Two-Sided Alternating Power Iteration**
  – *Algorithmic Fix:* Never form A^TA! Instead, alternate matrix-vector products with A and A^T:
  $$\tilde{\mathbf{u}} \leftarrow A\mathbf{v}^{(t)}, \quad \mathbf{u}^{(t+1)} \leftarrow \frac{\tilde{\mathbf{u}}}{\|\tilde{\mathbf{u}}\|_2}, \qquad \tilde{\mathbf{v}} \leftarrow A^T\mathbf{u}^{(t+1)}, \quad \sigma_1^{(t+1)} \leftarrow \|\tilde{\mathbf{v}}\|_2, \quad \mathbf{v}^{(t+1)} \leftarrow \frac{\tilde{\mathbf{v}}}{\sigma_1^{(t+1)}}.$$
  Each iteration performs just two matrix-vector products ($A\mathbf{v}$ and $A^T\mathbf{u}$) in $\Theta(mn)$ time and $\Theta(m+n)$ extra space, without ever squaring entries in memory!

- **Stage 4 (V4): Sequential Top-k Extraction via Hotelling Rank-1 Deflation**
  – *Upgrade:* Once Stage 3 converges to the dominant triplet $(\sigma_1, \mathbf{u}_1, \mathbf{v}_1)$, how do we find the *second* triplet $(\sigma_2, \mathbf{u}_2, \mathbf{v}_2)$? We subtract out the rank-1 layer we just found (**Hotelling Deflation**):
  $$A^{(1)} \leftarrow A - \sigma_1\mathbf{u}_1\mathbf{v}_1^T,$$
  and run Stage 3 again on $A^{(1)}$! Repeating this k times ($A^{(j)} = A^{(j-1)} - \sigma_j\mathbf{u}_j\mathbf{v}_j^T$) extracts the top k singular triplets $(\sigma_1, \mathbf{u}_1, \mathbf{v}_1), \ldots, (\sigma_k, \mathbf{u}_k, \mathbf{v}_k)$ in $\Theta(k \cdot T \cdot mn)$ time.

- **Stage 5 (V5): Truncated Rank-k Factored Data Structure** (U_k, Σ_k, V_k^T)
  – *Data Structure Upgrade:* Store the compressed matrix as three compact arrays: $U_k \in \mathbb{R}^{m\times k}$ (columns $\mathbf{u}_1..\mathbf{u}_k$), $\boldsymbol{\sigma}_k \in \mathbb{R}^k$, and $V_k^T \in \mathbb{R}^{k\times n}$ (rows $\mathbf{v}_1^T..\mathbf{v}_k^T$). Total memory drops from mn floats to $k(m+n+1)$ floats, and multiplying the compressed matrix by any vector $\mathbf{x} \in \mathbb{R}^n$ via $U_k(\boldsymbol{\sigma}_k \odot (V_k^T\mathbf{x}))$ takes only $\Theta(k(m+n))$ time instead of $\Theta(mn)$!

Part A: Hand-Worked 3×2 SVD, Rank-1 Approximation & Alternating Power Trace

Consider the concrete 3×2 matrix:
$$A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

Q1. Exact SVD by Hand (Stage 2 Theory):

- Compute the 2×2 symmetric Gram matrix $G = A^TA$ by hand.
- Solve the characteristic equation $\det(G - \lambda I) = 0$ to find the eigenvalues $\lambda_1 > \lambda_2 > 0$ and the exact singular values $\sigma_1 = \sqrt{\lambda_1}$ and $\sigma_2 = \sqrt{\lambda_2}$.
- Find the normalized right singular vectors $\mathbf{v}_1, \mathbf{v}_2 \in \mathbb{R}^2$ (unit eigenvectors of G), and compute the left singular vectors $\mathbf{u}_1 = \frac{1}{\sigma_1}A\mathbf{v}_1 \in \mathbb{R}^3$ and $\mathbf{u}_2 = \frac{1}{\sigma_2}A\mathbf{v}_2 \in \mathbb{R}^3$.

Q2. Best Rank-1 Approximation, Hotelling Deflation & Frobenius Error (Stages 4 & 5):

- Using your $\sigma_1, \mathbf{u}_1, \mathbf{v}_1$ from Question 1, compute the 3×2 rank-1 approximation matrix $A_1 = \sigma_1\mathbf{u}_1\mathbf{v}_1^T$ by hand. (Notice that all square roots cancel cleanly!)
- Compute the Hotelling deflated matrix $A^{(1)} = A - A_1$ by hand and verify that $A^{(1)}$ is identical to the second rank-1 layer $\sigma_2\mathbf{u}_2\mathbf{v}_2^T$.

- Compute the exact Frobenius norm error $\|A - A_1\|_F = \sqrt{\sum_{i=1}^{3}\sum_{j=1}^{2}(A_{ij} - (A_1)_{ij})^2}$ by summing the squares of the 6 entries of $A - A_1$, and confirm that $\|A - A_1\|_F = \sigma_2$!

Q3. Tracing Matrix-Free Alternating Power Iteration (Stage 3): Starting from the initial unit vector $\mathbf{v}^{(0)} = \begin{pmatrix}1\\0\end{pmatrix}$, trace **one full iteration** of Stage 3 on A by hand:

- Compute $\tilde{\mathbf{u}} = A\mathbf{v}^{(0)}$, its Euclidean norm $\|\tilde{\mathbf{u}}\|_2$, and the normalized vector $\mathbf{u}^{(1)} = \tilde{\mathbf{u}}/\|\tilde{\mathbf{u}}\|_2$.
- Compute $\tilde{\mathbf{v}} = A^T\mathbf{u}^{(1)}$, the singular value estimate $\sigma_1^{(1)} = \|\tilde{\mathbf{v}}\|_2$, and the updated unit vector $\mathbf{v}^{(1)} = \tilde{\mathbf{v}}/\sigma_1^{(1)}$. Compare $\sigma_1^{(1)}$ and $\mathbf{v}^{(1)}$ against the exact $\sigma_1 = \sqrt{45} \approx 6.7082$ and $\mathbf{v}_1 = (1/\sqrt{2}, 1/\sqrt{2})^T \approx (0.7071, 0.7071)^T$!

Part B: Frobenius Energy Retention & Memory Compression Crossover

Q4. Concrete Energy Retention & Memory Compression Calculation (Stages 4 & 5):

- The fraction of total Frobenius "energy" retained by a rank-k SVD approximation is given by $\frac{\sigma_1^2+\cdots+\sigma_k^2}{\sigma_1^2+\cdots+\sigma_r^2}$. Using $\sigma_1 = \sqrt{45}$ and $\sigma_2 = \sqrt{5}$ from Part A, calculate the exact percentage of total energy captured by the rank-1 matrix A_1.
- Storing an $m \times n$ matrix densely (Stage 1) takes mn floats, whereas storing the Stage 5 factored form $(U_k, \boldsymbol{\sigma}_k, V_k^T)$ takes $k(m+n+1)$ floats. For a 1000×1000 grayscale image compressed at rank $k = 25$, calculate how many floats Stage 1 stores versus Stage 5, and compute the compression ratio $\frac{mn}{k(m+n+1)}$. Above what integer rank $k_{\max}$ would the factored form actually use *more* floats than the original 1000×1000 image?

Q5. 5-Stage Complexity & Storage Comparison Table: Fill out a comparison table across **V1–V5** comparing: construction time, storage space (floats), matrix-vector multiply time ($A\mathbf{x}$), and Frobenius approximation error.

Part C: Implementation & Image Compression Lab (All 5 Versions)

Implement the SVD compression ladder in Python/NumPy (**without** calling `np.linalg.svd` inside your V2–V5 solvers; use only matrix-vector products and vector norms):

1. **Experiment 1 (V2 Gram Power vs. V3/V4 Matrix-Free Deflation Accuracy & Speed):** On rectangular matrices of size $m \times n = 2000 \times 400$, compare the wall-clock time to extract the top $k = 10$ singular triplets using V2 (forming $G = A^TA$ first) vs. V3+V4 (matrix-free alternating power + Hotelling deflation), and verify that the extracted singular values match `np.linalg.svd` to within 10^{-6}.
2. **Experiment 2 (Visual Image Compression & Eckart-Young Energy Curve):** Load or generate a 256×256 grayscale test image A. Reconstruct and display the compressed image A_k for $k \in \{1, 3, 10, 25, 60\}$ using your Stage 5 factored representation, and plot both (a) the singular value spectrum σ_i and (b) the empirical error $\|A - A_k\|_F$ vs. theoretical $\sqrt{\sum_{i>k}\sigma_i^2}$ and memory compression ratio.

PROBLEM 14: 3D RAY TRACING & BOUNDING VOLUME HIERARCHIES

Spatial Data Structures for 3D Meshes: From Triangle Soup to Möller–Trumbore & SAH-BVH Trees

Core Data Structures & Concepts: Indexed Face Sets, Barycentric Coordinates, Axis-Aligned Bounding Boxes (AABB), BVH Binary Trees

Algorithmic Progression: V1 (Triangle Soup Scan) → V2 (Indexed Mesh + Möller–Trumbore) → V3 (AABB Slab Test) → V4 (Median BVH Tree) → V5 (SAH-BVH + Ordered Pruning)

How Pixar, Video Games & Self-Driving LiDAR Cast Billions of 3D Rays per Second

How does a computer render a photorealistic 3D movie frame or simulate a self-driving car's LiDAR laser scanner? Every pixel on your screen shoots a mathematical **3D ray** $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ $(t > 0)$ starting from the camera eye $\mathbf{o} = (o_x, o_y, o_z)$ in direction $\mathbf{d} = (d_x, d_y, d_z)$. The 3D scene is modeled as a **surface mesh** built out of N flat 3D triangles. To color a pixel, we must solve the **Closest-Hit Ray Casting Query**:

Find the triangle $\triangle_i \in \{1, \ldots, N\}$ intersected by $\mathbf{r}(t)$ with the **smallest positive distance** $t > 0$.

Consider the scale of the challenge: a single 1920×1080 image shoots $R \approx 2{,}000{,}000$ rays. If a 3D character mesh contains $N = 100{,}000$ triangles, testing every ray against every triangle in a brute-force loop requires $2 \times 10^6 \times 10^5 = \mathbf{200{,}000{,}000{,}000}$ (200 billion) ray-triangle tests *for a single frame*!

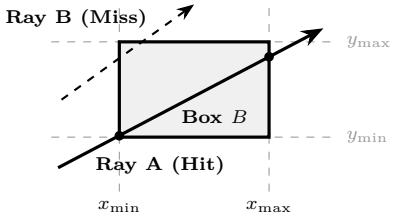
By upgrading both our **mesh storage data structure** and wrapping the triangles in a binary tree of **3D Bounding Boxes (a Bounding Volume Hierarchy, or BVH)**, we can skip 99.9% of the triangles on each ray—reducing the work per ray from $\Theta(N)$ down to $O(\log N)$ and turning hours of compute into milliseconds!

Visual Primer: Barycentric Coordinates, AABB Slabs & The BVH Spatial Tree

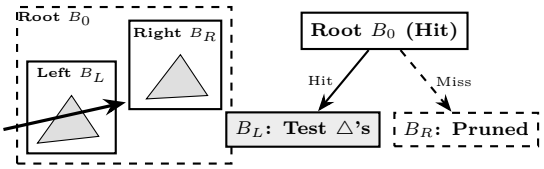
Three core geometric ideas power modern ray tracers:

1. **Barycentric Triangle Coordinates** (u, v)**:** Any point $\mathbf{p}$ on the plane of triangle $\triangle(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)$ can be written using two edge vectors $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$ and $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$ as $\mathbf{p}(u, v) = \mathbf{v}_0 + u\mathbf{e}_1 + v\mathbf{e}_2$. The point lies **inside** the triangle if and only if $u \geq 0$, $v \geq 0$, and $u + v \leq 1$.
2. **Axis-Aligned Bounding Box (AABB) Slab Test:** A 3D box $B = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ is the intersection of 3 axis-aligned "slabs." A ray hits the box if and only if its t-intervals inside the x-slab, y-slab, and z-slab **all overlap**: $\max(t_{0,x}, t_{0,y}, t_{0,z}) \leq \min(t_{1,x}, t_{1,y}, t_{1,z})$.
3. **Bounding Volume Hierarchy (BVH):** A binary tree where every node stores a 3D AABB enclosing all triangles in its subtree. If a ray misses a parent's box, we prune its entire subtree in $O(1)$ time!

1. Kay–Kajiya AABB Slab Test



2. BVH Binary Tree (Subtree Pruning)



The 5-Stage 3D Mesh Ray Tracing Evolution Ladder

- **Stage 1 (V1): Unindexed "Triangle Soup" Array + Brute-Force Plane/Edge Scan ($\Theta(N)$ per Ray)**
  - *Data Structure:* Store an array of N independent triangles, where each triangle stores three 3D points $(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)$ (9 floats = 36 bytes per triangle).
  - *Bottleneck:* Every vertex in a closed mesh is shared by ≈ 6 adjacent triangles, so Triangle Soup duplicates every vertex $6\times$ in memory! Worse, testing all N triangles per ray takes $\Theta(N)$ time.
- **Stage 2 (V2): Indexed Face Set Shared-Vertex Mesh + Möller–Trumbore Barycentric Solver**
  - *Data Structure Upgrade:* Store each unique 3D vertex once in `vertices[0..V-1]` ($3V$ floats) and store each triangle in `faces[0..F-1]` as a triple of integer indices (i_0, i_1, i_2). By Euler's formula ($F \approx 2V$), vertex coordinate memory drops by $6\times$!
  - *Algorithmic Upgrade (Möller–Trumbore, 1997):* Instead of computing plane equations and 3D normals, solve $\mathbf{o} + t\mathbf{d} = \mathbf{v}_0 + u\mathbf{e}_1 + v\mathbf{e}_2$ directly for (t, u, v) in a single 3×3 Cramer's Rule step using two cross products $\mathbf{P} = \mathbf{d} \times \mathbf{e}_2$ and $\mathbf{Q} = (\mathbf{o} - \mathbf{v}_0) \times \mathbf{e}_1$:

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\mathbf{P} \cdot \mathbf{e}_1} \begin{bmatrix} \mathbf{Q} \cdot \mathbf{e}_2 \\ \mathbf{P} \cdot (\mathbf{o} - \mathbf{v}_0) \\ \mathbf{Q} \cdot \mathbf{d} \end{bmatrix}.$$

- **Stage 3 (V3): Whole-Mesh Axis-Aligned Bounding Box (AABB) Slab Rejection Filter**
  - *Upgrade:* Precompute the tightest 3D box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ around the mesh. Before testing any triangles, run the 6-division Kay–Kajiya slab test. Rays that miss the object are rejected in $O(1)$ time, though rays that hit the box still scan N triangles.
- **Stage 4 (V4): Binary Bounding Volume Hierarchy (BVH) via Longest-Axis Median Splits**
  - *Data Structure Upgrade:* Recursively partition the N triangles into a balanced binary tree of AABBs: at each node, find the longest axis (x, y, or z) of its triangles' centroids, split the triangles at the **median centroid** into Left ($N/2$) and Right ($N/2$) children, and stop at leaves of ≤ 4 triangles. Average query time plunges from $\Theta(N)$ to $O(\log N)$!
- **Stage 5 (V5): Surface Area Heuristic (SAH) Binned BVH + Front-to-Back Ordered Traversal**
  - *SAH Construction Upgrade:* Equal-count $(N/2, N/2)$ median splits can create huge overlapping bounding boxes when a few large wall triangles sit next to a dense cluster of tiny detail triangles. Since the probability a random ray hits a box B_C inside B_P is proportional to its **3D surface area** $\mathrm{SA}(B) = 2(\Delta x\Delta y + \Delta y\Delta z + \Delta z\Delta x)$, Stage 5 chooses the split plane that minimizes the expected ray cost:

$$C_{\mathrm{SAH}}(L, R) = C_{\mathrm{trav}} + \frac{\mathrm{SA}(B_L)}{\mathrm{SA}(B_P)} N_L C_{\mathrm{isect}} + \frac{\mathrm{SA}(B_R)}{\mathrm{SA}(B_P)} N_R C_{\mathrm{isect}}.$$

  - *Ordered Traversal Upgrade:* When a ray hits *both* child boxes B_L and B_R, always visit the **closer child** (smaller t_{enter}) first and tighten t_{closest} on any triangle hit. If the updated $t_{\mathrm{closest}} < t_{\mathrm{enter,\ far}}$, we **skip the far child completely** without even opening it!

Part A: Mesh Memory Math, Möller–Trumbore Trace & AABB Slab Test

Q1. Why Indexed Meshes Save Vertex Storage (Stage 1 vs. Stage 2): In a standard closed 3D triangle mesh (like a sphere or character model) with V vertices and F triangular faces, there are almost exactly twice as many triangles as vertices ($F = 2V$). Suppose a character mesh has $V = 50{,}000$ unique vertices and $F = 100{,}000$ triangles, and each float or integer uses 4 bytes:

- Calculate how many floats (and Megabytes) **Stage 1 (Triangle Soup: $9F$ floats)** uses to store vertex coordinates versus **Stage 2 (Indexed Face Set: $3V$ floats)**. By what factor does Stage 2 compress vertex coordinate storage?
- Including the $3F$ integer indices in `faces` (4 bytes each), how many total bytes per triangle does Stage 2 use compared to Stage 1?

Q2. Hand-Worked Möller–Trumbore Ray-Triangle Intersection (Stage 2): Consider the triangle with vertices $\mathbf{v}_0 = (0, 0, 0)$, $\mathbf{v}_1 = (4, 0, 0)$, and $\mathbf{v}_2 = (0, 4, 0)$ lying in the $z = 0$ plane, and cast the ray starting at origin $\mathbf{o} = (1, 1, 3)$ with direction $\mathbf{d} = (0, 0, -1)$.

- Compute the edge vectors $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$, $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$, and translation vector $\mathbf{T} = \mathbf{o} - \mathbf{v}_0$ by hand.
- Compute the cross products $\mathbf{P} = \mathbf{d} \times \mathbf{e}_2$ and $\mathbf{Q} = \mathbf{T} \times \mathbf{e}_1$, the determinant $\det = \mathbf{P} \cdot \mathbf{e}_1$, and the solution (t, u, v) by hand. Verify that $t > 0$, $u \geq 0$, $v \geq 0$, and $u + v \leq 1$, and write down the 3D hit point $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$. What does $\det = \mathbf{P} \cdot \mathbf{e}_1 = 0$ mean geometrically?

Q3. Hand-Worked AABB Slab Overlap Test (Stage 3): Consider the 3D cube $B = [0, 2] \times [0, 2] \times [0, 2]$ and two rays both starting at $\mathbf{o} = (-1, 1, 1)$:

- **Ray A** has direction $\mathbf{d}_A = (1, 0.2, 0.2)$. Compute the entry/exit intervals $[t_{0,x}, t_{1,x}]$, $[t_{0,y}, t_{1,y}]$, and $[t_{0,z}, t_{1,z}]$, then find $t_{\text{enter}} = \max(0, t_{0,x}, t_{0,y}, t_{0,z})$ and $t_{\text{exit}} = \min(t_{1,x}, t_{1,y}, t_{1,z})$. Does Ray A hit B?
- **Ray B** has direction $\mathbf{d}_B = (1, 2, 0)$. Compute its per-axis intervals and $t_{\text{enter}}, t_{\text{exit}}$. Why does Ray B miss the box even though it passes through each of the three individual coordinate slabs at different times?

Part B: BVH Construction Recurrences, SAH Cost & Front-to-Back Pruning

Q4. Hand-Worked Median Split vs. Surface Area Heuristic (SAH) Split (Stage 4 vs. Stage 5): Suppose a parent BVH node has bounding box B_P with surface area $\text{SA}(B_P) = 100$ enclosing $N = 10$ triangles: 2 large background wall triangles on the left and 8 tiny detail triangles clustered tightly on the right. Assume traversal cost $C_{\text{trav}} = 1$ and triangle intersection cost $C_{\text{isect}} = 2$ (so leaving all 10 triangles in an unsplit leaf costs $10 \times 2 = 20$).

- **Stage 4 Candidate (Equal-Count Median Split, $N_L = 5, N_R = 5$):** Forcing 3 right-cluster triangles into the left child stretches B_L across the room to $\text{SA}(B_L) = 90$, while $\text{SA}(B_R) = 20$. Compute the expected SAH cost $C_{\text{SAH}}(\text{Median})$ by hand.
- **Stage 5 Candidate (SAH Spatial Split, $N_L = 2, N_R = 8$):** Separating the 2 large wall triangles ($\text{SA}(B_L) = 50$) cleanly from the tight cluster of 8 tiny triangles ($\text{SA}(B_R) = 10$) gives an unbalanced count split $(2, 8)$. Compute $C_{\text{SAH}}(\text{SAH})$ by hand. By what factor does Stage 5 reduce the expected ray cost compared to Stage 4 and the unsplit leaf?

Q5. Front-to-Back Ordered Traversal Pruning Trace (Stage 5): Suppose a ray enters a parent node and hits *both* child bounding boxes: the left child box B_L has interval $[t_{\text{enter},L}, t_{\text{exit},L}] = [2.0, 5.0]$ and the right child box B_R has interval $[t_{\text{enter},R}, t_{\text{exit},R}] = [6.0, 9.0]$.

- If we visit B_L first and hit a triangle inside B_L at distance $t = 4.2$ (updating $t_{\text{closest}} \leftarrow 4.2$), explain mathematically why we can immediately prune B_R without testing any triangles inside B_R.
- What would happen if B_L and B_R overlapped ($[2.0, 7.0]$ and $[3.5, 9.0]$) and the triangle hit inside B_L was at $t = 5.5$? Could we skip B_R in that case, or must we still check B_R? Why?

Q6. 5-Stage Complexity Comparison Table (Stages 1–5): Summarize **V1**–**V5** in a complexity table comparing: mesh memory per triangle, preprocessing time, extra acceleration structure memory, and query time for casting R rays against N triangles.

Part C: Implementation & 3D Mesh Ray-Tracing Benchmark (All 5 Versions)

Implement all five stages of the 3D ray-tracing engine in Python/NumPy:

1. **Experiment 1 (Differential Correctness Verification):** Generate a 3D triangulated mesh (e.g., an icosphere + ground plane + detailed small sphere cluster with $N = 1{,}000$ triangles) and cast 2,000 camera rays across the scene. Assert that V1, V2, V3, V4, and V5 return the **exact same hit triangle ID and distance** t (to within 10^{-6}) for every single ray!
2. **Experiment 2 (Intersection Counter & Scaling from $N = 200$ to $N = 20{,}000$ Triangles):** Instrument your code to count (a) AABB slab tests per ray and (b) Möller–Trumbore triangle tests per ray. Plot average tests per ray and wall-clock rendering time across V1–V5 as N grows, and render a 128×128

shaded diffuse image showing the speedup of SAH-BVH + Ordered Traversal (V5) over Median-BVH (V4) and Brute Force (V1/V2).

PROBLEM 15: THE TRAVELING SALESPERSON PROBLEM (TSP)

From Factorial Brute Force to Bitmask DP, Metric MST 2-Approximation & 2-Opt Simulated Annealing

Core Data Structures & Concepts: Distance Matrices, Subset Bitmask DP Tables, Minimum Spanning Trees, Tour Permutation Arrays

Algorithmic Progression: V1 ($\Theta(n!)$ Brute Force) → V2 (Nearest-Neighbor Greedy) → V3 (Held-Karp Bitmask DP) → V4 (Metric MST 2-Approx) → V5 (2-Opt + Simulated Annealing)

Why One Simple Question Stumped Mathematicians for Two Centuries

A delivery truck, a circuit-board laser drill, or a space telescope must visit n locations (numbered $0, 1, \ldots, n-1$), given an $n \times n$ symmetric distance table $D[i][j]$ between every pair of cities. The **Traveling Salesperson Problem (TSP)** asks a deceptively simple question: what is the shortest closed tour visiting every city exactly once and returning to 0?

$$\min_{\pi} \; L(\pi) = D[0][\pi_1] + D[\pi_1][\pi_2] + \cdots + D[\pi_{n-1}][0], \qquad \pi = (0 \to \pi_1 \to \pi_2 \to \cdots \to \pi_{n-1} \to 0).$$

Why is TSP the most famous **NP-hard** problem in computer science?

- If we fix city 0 as the start and ignore whether we drive clockwise or counter-clockwise, there are $\frac{(n-1)!}{2}$ distinct tours.
- At $n = 10$ cities, $\frac{9!}{2} = 181{,}440$ tours (a computer checks them in 0.05 seconds).
- At $n = 25$ cities, $\frac{24!}{2} \approx 3.1 \times 10^{23}$ tours—checking 10^9 tours per second would take 9.8 **million years**!

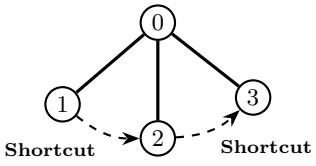
Because no polynomial-time exact algorithm is known for all n, engineers attack TSP with a 5-stage toolkit: first understanding why simple greedy choices get trapped, then compressing $\Theta(n!)$ down to $\Theta(n^2 2^n)$ via **Bitmask Dynamic Programming** for medium $n \leq 22$, and finally combining **Minimum Spanning Tree (MST) 2-Approximation** with **2-Opt Edge Uncrossing & Simulated Annealing** to solve 1,000-city problems in milliseconds!

Visual Primer: Bitmask Subset Encoding, MST Shortcutting & 2-Opt Uncrossing

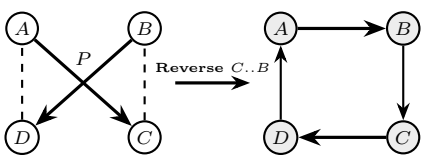
Three core algorithmic visuals guide our 5-stage TSP ladder:

1. **Subset Bitmasks (Stage 3):** Any subset $S \subseteq \{0, 1, \ldots, n-1\}$ can be stored as a single n-bit integer `mask` where bit i is 1 iff city $i \in S$ (e.g., for $n = 4$, $S = \{0, 1, 3\}$ is binary $1011_2 = 11$). Two paths that start at 0, visit the *exact same subset* S in different orders, and both end at city j can share a single DP state $C(S, j)$!
2. **MST Preorder Shortcutting (Stage 4):** Walking around a Minimum Spanning Tree visits every edge twice ($2 \cdot \text{cost(MST)}$). By skipping over already-visited cities directly to the next new city, the **Triangle Inequality** ($D_{ac} \leq D_{ab} + D_{bc}$) guarantees the shortcut tour never exceeds $2 \cdot \text{OPT}$!
3. **2-Opt Edge Swap (Stage 5):** Whenever two tour edges (A, C) and (B, D) cross in the plane, deleting them and reconnecting (A, B) and (C, D) (by reversing the sub-tour between C and B) uncrosses the tour and strictly reduces its length in $O(1)$ time!

1. Stage 4: MST Shortcutting



2. Stage 5: 2-Opt Uncrossing ($\Delta L < 0$)



The 5-Stage TSP Algorithmic Evolution Ladder

- **Stage 1 (V1): Exhaustive Permutation Enumeration ($\Theta(n!)$ Time, Exact)**
  - *Algorithm:* Fix city 0 and iterate over all $(n-1)!$ permutations of $\{1, \ldots, n-1\}$. Exact, but hits a computational wall around $n = 12$–14.
- **Stage 2 (V2): Nearest-Neighbor Greedy Construction ($\Theta(n^2)$ Time, Fast Heuristic)**
  - *Algorithm:* Start at a chosen city, and at each step jump to the **closest unvisited city** until all n cities are visited, then return to the start.
  - *Bottleneck:* Because Greedy never looks ahead, early short hops can leave isolated cities stranded far apart, forcing a terrible cross-map jump at the end of the tour!
- **Stage 3 (V3): Bellman–Held–Karp Bitmask Dynamic Programming ($\Theta(n^2 2^n)$ Time, Exact)**
  - *Overlapping Subproblems Insight:* To extend a partial path into the remaining unvisited cities, we do *not* need to know the exact permutation of how the past cities were visited—only **which subset** S **has been visited** (a bitmask `mask` $\in [1, 2^n - 1]$) and **which city** $j \in S$ **we are standing at right now**!
  - *Bellman Equation:* Let $C(S, j)$ be the minimum cost of a path starting at 0, visiting all cities in S, and ending at $j \in S \setminus \{0\}$. For $|S| = 2$, $C(\{0, j\}, j) = D[0][j]$. For $|S| \geq 3$:

$$C(S, j) = \min_{i \in S \setminus \{0, j\}} \Big[C(S \setminus \{j\},\, i) + D[i][j] \Big], \qquad \text{OPT} = \min_{j \in \{1..n-1\}} \Big[C(\{0..n-1\}, j) + D[j][0] \Big].$$

- **Stage 4 (V4): Metric MST Preorder Walk with Shortcutting ($O(n^2)$ Time, Provable 2-Approximation)**
  - *Algorithm:* Compute a Minimum Spanning Tree (MST) of the n cities (via Prim's or Kruskal's algorithm), perform a Depth-First Search (DFS) preorder walk of the tree vertices, and append the start vertex at the end. Whenever the Triangle Inequality holds, this tour is **guaranteed to cost at most** $2 \times$ **OPT** (and never worse)!
- **Stage 5 (V5): 2-Opt Local Search + Simulated Annealing ($\Theta(1)$ Delta per Swap)**
  - *2-Opt Swap:* Given any initial tour (from V2 or V4), repeatedly test pairs of non-adjacent tour edges $(u_1, v_1) = (\text{tour}[i], \text{tour}[i+1])$ and $(u_2, v_2) = (\text{tour}[j], \text{tour}[(j+1) \bmod n])$. Reversing the subarray `tour[i+1..j]` replaces (u_1, v_1) and (u_2, v_2) with (u_1, u_2) and (v_1, v_2), changing the total tour length in $O(1)$ time by:

$$\Delta L(i, j) = \big(D[u_1][u_2] + D[v_1][v_2]\big) - \big(D[u_1][v_1] + D[u_2][v_2]\big).$$

  - *Simulated Annealing Upgrade:* Pure greedy 2-Opt only accepts $\Delta L < 0$ and can get stuck in a local minimum. Simulated Annealing also accepts slightly worse moves ($\Delta L > 0$) with probability $P = \exp(-\Delta L / T)$ while slowly cooling the temperature $T_{k+1} = \gamma T_k$ ($\gamma \approx 0.995$) to escape local traps!

Part A: Hand-Worked Trace of All 5 Stages on a 4-City Instance

Consider the 4-city symmetric TSP instance on vertices $\{0, 1, 2, 3\}$ with distance matrix:

$$D = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}.$$

Q1. Stage 1 (Brute Force) & Stage 2 (Where Nearest-Neighbor Greedy Gets Trapped):

- List all $\frac{(4-1)!}{2} = 3$ distinct undirected closed tours starting at 0, compute the exact total length of each tour, and identify the optimal tour length OPT.

- Now trace **Nearest-Neighbor Greedy (Stage 2)** starting from **city** 3. Which city does Greedy visit first from 3? Which city does it visit second? Why is Greedy then *forced* to traverse the single worst edge in the entire graph ($D[1][2] = 35$), and what is the resulting tour and its total cost?

Q2. Stage 3 (Hand-Worked Held-Karp Bitmask DP Table): Represent each subset $S \subseteq \{0, 1, 2, 3\}$ (with $0 \in S$) both as a set and as a 4-bit binary integer $\texttt{mask} = b_3b_2b_1b_0$.

- Write down the 3 base-case values $C(S, j)$ for $|S| = 2$: $S = \{0, 1\}, \{0, 2\}, \{0, 3\}$.
- Compute all 6 DP values $C(S, j)$ for subsets of size $|S| = 3$: $S = \{0, 1, 2\}$, $\{0, 1, 3\}$, and $\{0, 2, 3\}$.
- Compute all 3 DP values $C(\{0, 1, 2, 3\}, j)$ for $j \in \{1, 2, 3\}$, and evaluate the final tour closure $\min_{j\in\{1,2,3\}}[C(\{0, 1, 2, 3\}, j) + D[j][0]]$ to confirm it recovers OPT = 80.

Q3. Stage 4 (MST Bound) & Stage 5 (Uncrossing Greedy with One 2-Opt Swap):

- Find the Minimum Spanning Tree (MST) of the 4-city distance matrix D by hand. What are its 3 edges and total weight cost(MST)? Verify that $\text{cost(MST)} \leq \text{OPT} \leq 2 \cdot \text{cost(MST)}$.
- Take the suboptimal Greedy tour tour $= [3, 0, 1, 2]$ (cycling $3 \to 0 \to 1 \to 2 \to 3$, cost 95) from Question 1. Apply a **single 2-Opt move** that deletes the two non-adjacent edges $(3, 0)$ and $(1, 2)$ and reconnects $(3, 1)$ and $(0, 2)$ (by reversing the subarray $[0, 1]$). Compute the $O(1)$ delta ΔL using the Stage 5 formula and verify that $95 + \Delta L = 80$!

Part B: Concrete Scaling Numbers & 5-Stage Tradeoff Table

Q4. Concrete Operation Counts: Brute Force vs. Held-Karp vs. Greedy (Stages 1–3): The number of tours or basic operations required by our first three stages on n cities is given by:

$$N_{\text{Brute}}(n) = \frac{(n-1)!}{2}, \qquad N_{\text{Held-Karp}}(n) = n^2 2^n, \qquad N_{\text{Greedy}}(n) = n^2.$$

- Plug $n = 10$ cities and $n = 20$ cities into all three formulas and present the results in a small table.
- By what numerical factor is **Stage 3 (Held-Karp Bitmask DP)** faster than **Stage 1 (Brute Force)** at $n = 20$ cities? Why does even Held-Karp become too slow around $n = 30$ cities ($2^{30} \approx 10^9$), motivating Stages 4 and 5?

Q5. 5-Stage Complexity & Tradeoff Comparison Table: Construct a comparison table across **V1–V5** comparing: worst-case time complexity, space complexity, whether it guarantees the exact optimal tour (or 2× bound), and maximum practical n solvable in under 1 second.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five TSP stages in Python:

1. **Experiment 1 (Exact Optimality & Approximation Ratios at Small/Medium n):** On 20 random 2D Euclidean instances of size $n = 11$ (and $n = 16$ for V2–V5), assert that V1 and V3 return the exact same optimal tour length OPT. Record the empirical approximation ratio cost/OPT and runtime of V2 (Greedy), V4 (MST Preorder), V5a (Greedy + 2-Opt), and V5b (2-Opt + Simulated Annealing).
2. **Experiment 2 (Visual Uncrossing of a 150-City Euclidean Tour):** Generate $n = 150$ random 2D points and plot the side-by-side 2D tour geometry and total length for: (a) Initial random tour, (b) Stage 2 Nearest-Neighbor Greedy, (c) Stage 4 MST Preorder Shortcut tour, and (d) Stage 5 2-Opt + Simulated Annealing tour (alongside the cost-vs.-iteration convergence curve).

PROBLEM 16: SUBSTRING SEARCH & PATTERN MATCHING

Exact String Matching: From Sliding Windows to Rolling Hashes & The KMP Automaton

Core Data Structures & Concepts: String Arrays, Modular Arithmetic Fingerprints, Prefix-Border Tables (π-Arrays), Hash Sets

Algorithmic Progression: V1 (Naive Sliding Window) → V2 (Additive Checksum) → V3 (Rabin-Karp Polynomial Rolling Hash) → V4 (Knuth-Morris-Pratt Automaton) → V5 (Multi-Pattern Rolling-Hash Set)

Why Fast String Matching Powers Search Engines, Security & Genomics

Every time you press `Ctrl+F` in a text editor, run `grep` in a terminal, scan network packets for malware signatures, check an essay for plagiarism, or align short DNA reads against the 3,000,000,000-base-pair human genome, your computer solves the **Exact String Matching Problem**:

> Given a long **text string** $T[0..n-1]$ of length n and a shorter **pattern string** $P[0..m-1]$ of length m (with $n \gg m$), find every starting index $i \in \{0, 1, \ldots, n-m\}$ where the text slice $T[i..i+m-1]$ matches $P[0..m-1]$ character for character.

At first glance, this looks effortless: just slide the pattern P across T one shift i at a time and compare the m characters. On random English text, that simple loop usually rejects a wrong shift on the very first character! However, on repetitive inputs—such as biological DNA sequences (`AAAA...A`) or adversarial network packets—the naive loop catastrophically degrades to $\Theta(nm)$ operations, repeatedly re-scanning the same text characters over and over.

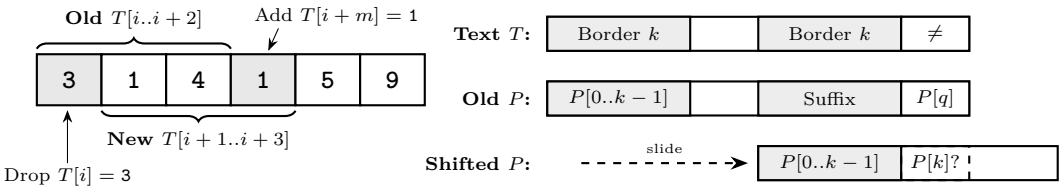
In this project, you will explore **two completely different algorithmic breakthroughs** that break the $\Theta(nm)$ barrier down to linear $\Theta(n+m)$ time:

1. **Algebraic Fingerprinting (Stages 2, 3 & 5 — Rabin-Karp):** Turn each m-character window into a compact numerical "fingerprint" (hash value) that updates in $O(1)$ time as the window slides by one character.
2. **Deterministic Prefix-Border Reuse (Stage 4 — Knuth-Morris-Pratt):** Precompute the internal self-overlaps of the pattern P so the text cursor **never backs up**, guaranteeing worst-case $\Theta(n+m)$ time.

Visual Primer: Why Naive Backing-Up Wastes Work vs. Rolling Fingerprints & Borders

Suppose we search for pattern $P =$ `AAAAB` ($m = 5$) inside text $T =$ `AAAAAAAAAB` ($n = 10$). At shift $i = 0$, the naive algorithm matches four `A`'s before failing on the fifth character ($P[4] =$ `B` $\neq T[4] =$ `A`). It then throws away everything it just learned, backs the text pointer all the way back to $i = 1$, and re-reads $T[1..4]$ from scratch!

1. $O(1)$ Rolling Hash (Stages 2–3) **2. KMP Self-Overlap Border $\pi[q]$ (Stage 4)**



The 5-Stage String Matching Evolution Ladder

Let us trace how five successive algorithmic designs solve pattern matching across text $T[0..n-1]$ and pattern $P[0..m-1]$:

- **Stage 1: Naive Character-by-Character Sliding Window**
  - *Mechanism:* For each alignment $i = 0, 1, \ldots, n-m$, compare $T[i+j]$ against $P[j]$ for $j = 0, 1, \ldots, m-1$, breaking out of the inner loop on the first mismatch.
  - *Bottleneck:* When $T = \texttt{A}^n$ and $P = \texttt{A}^{m-1}\texttt{B}$, every alignment performs m comparisons before failing on the last character, costing $(n-m+1)m = \Theta(nm)$ time.
- **Stage 2: Simple Additive Character-Sum Rolling Checksum**
  - *What Changed:* Map each character to its integer code (e.g., `'0'` $\mapsto 0, \ldots,$ `'9'` $\mapsto 9$ or ASCII code) and define the window checksum as the simple sum $S(T[i..i+m-1]) = \sum_{j=0}^{m-1} T[i+j]$. When shifting $i \to i+1$, update the sum in $O(1)$ time by subtracting the outgoing character and adding the incoming character:
    $$S_{\text{next}} = S_{\text{curr}} - T[i] + T[i+m].$$
    We only run the $O(m)$ character-by-character verification when $S(T[i..i+m-1]) = S(P)$.
  - *Bottleneck:* Addition is **commutative** ($2+6+5 = 5+6+2 = 4+4+5 = 13$). Every anagram or equal-sum window triggers a **spurious hit** (false-positive collision), forcing $\Theta(nm)$ slow verifications!
- **Stage 3: Rabin-Karp Polynomial Rolling Hash Modulo Prime q (Karp & Rabin, 1987)**
  - *What Changed:* Give each character position a **place value** in base B (where $B \geq |\Sigma|$, e.g., $B = 10$ for digits or $B = 256$ for ASCII) modulo a prime q:
    $$H(T[i..i+m-1]) = \left(\sum_{j=0}^{m-1} T[i+j] \cdot B^{m-1-j} \right) \bmod q.$$
    Precompute the highest place value $h = B^{m-1} \bmod q$ once in $O(m)$ time. Then shifting $i \to i+1$ takes $O(1)$ time via Horner's rule:
    $$H(T[i+1..i+m]) = \Big(B \cdot \big(H(T[i..i+m-1]) - T[i] \cdot h \big) + T[i+m] \Big) \bmod q.$$
  - *Advantage:* Order matters (`265` $\neq$ `562`)! For a large prime q, false collisions occur with probability $\approx 1/q$, yielding $\Theta(n+m)$ expected time.
- **Stage 4: Knuth-Morris-Pratt (KMP) Prefix-Border Automaton (1977)**
  - *What Changed:* Even Rabin-Karp has $\Theta(nm)$ worst-case time if an adversary crafts hash collisions. KMP eliminates hashing altogether by precomputing the **Prefix-Border Array** $\pi[0..m-1]$, where $\pi[q]$ is the length of the longest **proper prefix** of $P[0..q]$ that is also a **suffix** of $P[0..q]$:
    $$\pi[q] = \max \big\{ k \in \{0, 1, \ldots, q\} : P[0..k-1] = P[q-k+1..q] \big\}.$$
    During text scanning, maintain the current matched prefix length $k \in \{0, \ldots, m\}$. If $T[i]$ mismatches $P[k]$, immediately fall back to $k \leftarrow \pi[k-1]$ without ever decrementing i!
  - *Guarantee:* Strict $\Theta(n+m)$ worst-case time on **every** input string.
- **Stage 5: Multi-Pattern Rolling-Hash Set for Simultaneous K-Phrase Plagiarism Detection**
  - *What Changed:* Suppose you want to check whether a document T of length n contains **any** of $K =$ 10,000 suspect phrases $P^{(1)}, \ldots, P^{(K)}$, each of length m. Running KMP separately K times costs $\Theta(K(n+m))$. Instead, hash all K patterns into a **Hash Set** $\mathcal{H} = \{H(P^{(1)}), \ldots, H(P^{(K)})\}$ in $\Theta(Km)$ time, then roll a single Rabin-Karp window across T, checking $H(T[i..i+m-1]) \in \mathcal{H}$ in $O(1)$ expected time per character! Total expected time: $\Theta(n+Km)$.

Part A: Hand-Worked Traces of Additive Checksums, Rabin-Karp & KMP (Stages 1–4)

Q1. Trace Stage 2 (Additive Checksum) vs. Stage 3 (Rabin-Karp Modulo $q = 13$): Let the alphabet be decimal digits ($B = 10$), the pattern be $P =$ `265` ($m = 3$), and the text be the first 11 digits of π:

$$T[0..10] = \texttt{31415926535} \quad (n = 11, \text{ giving } n-m+1 = 9 \text{ alignments } i = 0, 1, \ldots, 8).$$

- First, compute the Stage 2 additive target sum $S(P) = 2 + 6 + 5 = 13$, and compute the Stage 3 Rabin-Karp target hash $H(P) = 265 \bmod 13$ and multiplier $h = 10^{3-1} \bmod 13 = 100 \bmod 13$.
- Construct a 9-row table for alignments $i = 0, 1, \ldots, 8$ showing: (a) the 3-digit window $T[i..i+2]$, (b) its additive checksum S_i obtained via $S_i = S_{i-1} - T[i-1] + T[i+2]$, and (c) its Rabin-Karp rolling hash H_i obtained via $H_i = \big(10(H_{i-1} - T[i-1] \cdot h) + T[i+2]\big) \bmod 13$.
- Identify every **spurious hit** (collision where the hash matches P but the substring is not `265`) and every **true hit** for both Stage 2 and Stage 3.

Q2. Trace Stage 4 (KMP Prefix-Border Table π & Automaton Scan): Consider the 10-character pattern $P[0..9] =$ `ABABACABAB`.

- Compute the prefix-border array $\pi[0..9]$ by hand. At index $q = 5$ ($P[5] =$ `C`, where the previous matched border had length $k = \pi[4] = 3$), write down explicitly how the fallback loop $k \leftarrow \pi[k-1]$ tests smaller borders before setting $\pi[5]$.
- Now trace the KMP search algorithm on the 12-character text $T[0..11] =$ `ABABABACABAB` using your π-table. For each text index $i = 0, 1, \ldots, 11$, record the character $T[i]$, the sequence of fallback states k visited (if any mismatch occurs), the updated matched length k at the end of step i, and the exact shift where the full match $k = 10$ is reported.

Part B: Repetitive DNA Comparison Count & Multi-Pattern Scaling

Q3. Counting Character Comparisons on a Repetitive DNA Trap (Stage 1 vs. Stage 4): Consider searching for the pattern $P =$ `AAAB` ($m = 4$) inside the repetitive text $T =$ `AAAAAAAAB` ($n = 9$, with 6 shifts $i = 0, 1, 2, 3, 4, 5$).

- For **Stage 1 (Naive Sliding Window)**, write down how many character comparisons are performed at each shift $i \in \{0, 1, 2, 3, 4, 5\}$ before either mismatching on `B` or succeeding at $i = 5$, and compute the total number of character comparisons across all 6 shifts.
- For **Stage 4 (KMP)**, write down the 4-element prefix-border table $\pi[0..3]$ for $P =$ `AAAB`. When $T[3] =$ `A` mismatches $P[3] =$ `B` at $i = 3$, what index $k \leftarrow \pi[2]$ does KMP fall back to, and why does this let KMP avoid re-checking $T[1..3]$?

Q4. Rabin-Karp Modular Arithmetic, Stage 5 Multi-Pattern Search & Summary Table:

- Why must we apply $\bmod q$ at each arithmetic step in Rabin-Karp rather than computing the full integer $\sum_{j=0}^{m-1} T[i+j]B^{m-1-j}$ directly when $m = 500$?
- Compare searching a document of length n for K patterns of length m using (a) K independent KMP scans versus (b) Stage 5 Multi-Pattern Rabin-Karp with a hash set of size K. Express the expected and worst-case time and space complexity of all 5 stages in a summary table.

Part C: Implementation & Empirical Benchmarking Lab

Implement all five versions in Python:

- `v1_naive_match(T, P)`: Standard character-by-character sliding window.
- `v2_additive_checksum(T, P)`: Rolling character sum with verification on checksum match.
- `v3_rabin_karp(T, P, B=256, q=1000000007)`: Polynomial rolling hash modulo prime q.
- `v4_kmp_match(T, P)`: Precomputing $\pi[0..m-1]$ and scanning T without backing up.
- `v5_multi_rabin_karp(T, pattern_list, m)`: Simultaneous K-pattern hash-set scanner.

Required Experiments & Plots:

1. **Correctness Verification:** Assert that `v1`, `v2`, `v3`, and `v4` return the exact same list of match indices on random strings, overlapping patterns ($P =$ `ABA` in $T =$ `ABABABA`), and boundary cases ($m = 1$, $m = n$, no matches).

2. **Experiment 1 (Natural Text vs. Adversarial Periodic DNA Trap):** Benchmark `v1`–`v4` on (a) natural English text of length $n = 200{,}000$ and (b) adversarial periodic strings $T =$ `'A'`^{100000} and $P =$ `'A'`^{500}`'B'`. Record both wall-clock runtime and the exact number of character comparisons, demonstrating the $500\times$ blowup of `v1` and `v2` on periodic/anagram inputs while `v3` and `v4` remain flat.

3. **Experiment 2 (Multi-Pattern Plagiarism Scaling):** For fixed document length $n = 100{,}000$ and phrase length $m = 32$, sweep the number of target patterns $K \in \{1, 10, 50, 200, 1000\}$ and plot the runtime of K calls to `v4_kmp_match` against a single pass of `v5_multi_rabin_karp`.

PROBLEM 17: MARKOV CHAIN MONTE CARLO (MCMC)

From Exhaustive Search & Hill-Climbing to Metropolis-Hastings: Ising Spins & Cipher Breaking

Core Data Structures & Concepts: 2D Spin Grids, Permutation Arrays, Bigram Transition Matrices, Pre-Aggregated Count Tables

Algorithmic Progression: V1 (Exhaustive / Uniform Guessing) → V2 (Unigram Frequency Matching) → V3 (Greedy Bigram Hill-Climbing) → V4 (Metropolis-Hastings MCMC) → V5 ($O(1)$ Local ΔE Lookup & 27×27 Count Matrix)

How Do You Search a Space Larger Than the Number of Atoms in the Universe?

Many of the most fascinating problems in science, cryptography, and artificial intelligence share a common structure: we have a gigantic set of possible configurations Ω (called the **state space**), and each state $s \in \Omega$ has an **energy** $E(s)$ (where lower energy means a better or more probable state) or a **Boltzmann probability**:

$$\pi(s) = \frac{1}{Z} e^{-E(s)/T}, \qquad \text{where } Z = \sum_{s' \in \Omega} e^{-E(s')/T}.$$

Consider two famous examples that you will build in this project:

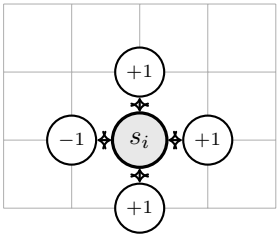
1. **Statistical Physics (The 2D Ising Ferromagnet):** A square $L \times L$ sheet of iron is modeled as a grid of $N = L^2$ microscopic atomic magnets ("spins") $s_i \in \{-1, +1\}$ pointing down (-1) or up $(+1)$. Neighboring spins prefer to align: the total energy is $E(s) = -J \sum_{\langle i,j \rangle} s_i s_j$. Even for a modest 100×100 grid, there are $|\Omega| = 2^{10,000} \approx 10^{3010}$ possible spin configurations!
2. **Cryptanalysis (Breaking Substitution Ciphers):** A secret message $Y = y_1 y_2 \dots y_N$ is encrypted by scrambling the 26 letters of the alphabet plus `SPACE` ($|\Sigma| = 27$) using a secret permutation key σ. There are $27! \approx 1.08 \times 10^{28}$ possible keys!

Because summing over $2^{10,000}$ or $27!$ states would take trillions of times longer than the age of the universe, the normalizing constant Z (called the **partition function**) is completely uncomputable. In 1953, Nicholas Metropolis, Arianna Rosenbluth, Marshall Rosenbluth, Augusta Teller, and Edward Teller discovered a breathtaking workaround: construct a random walk (**Markov Chain**) whose step-by-step acceptance rule depends only on the **ratio** of probabilities $\frac{\pi(s')}{\pi(s)} = e^{-(E(s')-E(s))/T}$—so the impossible constant Z **cancels out completely**!

Visual Primer: Local Energy Updates & Escaping Local Traps via Temperature T

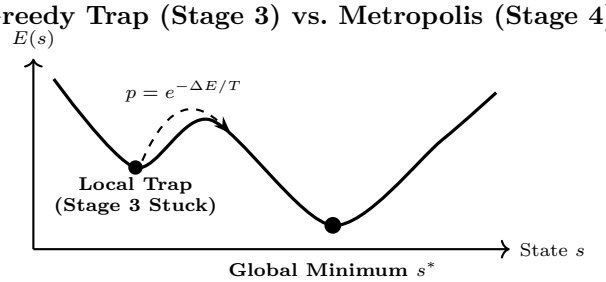
Why do simple greedy algorithms fail on complex energy landscapes, and why are local updates so fast?

1. Local Spin Flip on 2D Grid



Flipping $s_i \to -s_i$ only affects its **4 nearest bonds** ($O(1)$ work)

2. Greedy Trap (Stage 3) vs. Metropolis (Stage 4)



The 5-Stage MCMC & Data Structure Evolution Ladder

Observe how five stages of algorithmic and data-structure refinement transform an impossible search into a fast, million-step-per-second simulator and cipher-breaker:

- **Stage 1: Exhaustive Enumeration or Pure Uniform Random Guessing**
  - *Mechanism:* Enumerate all states $s \in \Omega$ (or draw purely random guesses $s \sim \text{Uniform}(\Omega)$) and track the lowest energy $E(s)$.
  - *Failure Mode:* With $|\Omega| = 2^N$ or 27!, the fraction of low-energy states is vanishingly tiny ($\approx 10^{-25}$), so random guessing never finds a coherent magnet or readable English decryption.
- **Stage 2: Single-Site / Unigram Frequency Matching (Independent Marginals)**
  - *What Changed:* For cipher breaking, count how often each letter appears in the ciphertext Y and match the most frequent ciphertext letter to `'E'`, the second most frequent to `'T'`, and so on, sorting two length-27 frequency arrays in $O(N + |\Sigma| \log |\Sigma|)$ time.
  - *Failure Mode:* Unigrams ignore **pairwise correlations** between adjacent characters (or adjacent spins)! In a short 300-character message, sampling noise scrambles the rank order of `E, T, A, O, I, N, S, H, R`, producing unreadable gibberish.
- **Stage 3: Greedy Pairwise Swap Hill-Climbing on Bigram Log-Likelihood**
  - *What Changed:* Capture neighbor interactions! For a candidate decryption key σ and reference English bigram transition matrix $M(a, b) = \Pr(\text{next char is } b \mid \text{current char is } a)$, define the **energy** $E(\sigma)$ as the negative log-likelihood:

$$\log \mathcal{L}(\sigma) = \sum_{t=1}^{N-1} \log M\big(\sigma(y_t),\, \sigma(y_{t+1})\big), \qquad E(\sigma) = -\log \mathcal{L}(\sigma).$$

    At each step, propose a small local move (flipping one spin s_i, or swapping two letters in σ) and accept the move **if and only if** $\Delta E = E(s') - E(s) \leq 0$ (strictly downhill).
  - *Failure Mode:* As shown in the diagram above, greedy hill-climbing immediately gets trapped in a **local minimum** (where a few common words are right, but fixing the remaining letters requires temporarily breaking a partially matched pair) and freezes forever!
- **Stage 4: Metropolis-Hastings MCMC with Temperature $T > 0$ (1953)**
  - *What Changed:* When a symmetric local proposal $s \to s'$ decreases energy ($\Delta E \leq 0$), always accept it ($A = 1$). When it **increases** energy ($\Delta E > 0$, an uphill move), accept it with probability:

$$A(s \to s') = \min\Big(1,\, e^{-\Delta E/T}\Big) = \min\left(1,\, \frac{\pi(s')}{\pi(s)}\right).$$

    Draw a uniform random number $u \sim \text{Uniform}(0, 1)$ and move to s' if $u < A(s \to s')$; otherwise stay at s.
  - *Breakthrough:* Thermal fluctuations ($T > 0$) allow the chain to climb over energy barriers out of local traps and provably sample the exact Boltzmann distribution $\pi(s)$!
- **Stage 5: $O(1)$ Local ΔE Lookup Table & Pre-Aggregated 27×27 Cipher Count Matrix**
  - *Bottleneck in Stage 4:* Recomputing the full lattice energy $E(s)$ from scratch costs $\Theta(L^2)$ per spin flip, and rescanning a ciphertext of length $N = 10{,}000$ costs $\Theta(N)$ per letter swap!
  - *Data Structure Upgrade 1 (Ising):* Only the 4 bonds touching spin s_i change when $s_i \to -s_i$, so $\Delta E_i = 2J s_i \sum_{j \in \mathcal{N}(i)} s_j$. Moreover, since $\sum_{j \in \mathcal{N}(i)} s_j \in \{-4, -2, 0, +2, +4\}$, precomputing a **5-entry table** of $e^{-\Delta E/T}$ reduces every Metropolis step to $O(1)$ integer arithmetic with zero `exp()` calls!
  - *Data Structure Upgrade 2 (Cipher):* Before running MCMC, scan the ciphertext Y **once** to build a 27×27 integer **Ciphertext Bigram Count Matrix** $C[a][b]$ = count of adjacent pair (a, b) in Y. Then $\log \mathcal{L}(\sigma) = \sum_{a,b} C[a][b] \log M[\sigma(a)][\sigma(b)]$, and swapping two letters in σ only updates two rows and two columns of C in $O(|\Sigma|)$ time—completely independent of text length N!

Part A: Hand-Worked Ising Spin Flips & $O(1)$ Lookup Table (Stages 3–5)

Consider a 2×2 periodic 2D Ising grid (a torus where the left/right edges wrap around and top/bottom edges wrap around) with 4 spins $s = (s_{00}, s_{01}, s_{10}, s_{11}) \in \{-1, +1\}^4$ and coupling $J = 1$. On an $L \times L$ periodic torus

where each spin i has 4 nearest neighbors $\mathcal{N}(i)$ (up, down, left, right), flipping a single spin $s_i \to -s_i$ only changes its 4 neighbor bonds, giving the $O(1)$ local energy change formula:

$$\Delta E_i = E(s') - E(s) = 2Js_iS_i, \qquad \text{where } S_i = \sum_{j \in \mathcal{N}(i)} s_j.$$

Q1. Building the 5-Entry Metropolis Lookup Table (Stage 5):

- Because each of the 4 neighbors $s_j \in \{-1, +1\}$ is either -1 or $+1$, the neighbor sum S_i can only take 5 possible even integer values: $S_i \in \{-4, -2, 0, +2, +4\}$.
- For $s_i = +1$, $J = 1$, and temperature $T = 2.0$, fill out a 5-row lookup table showing each possible value of S_i, the resulting energy change $\Delta E_i = 2s_iS_i$, the Boltzmann factor $\exp(-\Delta E_i/2.0)$, and the Metropolis acceptance probability $A = \min(1, \exp(-\Delta E_i/2.0))$. Explain why precomputing this tiny array before the simulation avoids calling `math.exp()` inside the million-step MCMC loop!

Q2. Hand-Worked Metropolis Step on a 2×2 Lattice: Suppose our 2×2 grid is currently in the "one-misaligned-spin" state:

$$s = \begin{pmatrix} s_{00} & s_{01} \\ s_{10} & s_{11} \end{pmatrix} = \begin{pmatrix} +1 & +1 \\ +1 & -1 \end{pmatrix}, \qquad J = 1, \quad T = 2.0,$$

where each spin has 4 periodic neighbors (two horizontal, two vertical; e.g., the 4 neighbors of s_{00} are s_{01} to the right, s_{01} to the left by wrap-around, s_{10} below, and s_{10} above by wrap-around, so $S_{00} = 2s_{01} + 2s_{10}$).

- Compute ΔE_{00} and the Metropolis acceptance probability $A_{00} = \min(1, e^{-\Delta E_{00}/2.0})$ if we propose flipping s_{00} from $+1 \to -1$. What would Stage 3 (Greedy Hill-Climbing) do here vs. Stage 4 (Metropolis)?
- Compute ΔE_{11} and the acceptance probability A_{11} if we propose flipping s_{11} from $-1 \to +1$ (creating the uniform all-+1 ground state).

Part B: Floating-Point Underflow & Pre-Aggregated Bigram Count Matrix

Q3. Floating-Point Underflow & Why We Use Log-Likelihoods (Stages 3–4):

- Suppose the average bigram transition probability in English is $M(a, b) \approx 1/27 \approx 0.037$. If a ciphertext has length $N = 1{,}000$, roughly how small is the raw likelihood product $\mathcal{L}(\sigma) = \prod_{t=1}^{999} M(\sigma(y_t), \sigma(y_{t+1}))$ expressed as a power of 10?
- Given that standard 64-bit floating-point numbers underflow to `0.0` below $\approx 10^{-308}$, explain why working with the sum of log-probabilities $\log \mathcal{L}(\sigma) = \sum_{t=1}^{N-1} \log M(\sigma(y_t), \sigma(y_{t+1}))$ is essential for cipher breaking.

Q4. Hand-Worked Bigram Count Matrix $C[a][b]$ & 5-Stage Summary Table (Stage 5):

- Consider a 3-letter toy alphabet $\Sigma = \{$`A, B, C`$\}$ and a 6-character ciphertext $Y =$ `ABABAC` (which contains $N - 1 = 5$ adjacent character pairs: `AB`, `BA`, `AB`, `BA`, `AC`). Write out the 3×3 integer bigram count matrix $C[a][b]$ by hand, verifying that its entries sum to 5.
- Explain in plain words why pre-counting the 27×27 matrix $C[a][b]$ once before the MCMC loop allows Stage 5 to score a key on a 50,000-character book in $O(|\Sigma|^2)$ (or $O(|\Sigma|)$) time per step instead of re-scanning all 50,000 characters on every swap!
- Complete a 5-stage summary table comparing **V1**–**V5** across: data structure used, time per move/iteration (for Ising grid of size L^2 and cipher of length N), and ability to escape local minima.

Part C: Implementation & Empirical Benchmarking Lab

Implement the complete evolutionary ladder across both domains:

1. **Experiment 1 (2D Ising Phase Transition & Stage 4 vs. Stage 5 Speedup):** Implement an $L \times L$ periodic Ising lattice ($L = 24$) comparing Stage 4 (naive ΔE or repeated `math.exp` calls) against Stage 5 ($O(1)$ local neighbor sum + 5-entry precomputed lookup table). Sweep temperature $T \in [1.2, 3.4]$ across Onsager's exact critical Curie temperature $T_c = \frac{2}{\ln(1+\sqrt{2})} \approx 2.269$. Plot the average absolute magnetization $|M(T)| = \frac{1}{L^2}\left|\sum_i s_i\right|$ vs. T and compare the wall-clock runtime per 10^6 Monte Carlo proposals between Stage 4 and Stage 5.

2. **Experiment 2 (Cracking a Substitution Cipher Across Stages 1–5):** Build a 27×27 log-bigram matrix $\log M[a][b]$ (with Laplace smoothing $+1$ so no entry is $\log 0$) from a reference English text, and encrypt a 400-character secret passage. Compare:

   - `v1_random_search`: Best of 10,000 uniform random keys,
   - `v2_unigram_match`: Sorting ciphertext vs. English single-letter frequencies,
   - `v3_greedy_hill_climb`: Accepting only swaps with $\Delta \log \mathcal{L} \geq 0$,
   - `v4_metropolis_scan`: Metropolis MCMC rescanning all N characters per step,
   - `v5_metropolis_count_matrix`: Metropolis MCMC using the 27×27 count matrix $C[a][b]$.

   Plot $\log \mathcal{L}(\sigma^{(t)})$ vs. iteration t and vs. wall-clock time, and print the decoded plaintext from each stage!

PROBLEM 18: STORAGE ENGINES: LRU, B-TREES & LSM-TREES

Database Storage Engines Under the Hood: From $O(1)$ LRU Caches to B-Trees & LSM-Trees

Core Data Structures & Concepts: Hash Tables + Doubly Linked Lists, High-Fanout Paged B-Trees, MemTables, SSTables & Bloom Filters

Algorithmic Progression: V1 (Unindexed Array Cache) → V2 ($O(1)$ Hash + DLL LRU Cache) → V3 (Pointer Binary Search Tree) → V4 (Paged High-Fanout B-Tree) → V5 (Log-Structured Merge Tree with Bloom Filters)

Why Real Databases Don't Use Simple Textbook Arrays or Binary Trees

When you write a Python dictionary or a binary search tree in an introductory programming class, all your data lives in fast Random Access Memory (RAM), where reading a single byte takes ≈ 50 nanoseconds (5×10^{-8} seconds). However, a production database (like PostgreSQL, MySQL, RocksDB, or Redis) must store terabytes of data across the **Memory Hierarchy Gap**:

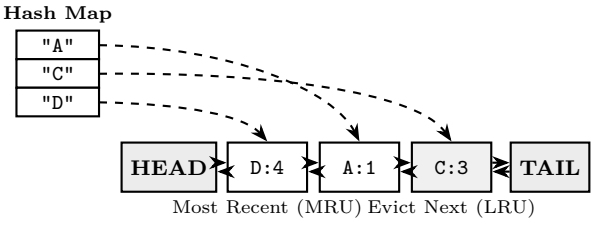
1. **RAM is fast but small and volatile:** Only the "hottest" (most recently used) fraction of data can sit in RAM at any moment. When the RAM buffer fills up, which item should we kick out (**evict**), and how do we do both lookups and evictions in **strict** $O(1)$ **time**?
2. **Disks and SSDs read and write in 4 KB–16 KB blocks (pages):** Fetching 1 byte from disk costs the exact same time ($\approx$ 100,000 nanoseconds) as fetching a full 16,384-byte page! If you store $N = 10^9$ keys in a balanced Binary Search Tree (BST), each key lookup chases $\log_2(10^9) \approx 30$ random pointers across disk—making queries 15× slower than necessary.
3. **Random disk writes are much slower than sequential appends:** Updating a B-Tree page in place for every tiny insert wears out SSD flash blocks and bottlenecks write-heavy workloads.

In this guided project, you will engineer the exact 5-stage sequence of composite data structures that power every modern operating system and database engine.

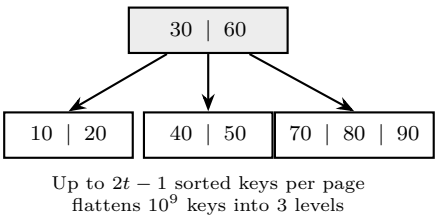
Visual Primer: Fusing Data Structures ($O(1)$ LRU Cache & High-Fanout B-Tree Pages)

Often in data structure design, **no single textbook structure** satisfies all our requirements, so we **fuse two structures together**:

1. Stage 2: Hash Map + DLL ($O(1)$ LRU Cache)



2. Stage 4: Paged B-Tree ($t = 2$)



The 5-Stage Storage & Caching Data Structure Ladder

- **Stage 1: Unindexed Array Cache or Hash-Only Cache ($O(C)$ Lookup or Eviction)**
  - *Mechanism:* Store up to C cached '(key, value, timestamp)' entries in a flat array (scanning $O(C)$ to find a key), *or* store them in a Hash Table (which gives $O(1)$ lookup, but requires scanning all C buckets in $\Theta(C)$ time whenever the cache is full to find the oldest timestamp to evict).

- **Stage 2: Strict $O(1)$ LRU Cache (Hash Table + Doubly Linked List with Sentinels)**
  - *Data Structure Upgrade:* Maintain a **Doubly Linked List (DLL)** ordered from Most Recently Used (immediately right of a dummy `HEAD` sentinel) to Least Recently Used (immediately left of a dummy `TAIL` sentinel), **and** a **Hash Map** mapping each 'key' directly to the memory address of its DLL 'Node'.
  - *Why Both Are Needed:* On `get(key)`, the Hash Map locates 'node' in $O(1)$ time, and because the list is *doubly* linked, we can splice 'node' out of the middle ('node.prev.next = node.next', 'node.next.prev = node.prev') and re-insert it right after `HEAD` in $O(1)$ pointer assignments! On `put(key, val)` when full, 'TAIL.prev' gives the exact LRU victim in $O(1)$ time!
- **Stage 3: Pointer-Based Binary Search Tree (BST) Index**
  - *Mechanism:* To support ordered keys and range queries ('SELECT * WHERE k BETWEEN 20 AND 80') that a hash table cannot answer, store keys in a balanced BST (1 key and 2 child pointers per node).
  - *Bottleneck:* Poor spatial locality! Each step down the tree follows a random memory/disk pointer, costing $\approx \log_2 N$ separate page fetches.
- **Stage 4: High-Fanout Paged B-Tree (Bayer & McCreight, 1972)**
  - *Data Structure Upgrade:* Parameterize each tree node by a **minimum degree** $t \geq 2$ chosen so one node fits inside one hardware disk page (or CPU cache line):
    1. Every node other than the root holds at least $t - 1$ sorted keys (and t child pointers if internal).
    2. Every node holds at most $2t - 1$ sorted keys (and $2t$ child pointers if internal).
    3. All leaf nodes sit at the **exact same depth** h.
  - *Proactive Top-Down Splitting:* When walking down the tree to insert a key k, whenever we encounter a **full node** containing $2t - 1$ keys, we immediately **split** it around its median key (index $t - 1$): the left $t - 1$ keys stay in the left node, the right $t - 1$ keys go into a new right node, and the median key is pushed up into the parent (which is guaranteed not to be full because we already checked it on the way down!).
- **Stage 5: Log-Structured Merge (LSM) Tree with Bloom Filters (O'Neil et al., 1996)**
  - *What Changed:* Even a B-Tree must read and overwrite a random 16 KB disk page on an insert. An LSM-Tree buffers all new writes in a fast in-memory sorted **MemTable** (e.g., a balanced tree or array of size M). When the MemTable fills up, it is flushed sequentially to disk as an immutable **Sorted String Table (SSTable)** run at Level L_0, and background **merge-sort compaction** cascades runs down levels $L_1, L_2, \ldots, L_L$ (each $T\times$ larger than the previous).
  - *Bloom Filter Acceleration:* Because a key could live in any of the L levels, a naive point lookup might binary-search L separate SSTable files. Attaching a compact in-memory **Bloom Filter** (with false-positive probability $\varepsilon \approx 1\%$) to each SSTable lets queries skip every run that does not contain the key in $O(1)$ RAM bit-checks!

Part A: Hand-Worked $O(1)$ LRU Cache & Top-Down B-Tree Splits (Stages 2 & 4)

Q1. Trace Stage 2 ($O(1)$ LRU Cache with Sentinels): Suppose an initially empty LRU cache has capacity $C = 3$ and uses dummy `HEAD` (MRU side) and `TAIL` (LRU side) sentinel nodes.

- Trace the exact sequence of 6 operations below by hand. After **each** operation, draw or write the Doubly Linked List order from `HEAD` to `TAIL`, state the contents of the Hash Map, and record the return value (for `get`) or evicted key (for `put`):

  `put(A, 1)`, `put(B, 2)`, `put(C, 3)`, `get(A)`, `put(D, 4)`, `get(B)`.

- Write the exact 2 lines of pointer assignments required to `_remove(node)` from a doubly linked list and the 4 lines required to `_insert_after_head(node)`. Why does having dummy `HEAD` and `TAIL` sentinel nodes eliminate all `if node.prev is None` null-check branches?

Q2. Trace Stage 4 (B-Tree Proactive Top-Down Insertion for $t = 2$): A B-Tree of minimum degree $t = 2$ (often called a **2-3-4 Tree**, where every node holds at least $t - 1 = 1$ key and at most $2t - 1 = 3$ keys) starts empty.

- Insert the following 11 keys in order by hand using **proactive top-down splitting** (split any 3-key node as soon as your search path touches it on the way down):

  `10, 20, 30, 40, 50, 60, 70, 80, 90, 25, 35`.

  Draw the complete B-Tree after each node split occurs and draw the final tree after `35` is inserted.

Part B: Concrete Hardware Sizing, LSM Bloom Filters & Summary Table (Stages 3–5)

Q3. Concrete Hardware Sizing (Stage 3 BST vs. Stage 4 B-Tree): A B-Tree of minimum degree $t \geq 2$ containing N keys has height bounded by:

$$h \leq \log_t \left(\frac{N+1}{2} \right).$$

Suppose your database indexes $N = 10^9$ (one billion) rows on an SSD with 16 KB (16,384-byte) pages. Each key is 8 bytes and each child/record pointer is 8 bytes (so packing $2t-1$ keys and $2t$ pointers into a page uses $\approx 16 \times (2t)$ bytes).

- Calculate the maximum branching factor $2t$ and minimum degree t that fit inside one 16,384-byte page.
- Using the height formula above, compare the worst-case height of a Binary Search Tree ($\approx \log_2(10^9)$) against the maximum height h of your B-Tree ($\leq \log_t((10^9+1)/2)$). If the root and depth-1 pages of the B-Tree are permanently pinned in your RAM LRU cache, how many actual SSD page reads are needed to find any key among 1,000,000,000 records?

Q4. Stage 5 LSM-Tree Bloom Filters & 5-Stage Summary Table: Consider a leveled LSM-Tree with $L = 4$ disk levels (L_1, L_2, L_3, L_4).

- Explain in plain words why sequential append/merge writing in an LSM-Tree is faster on disks and SSDs than overwriting random B-Tree pages in place on every insert.
- If a user queries a key that **does not exist** in the database, a naive LSM-Tree must search all $L = 4$ disk levels (4 disk reads). If each level has an in-memory Bloom Filter with false-positive rate $\varepsilon = 0.01$ (and zero false negatives), what is the expected number of wasted disk reads for a missing key?
- Complete a 5-stage summary table comparing **V1**–**V5** across: underlying data structure, lookup cost, insert/evict cost, and primary use case.

Part C: Implementation & Empirical Benchmarking Lab

Implement the storage-engine components in Python:

- `v1_array_lru(capacity)`: Flat list cache scanning $O(C)$ on every access.
- `v2_fast_lru(capacity)`: Strict $O(1)$ custom `Node` Doubly Linked List (with dummy `head`/`tail`) + Python `dict` (do not use `collections.OrderedDict`).
- `v3_bst_index`: Standard pointer-based Binary Search Tree.
- `v4_btree_index(t)`: Configurable minimum-degree t B-Tree supporting proactive top-down `insert`, `search`, and ordered range scan `range_query(low, high)`.
- `v5_lsm_engine(memtable_limit, bloom_bits)`: Simplified LSM-Tree with an in-memory sorted MemTable, immutable sorted runs (SSTable lists), and a bit-array Bloom Filter per run.

Required Experiments & Plots:

1. **Automated Invariant & Oracle Tests:** Run 20,000 random `get`/`put` operations comparing `v1_array_lru` and `v2_fast_lru`, and assert that your `v4_btree_index` maintains sorted order, uniform leaf depth, and $[t-1, 2t-1]$ node occupancy after inserting 50,000 keys.

2. **Plot 1 (LRU Scaling vs. Capacity C):** Fix 100,000 operations and sweep cache capacity $C \in \{32, 128, 512, 2048, 8192\}$, plotting throughput (ops/sec) of `v1_array_lru` ($O(C)$) vs. `v2_fast_lru` ($O(1)$ flat curve).

3. **Plot 2 (BST vs. High-Fanout B-Tree & LSM Lookup Speed):** On $N = 100{,}000$ keys, compare point-lookup time, tree height, and range-scan speed across `v3_bst_index`, `v4_btree_index` with $t \in \{2, 8, 32, 128\}$, and `v5_lsm_engine` (with and without Bloom Filters on missing-key queries).

PROBLEM 19: NETWORK FLOWS & IMAGE SEGMENTATION

From Greedy Path Pushing to Ford-Fulkerson, Edmonds-Karp & Graph-Cut Segmentation

Core Data Structures & Concepts: Adjacency Lists with Forward/Reverse Residual Edges, BFS Queues, Capacity Scaling, s-t Pixel Graphs

Algorithmic Progression: V1 (Forward-Only Greedy) → V2 (Ford-Fulkerson Residual DFS) → V3 (Edmonds-Karp Shortest-Path BFS) → V4 (Capacity-Scaling Δ-BFS) → V5 (Min-Cut Extraction & Image Segmentation)

From Cold War Railway Bottlenecks to Computer Vision Image Segmentation

Imagine managing a pipeline network, an internet routing grid, or a railway system modeled as a directed graph $G = (V, E)$ where each directed edge $(u \to v) \in E$ has a maximum **capacity** $c(u, v) \geq 0$. We want to ship as much material (or data) as possible from a **source node** s to a **sink node** t subject to two simple physical laws:

1. **Capacity Rule:** The flow $f(u, v)$ on every edge satisfies $0 \leq f(u, v) \leq c(u, v)$.
2. **Conservation Rule (Kirchhoff's Law):** For every intermediate node $v \notin \{s, t\}$, total flow entering v equals total flow leaving v:
$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w).$$

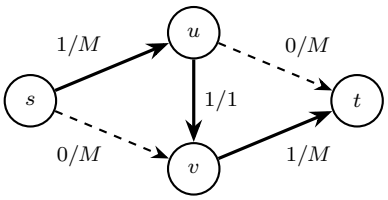
In 1956, Lester Ford and Delbert Fulkerson proved a remarkable duality theorem (**Max-Flow = Min-Cut**): the maximum total flow $|f^*|$ that can ever travel from s to t is **identically equal** to the minimum total capacity of edges that must be severed to disconnect s from t!

Even more surprisingly, computer vision researchers discovered that separating the **Foreground** from the **Background** of a noisy photograph is mathematically equivalent to finding a **Minimum s-t Cut** on a pixel grid graph. In this guided project, you will discover why naive greedy routing fails, build the **Residual Graph** data structure, see how Breadth-First Search (**Edmonds-Karp**) and **Capacity Scaling** defeat catastrophic 10^9-step traps, and segment noisy images cleanly!

Visual Primer: Why Forward-Only Greedy Gets Trapped & How Backward Edges Undo Mistakes

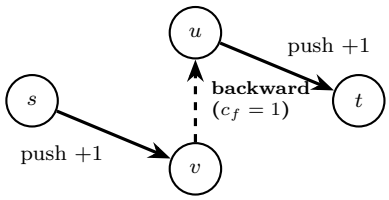
Consider the 4-node diamond network below where edges $(s \to u)$, $(u \to t)$, $(s \to v)$, $(v \to t)$ all have capacity M, and the middle bridge $(u \to v)$ has capacity 1. Clearly, the true maximum flow is $2M$ (send M along the top $s \to u \to t$ and M along the bottom $s \to v \to t$).

1. Stage 1 Trap: Greedy Path $s \to u \to v \to t$



If $M = 1$, forward capacity on (s, u) and (v, t) hits 0; forward-only greedy stops at flow $1 < 2$.

2. Stage 2 Fix: Backward Edge $(v \to u)$



Pushing 1 unit along $s \to v \to u \to t$ **cancels** the flow on (u, v) and achieves optimal flow 2.

The 5-Stage Network Flow & Graph-Cut Evolution Ladder

- **Stage 1: Naive Forward-Only Greedy Path Pushing**
  - *Mechanism:* Repeatedly find any path from s to t with remaining forward capacity $c(u, v) - f(u, v) > 0$ and add its bottleneck capacity $\Delta = \min_{(u,v) \in P}(c(u, v) - f(u, v))$ to $f(u, v)$.

– *Failure Mode:* Cannot undo an early greedy routing mistake across a cross-edge (as shown above for $M = 1$, stopping at flow 1 instead of 2).

- **Stage 2: Residual Graph with Backward Edges + Ford-Fulkerson DFS (1956)**
  – *Data Structure Upgrade:* For every directed edge $(u \to v)$ with capacity $c(u, v)$ and current flow $f(u, v)$, maintain **two linked edges** in the **Residual Graph** G_f:
    1. A **forward residual edge** $(u \to v)$ with remaining capacity $c_f(u, v) = c(u, v) - f(u, v)$ (how much *more* flow we can push forward).
    2. A **backward residual edge** $(v \to u)$ with residual capacity $c_f(v, u) = f(u, v)$ (how much existing forward flow we can *cancel/redirect* backward).

    Whenever we push Δ units along a residual edge $(a \to b)$, we simply update `c_f[a][b] -= Delta` and `c_f[b][a] += Delta` in $O(1)$ time!
  – *Bottleneck:* If we use Depth-First Search (DFS) to pick arbitrary augmenting paths on the diamond graph with $M = 10^9$, alternating $s \to u \to v \to t$ and $s \to v \to u \to t$ pushes only $\Delta = 1$ unit per step, taking $2M = 2{,}000{,}000{,}000$ iterations ($O(E \cdot |f^*|)$ pseudo-polynomial time)!

- **Stage 3: Edmonds-Karp Shortest-Augmenting-Path BFS (1972)**
  – *What Changed:* Replace DFS with **Breadth-First Search (BFS)** in G_f, always augmenting along a path from s to t with the **fewest number of edges** (unweighted shortest path).
  – *Guarantee:* Avoids long winding detours across cross-edges! Using BFS guarantees at most $O(VE)$ augmentations and strict $O(VE^2)$ polynomial time regardless of how huge the capacities M are.

- **Stage 4: Capacity-Scaling Shortest Augmenting Paths (Gabow, 1985)**
  – *What Changed:* Even with BFS, if a network has many tiny-capacity edges alongside huge-capacity highways, we want to prioritize pushing large chunks of flow first. Maintain a **scaling threshold** $\Delta = 2^{\lfloor \log_2 U \rfloor}$ (where $U = \max_e c(e)$ is the largest edge capacity). In the Δ-scaling phase, only traverse residual edges with $c_f(u, v) \geq \Delta$! When no such s-t path exists, halve $\Delta \leftarrow \Delta/2$ and repeat until $\Delta = 0$. Runs in $O(E^2 \log U)$ time and drastically cuts the number of augmentations in practice.

- **Stage 5: Minimum s-t Cut Extraction & Binary Image Segmentation**
  – *What Changed:* Once the residual graph G_f has no more s-t paths, run one final BFS from s following edges with $c_f(u, v) > 0$ to collect the reachable vertex set $S \subset V$, and set $T = V \setminus S$. By the Max-Flow Min-Cut Theorem, (S, T) is a **globally optimal Minimum s-t Cut**! Connecting s to every pixel p with foreground affinity $D_p(1)$, every pixel p to t with background affinity $D_p(0)$, and adjacent pixels (p, q) with smoothness penalty λ turns Min-Cut into an exact global image denoiser!

Part A: Hand-Worked Traces on the Diamond Trap & 6-Node Network (Stages 1–4)

Q1. Stage 2 Pathological DFS vs. Stage 3/4 on the $M = 10^9$ Diamond Network: Consider the 4-node diamond network in the Visual Primer with capacities $c(s, u) = 10^9$, $c(u, t) = 10^9$, $c(s, v) = 10^9$, $c(v, t) = 10^9$, and middle bridge $c(u, v) = 1$.

- Trace the first **2 iterations** of Stage 2 (Ford-Fulkerson DFS) if Iteration 1 picks path $P_1 = (s \to u \to v \to t)$ and Iteration 2 picks path $P_2 = (s \to v \to u \to t)$ using the backward edge $(v \to u)$. Write down the exact residual capacities c_f of all edges after Iteration 1 and after Iteration 2. How many total DFS iterations would this alternating choice take to reach the max flow 2×10^9?
- Now trace Stage 3 (**Edmonds-Karp BFS**) and Stage 4 (**Capacity Scaling** starting at $\Delta = 2^{\lfloor \log_2 10^9 \rfloor} = 2^{29} = 536{,}870{,}912$) on the exact same graph. How many augmenting paths does Edmonds-Karp take to finish? Why does neither Edmonds-Karp nor Capacity Scaling ever touch the $(u \to v)$ bridge?

Q2. Complete Hand Trace of Edmonds-Karp & Min-Cut Extraction: Consider the 6-vertex directed network on $V = \{s, u, v, w, x, t\}$ with 9 directed edges and capacities:

$$(s, u, 10),\ (s, v, 10),\ (u, v, 2),\ (u, w, 4),\ (u, x, 8),\ (v, x, 9),\ (x, w, 6),\ (w, t, 10),\ (x, t, 10).$$

- Run **Edmonds-Karp (Stage 3)** to completion by hand (breaking ties between equal-length BFS paths alphabetically, e.g., $s \to u \to w \to t$ before $s \to u \to x \to t$ before $s \to v \to x \to t$). At each augmentation step, record the shortest path P, its bottleneck $\Delta = c_f(P)$, and the cumulative flow $|f|$.
- From the final residual graph G_f, find the set $S \subset V$ of vertices reachable from s via edges with $c_f > 0$, list all original forward edges crossing from S to $T = V \setminus S$, and verify that their capacities sum to $|f^*|$.

Part B: Hand-Worked 2-Pixel Image Segmentation Min-Cut & 5-Stage Table (Stage 5)

Q3. Hand-Worked 2-Pixel Image Segmentation Min-Cut Trace (Stage 5): To segment two adjacent pixels $P = \{p_1, p_2\}$ into Foreground ($p_i \in S$) and Background ($p_i \in T$), we build a directed graph on $V = \{s, p_1, p_2, t\}$ with:

- Pixel p_1 (bright, likely Foreground): $c(s, p_1) = 9$ and $c(p_1, t) = 1$,
- Pixel p_2 (dark, likely Background): $c(s, p_2) = 1$ and $c(p_2, t) = 9$,
- Neighbor smoothness penalty between p_1 and p_2: $c(p_1, p_2) = \lambda$ and $c(p_2, p_1) = \lambda$.

Recall that the capacity $c(S, T)$ of an s-t cut (S, T) is the sum of capacities of all directed edges starting in S and ending in T.

- For a small smoothness penalty $\lambda = 2$, calculate the exact cut capacity $c(S, T)$ for all 4 possible s-t cuts: (i) Both Foreground: $S = \{s, p_1, p_2\}, T = \{t\}$; (ii) Both Background: $S = \{s\}, T = \{p_1, p_2, t\}$; (iii) p_1 Foreground, p_2 Background: $S = \{s, p_1\}, T = \{p_2, t\}$; (iv) p_1 Background, p_2 Foreground: $S = \{s, p_2\}, T = \{p_1, t\}$. Which cut achieves the minimum capacity?
- Repeat the calculation for a large smoothness penalty $\lambda = 10$. Which cut(s) are minimum now, and why does a large λ force neighboring pixels to take the same label?

Q4. 5-Stage Complexity & Guarantee Summary Table: Construct a comparison table across **V1–V5** comparing: path-selection method, worst-case number of augmentations, total worst-case time complexity, and whether it is guaranteed to find the true maximum flow.

Part C: Implementation & Empirical Benchmarking Lab

Implement all five stages in Python using an adjacency-list residual graph where each edge stores `(to, cap, rev_idx)`:

- `v1_greedy_forward`: Forward-only path pushing without backward edges.
- `v2_ford_fulkerson_dfs`: Residual graph with DFS path selection.
- `v3_edmonds_karp_bfs`: Residual graph with shortest-path BFS.
- `v4_capacity_scaling`: Residual graph with Δ-filtered BFS ($\Delta = 2^k, \ldots, 1$).
- `v5_graph_cut_segmenter(image, lambda_smooth)`: Builds the s-t pixel network, runs max-flow, and extracts the minimum cut S via residual reachability.

Required Experiments & Plots:

1. **Correctness & Trap Verification:** Run `v1`–`v4` on the 6-vertex graph from Part A (confirming `v2`, `v3`, `v4` all find $|f^*| = 19$ while `v1` can get stuck below 19) and on the diamond graph with $M \in \{10, 100, 1000, 10000\}$, plotting the number of augmentations of `v2` ($2M$) vs. `v3` and `v4` (2).
2. **Binary Image Denoising via Min-Cut:** Create a 24×24 binary icon (e.g., a circle or letter) and flip 20% of its pixels at random. Run `v5_graph_cut_segmenter` for smoothness penalties $\lambda \in \{0.0, 0.5, 1.5, 4.0\}$, plot the restored images side-by-side, and report pixel accuracy against the ground-truth clean image.

PROBLEM 20: ADVERSARIAL GAME TREES & MCTS

From 1-Ply Greedy & Minimax to Alpha-Beta Pruning, Zobrist Hashing & MCTS

Core Data Structures & Concepts: Game Trees, Alpha-Beta Search Windows, 64-Bit Zobrist XOR Transposition Tables, UCT Rollout Trees

Algorithmic Progression: V1 (1-Ply Greedy) → V2 (Depth-d Minimax) → V3 (Alpha-Beta Pruning) → V4 (Move Ordering + Zobrist XOR Hash Table) → V5 (Monte Carlo Tree Search with UCT)

How Computers Mastered Chess, Checkers, Connect Four & Go

In a two-player, turn-based, zero-sum board game of perfect information—such as Tic-Tac-Toe, Connect Four, Checkers, Othello, Chess, or Go—there is no luck or hidden cards. Player **MAX** (you) wants to maximize the final score, while Player **MIN** (your opponent) wants to minimize it.

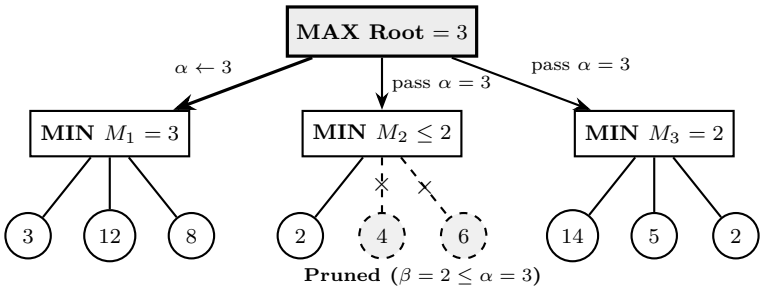
Why can't we just evaluate the board after each of our candidate moves and pick the move that looks best right now (**1-Ply Greedy**)? Because a greedy move might grab a free pawn while walking straight into a 2-move checkmate! To play intelligently, an AI must **look ahead** d turns ("plies") assuming that at every turn, **MIN will always choose the reply that is worst for MAX**.

However, looking ahead creates an exponential explosion: if a player has b legal moves per turn (the **branching factor**), a tree of depth d has b^d leaves! In Chess ($b \approx 35$), looking ahead just $d = 10$ plies requires $35^{10} \approx 2.7 \times 10^{15}$ board evaluations. In this final guided project, you will trace the 5-stage evolution of game-playing AI that enabled DeepBlue and AlphaGo: from 1-ply heuristics and recursive **Minimax** to **Alpha-Beta (α-β) Pruning**, $O(1)$ **Zobrist XOR Transposition Tables** (which cut the effective tree size from b^d down to $\approx b^{d/2}$, doubling the searchable depth!), and finally **Monte Carlo Tree Search (MCTS)**.

Visual Primer: Minimax Tree Evaluation & Alpha-Beta (α-β) Cutoff Windows

Consider a 2-ply game tree ($b = 3, d = 2$) where the root is a **MAX** node ($\triangle$) and its three children M_1, M_2, M_3 are **MIN** nodes ($\triangledown$). Look at what happens after we finish evaluating $M_1 = \min(3, 12, 8) = 3$:

- MAX at the root now knows: *"By choosing M_1, I am already guaranteed a score of at least $\alpha = 3$!"*
- Next, MAX inspects M_2's first leaf, which has value **2**. Because M_2 is a **MIN** node, M_2's final value can only be **less than or equal to** 2 ($\beta = 2 \leq \alpha = 3$) no matter what the remaining leaves (4 and 6) are! Therefore, MAX will *never* choose M_2 over M_1, and we can **prune (skip)** 4 and 6 immediately!



The 5-Stage Game-Playing AI Evolution Ladder

- **Stage 1: 1-Ply Greedy Static Evaluation**
  - *Mechanism:* Generate the b legal moves from the current board s, evaluate a fast heuristic function $\text{Eval}(s')$ on each resulting board, and pick $\arg\max_{s'} \text{Eval}(s')$.
  - *Failure Mode:* Zero opponent lookahead ($d = 1$); falls into immediate 2-ply tactical traps.

- **Stage 2: Depth-d Recursive Minimax Search (von Neumann, 1928; Shannon, 1950)**
  - *What Changed:* Recursively expand the game tree to depth d:

$$V(s,d) = \begin{cases} \text{Eval}(s) & \text{if } d = 0 \text{ or } s \text{ is terminal,} \\ \max_{s' \in \text{Moves}(s)} V(s', d-1) & \text{if it is MAX's turn at } s, \\ \min_{s' \in \text{Moves}(s)} V(s', d-1) & \text{if it is MIN's turn at } s. \end{cases}$$

  - *Bottleneck:* Examines **every** node in the tree, always costing $\Theta(b^d)$ leaf evaluations.
- **Stage 3: Alpha-Beta (α-β) Window Pruning**
  - *Data Structure Upgrade:* Pass two running bounds $[\alpha, \beta]$ down every recursive call (initialized to $[-\infty, +\infty]$ at the root):
    - α: the highest score **MAX** is already guaranteed along the current path to the root.
    - β: the lowest score **MIN** is already guaranteed along the current path to the root.

    At a MAX node, update $\alpha \leftarrow \max(\alpha, v)$; at a MIN node, update $\beta \leftarrow \min(\beta, v)$. The instant $\alpha \geq \beta$, **break out of the loop** (prune all remaining sibling moves)!
  - *Crucial Guarantee:* Alpha-Beta returns the **exact same optimal move and score as Minimax**, while pruning dead subtrees!
- **Stage 4: Move Ordering + Iterative Deepening + $O(1)$ Zobrist XOR Transposition Table**
  - *Two Synergistic Upgrades:*
    1. **Move Ordering:** Alpha-Beta prunes the most branches when the **best move is searched first**! By searching the best move from depth $d-1$ (**Iterative Deepening**) and center/capture moves first, leaf evaluations drop from b^d to $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1 = \Theta(b^{d/2})$.
    2. **Zobrist 64-Bit XOR Transposition Table (1970):** Different move orders often reach the **exact same board position** (e.g., playing Column 3 then Column 4 vs. Column 4 then Column 3). Pre-generate a table of random 64-bit integers `Z[cell][piece]`. The board's hash H is the bitwise XOR ($\oplus$) of `Z[c][p]` for all pieces on the board. Dropping or moving a piece updates H in $O(1)$ **time via a single XOR** ($H_{\text{new}} = H_{\text{old}} \oplus$ `Z[c][p]`), allowing a Hash Table to cache already-searched board states!
- **Stage 5: Monte Carlo Tree Search (MCTS) with UCT (Kocsis & Szepesvári, 2006)**
  - *What Changed:* In games like Go ($b \approx 250$), even $b^{d/2}$ is too large and writing a good static Eval(s) is hard. MCTS builds an asymmetric search tree over K iterations, where each iteration performs 4 steps: **(1) Selection** using the **UCT (Upper Confidence Bound for Trees)** formula:

$$\text{UCT}(v_i) = \underbrace{\frac{w_i}{n_i}}_{\text{Exploitation (win rate)}} + \underbrace{c\sqrt{\frac{\ln N}{n_i}}}_{\text{Exploration (uncertainty)}},$$

    (2) Expansion of a new child node, **(3) Simulation (Rollout)** of a fast random game to the end, and **(4) Backpropagation** updating visit counts $n_i \leftarrow n_i + 1$ and wins $w_i \leftarrow w_i + r$ up to the root!

Part A: Hand-Worked Minimax, Alpha-Beta Pruning & Move Ordering (Stages 2–4)

Q1. Trace Stage 2 (Minimax) & Stage 3 (Alpha-Beta) on the 2-Ply Ternary Tree: Consider the depth-2 ternary tree ($b = 3, d = 2$) from the Visual Primer with MAX at the root, MIN nodes M_1, M_2, M_3 at depth 1, and 9 terminal leaf values from left to right:

$$M_1 : [3,\ 12,\ 8], \qquad M_2 : [2,\ 4,\ 6], \qquad M_3 : [14,\ 5,\ 2].$$

- Write down the exact Minimax value of M_1, M_2, M_3, and the root, and state how many of the 9 leaves Stage 2 evaluates.
- Trace Stage 3 (Alpha-Beta Pruning) from left to right by hand. For each node visited, write the incoming $[\alpha, \beta]$ window and the updated $[\alpha, \beta]$ after each child returns. List the exact leaves that are **pruned** ($\alpha \geq \beta$). Why is no leaf pruned inside $M_3 : [14, 5, 2]$ when searched in left-to-right order?

Q2. Stage 4 Best-Case vs. Worst-Case Move Ordering: Suppose we reorder the children of the root and the leaves inside each MIN node before running Alpha-Beta:

- **Best-First Ordering:** Reorder the 3 children of each MIN node in *increasing* order (so MIN sees its smallest/best payoff first: $M_1 : [3, 8, 12]$, $M_2 : [2, 4, 6]$, $M_3 : [2, 5, 14]$) and put MAX's best child M_1 first. Trace Alpha-Beta by hand on this reordered tree! How many of the 9 leaves are evaluated now, and verify that it matches the theoretical minimum formula $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ for $b = 3, d = 2$.
- **Worst-First Ordering:** Write down an ordering of the leaves of M_1, M_2, M_3 where Alpha-Beta prunes **zero** leaves (evaluates all 9 leaves).

Part B: Searchable Depth, Zobrist XOR Trace & UCT Calculation (Stages 3–5)

Q3. Comparing Searchable Depth: Minimax vs. Ideal Alpha-Beta (Stages 2–4): In a uniform game tree of branching factor b and even depth d, Stage 2 (Minimax) evaluates $N_{\text{MM}}(b, d) = b^d$ leaves, whereas Stage 4 (Alpha-Beta with best-first move ordering) evaluates only:

$$N_{\alpha\beta}(b, d) = 2b^{d/2} - 1.$$

- Plug $b = 3, d = 2$ into $2b^{d/2} - 1$ and verify that it matches the exact number of leaves (5) you evaluated in Part A Question 2!
- Suppose your computer can evaluate at most 1,000,000 leaves per move in a game with branching factor $b = 10$. What maximum depth d can pure Minimax ($10^d \leq 10^6$) reach, versus ideal Alpha-Beta ($2 \cdot 10^{d/2} \leq 10^6$)?

Q4. Hand-Worked 4-Bit Zobrist XOR Hash Trace ($O(1)$ Incremental Updates): Recall that bitwise exclusive-OR ($\oplus$) outputs 1 when two bits differ and 0 when they are the same ($0 \oplus 0 = 0$, $1 \oplus 1 = 0$, $1 \oplus 0 = 1$), so XORing any bitstring with itself cancels out ($x \oplus x = 0$).

- Suppose our current board has 4-bit Zobrist hash $H_{\text{old}} = \texttt{1010}_2$, and a piece p moves from square s_1 (where $\texttt{Z}[s_1][p] = \texttt{0110}_2$) to an empty square s_2 (where $\texttt{Z}[s_2][p] = \texttt{0011}_2$). Compute the updated hash bit-by-bit by hand:
$$H_{\text{new}} = H_{\text{old}} \oplus \texttt{Z}[s_1][p] \oplus \texttt{Z}[s_2][p].$$
- Now **undo** the move by XORing H_{new} with $\texttt{Z}[s_1][p] \oplus \texttt{Z}[s_2][p]$ a second time, and verify bit-by-bit that it restores $H_{\text{old}} = \texttt{1010}_2$!

Q5. Hand-Worked UCT Selection Step in MCTS & 5-Stage Summary Table (Stage 5): Suppose the root node of an MCTS tree has been visited $N = 100$ times and has three legal moves A, B, C with current statistics:

$$\text{Move } A: \ (w_A = 36, \ n_A = 50), \quad \text{Move } B: \ (w_B = 24, \ n_B = 40), \quad \text{Move } C: \ (w_C = 4, \ n_C = 10).$$

- Using exploration constant $c = 1.0$ and $\ln(100) \approx 4.605$, compute the Exploitation term w_i/n_i, the Exploration term $\sqrt{\ln(100)/n_i}$, and the total UCT score for each of A, B, C by hand. Which move does UCT select for the 101-st simulation, and why does it pick a move that does *not* currently have the highest empirical win rate?
- Complete a 5-stage summary table comparing **V1**–**V5** across: number of leaves/rollouts evaluated, memory complexity, and main tactical strength or weakness.

Part C: Implementation & Empirical Benchmarking Lab

Implement a playable game engine (e.g., **Connect Four** on a 6×7 grid or **5×5 Gomoku / Othello**) and all five AI stages:

- `v1_greedy_1ply(board)`: Static heuristic evaluation at depth 1.

- `v2_minimax(board, depth)`: Full recursive Minimax counting leaf evaluations.
- `v3_alphabeta(board, depth)`: Alpha-Beta pruning with default left-to-right column order.
- `v4_alphabeta_zobrist(board, depth)`: Alpha-Beta with center-first move ordering, incremental 64-bit Zobrist XOR hashing, and a transposition table.
- `v5_mcts_uct(board, rollouts, c)`: Monte Carlo Tree Search with UCT selection and random playouts.

Required Experiments & Plots:

1. **Exact Equivalence Assertion:** Over 30 mid-game boards at depths $d \in \{2, 3, 4, 5\}$, assert that `v2_minimax`, `v3_alphabeta`, and `v4_alphabeta_zobrist` return the **exact same minimax value**.
2. **Plot 1 (Exponential Node Reduction Across Stages 2–4):** Plot the number of evaluated nodes (on a log scale) and wall-clock time for $d \in \{1, 2, 3, 4, 5, 6, 7\}$ comparing `v2`, `v3`, and `v4` against the theoretical curves b^d and $2b^{d/2} - 1$.
3. **Plot 2 (AI Round-Robin Tournament):** Run a 40-game head-to-head tournament (swapping first/second player every game) pitting `v1_greedy_1ply`, `v4_alphabeta_zobrist(d=5)`, and `v5_mcts_uct` (with $100, 500, 2000$ rollouts) against each other, reporting win/draw/loss rates!